



Khoa Công nghệ thông tin - Học viện Kỹ thuật quân sự
Trụ sở: Tầng 19, nhà S1, 236 Hoàng Quốc Việt, Cầu Giấy, Hà Nội
Tel: (+84) 069515329 - Email: fit@mta.edu.vn
Website: <https://fit.mta.edu.vn>

<VOCABOOST – HỆ THỐNG HỌC TỪ VỰNG TOEIC THÔNG MINH>

THIẾT KẾ CHI TIẾT

Mã dự án	<2026_VocaBoost_MTA>
Phiên bản	V1.0.0
Ngày	20/06/2026

<HÀ NỘI, 6-2026>

NỘI DUNG SỬA ĐỔI

*M- Mới S – Sửa X - Xóa

[illegible]

TRANG KÝ

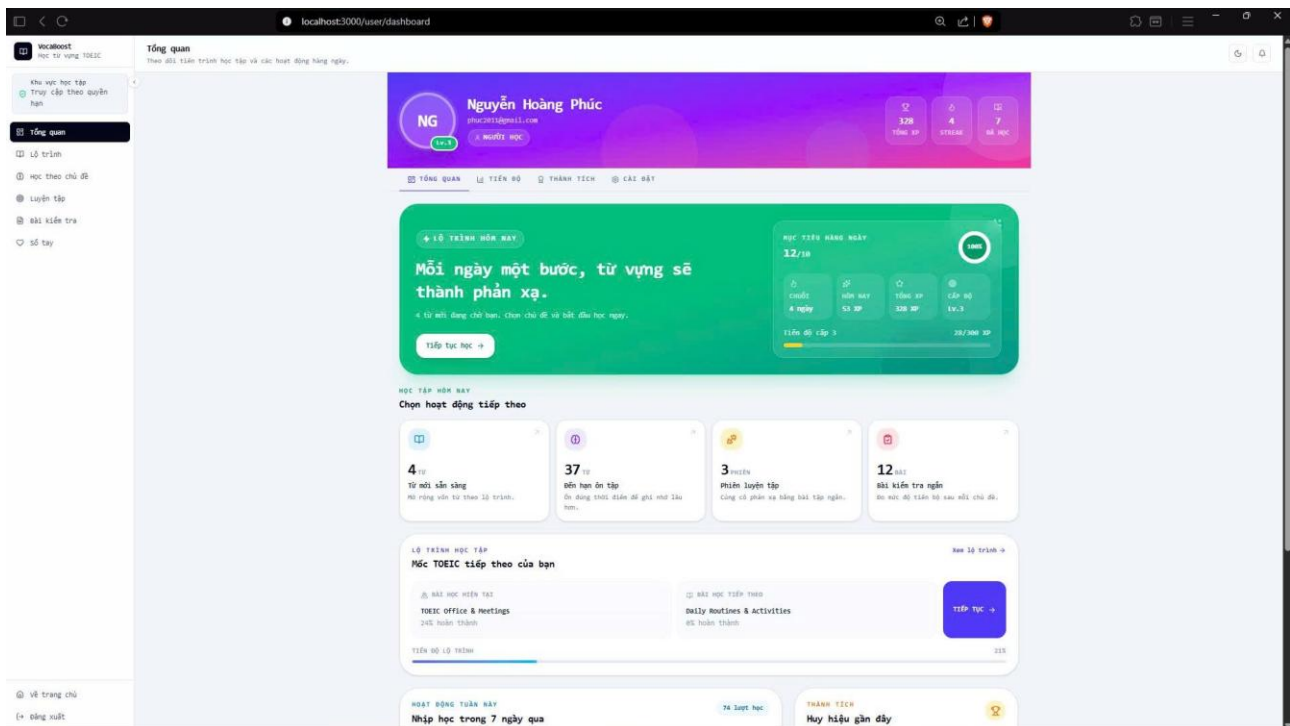
NGƯỜI LẬP: Nguyễn Hoàng Phúc _____ <Ngày> _____
<Vị trí: Backend, Architecture>

NGƯỜI KIỂM TRA: Lại Đức Tùng_____ <Ngày> _____
<Vị trí: Team Lead>

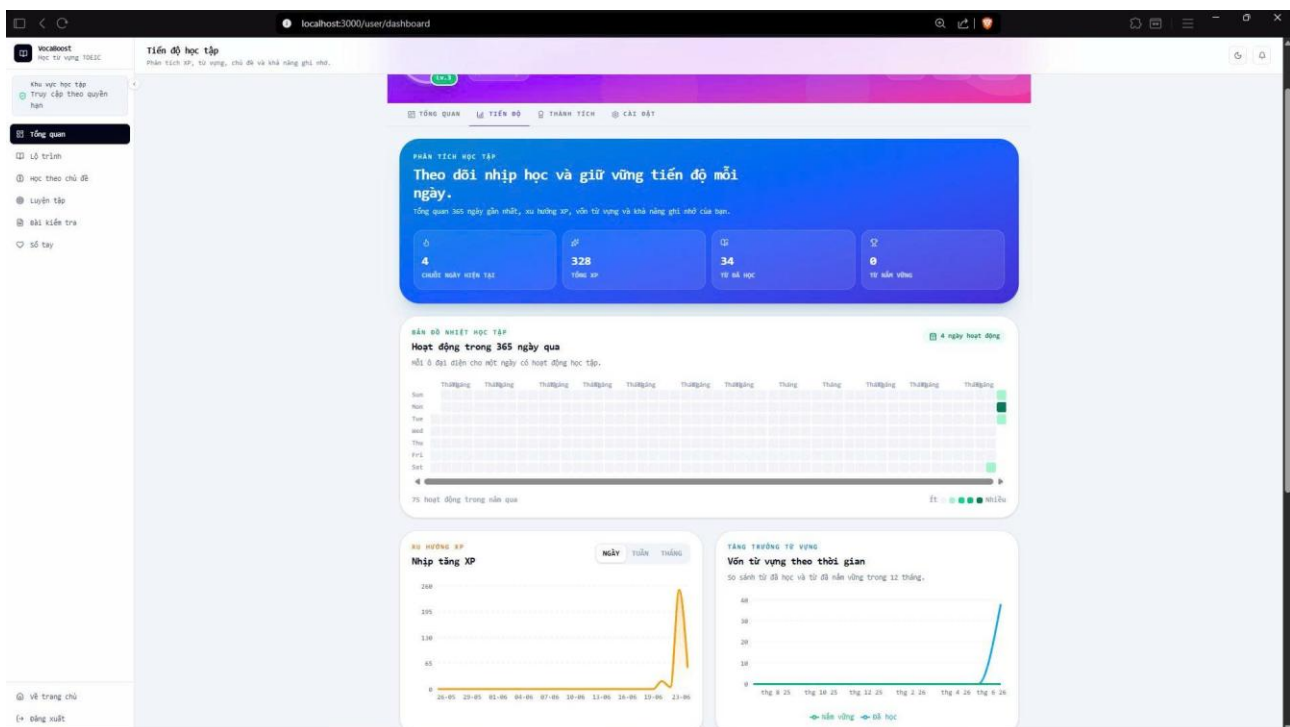
NGƯỜI PHÊ DUYỆT: Trần Tuấn Dũng _____ <Ngày> _____
<Vị trí: Frontend>

MỤC LỤC

1. GIỚI THIỆU	15
1.1. Mục đích	15
1.2. Phạm vi.....	15
1.3. Vai trò và trách nhiệm.....	15
1.4. Tài liệu tham khảo	15
1.5. Từ và thuật ngữ	15
2. TỔNG QUAN HỆ THỐNG	17
2.1. Tổng quan.....	17
2.2. Các yêu cầu chức năng.....	17
3. KIẾN TRÚC HỆ THỐNG	18
3.1. Mô hình kiến trúc	18
4. THIẾT KẾ LỚP.....	24
4.1. Class Diagram	24
1. Module Auth - Xác thực và phân quyền.....	24
2. Module Categories - Danh mục và chủ đề học	25
3. Module User Learning - Học từ vựng, SRS, mini-test, notebook.....	26
4. Module Learning Path & Progress.....	28
5. Module Gamification - XP, Level, Achievement	29
6. Module Admin - Quản trị hệ thống và nội dung	30
7. Module Creator - Biên tập nội dung học	32
8. Module Review - Kiểm duyệt nội dung	33
9. Module AI - Hỗ trợ soạn nội dung TOEIC	34
10. Module Report & Notification - Báo cáo và thông báo	35
11. Module System/Common - Hạ tầng backend	36
5. THIẾT KẾ DỮ LIỆU	38
6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG.....	39
6.1 Giao diện đăng nhập, người dùng	39
Hình 6.1.1: Giao diện đăng nhập, đăng kí hệ thống	40



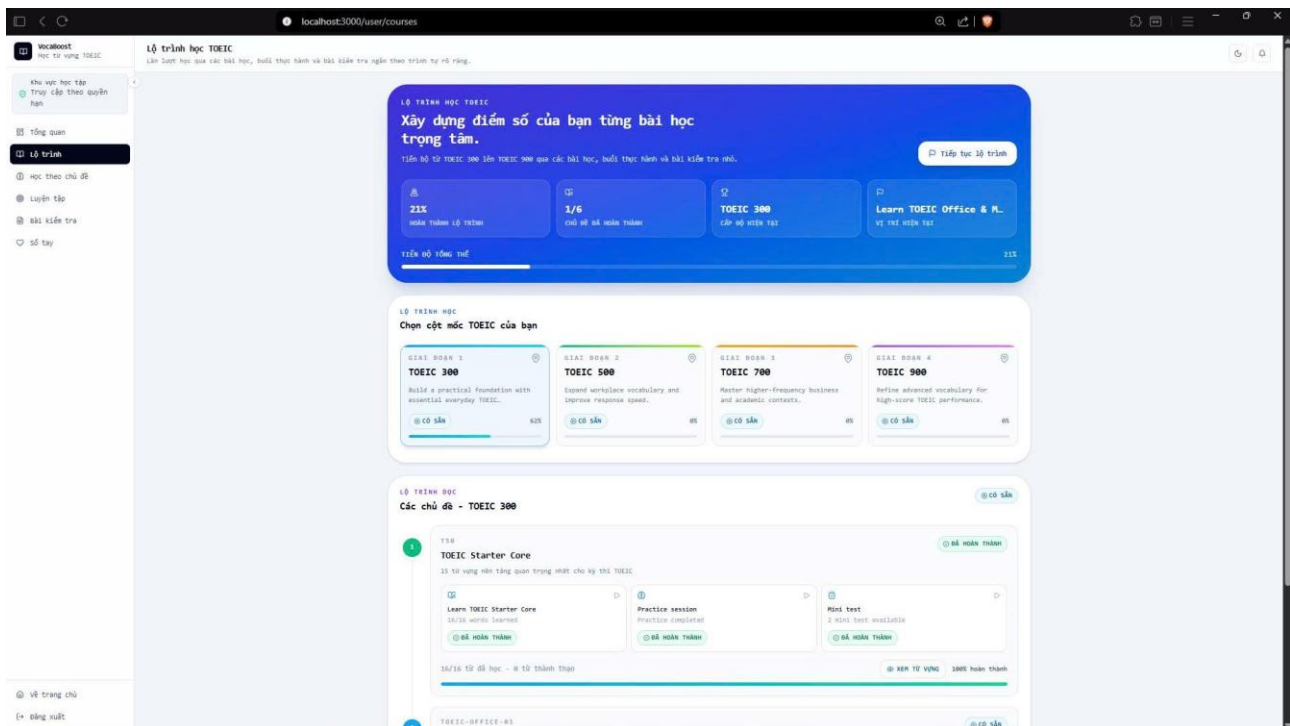
.....40



.....41

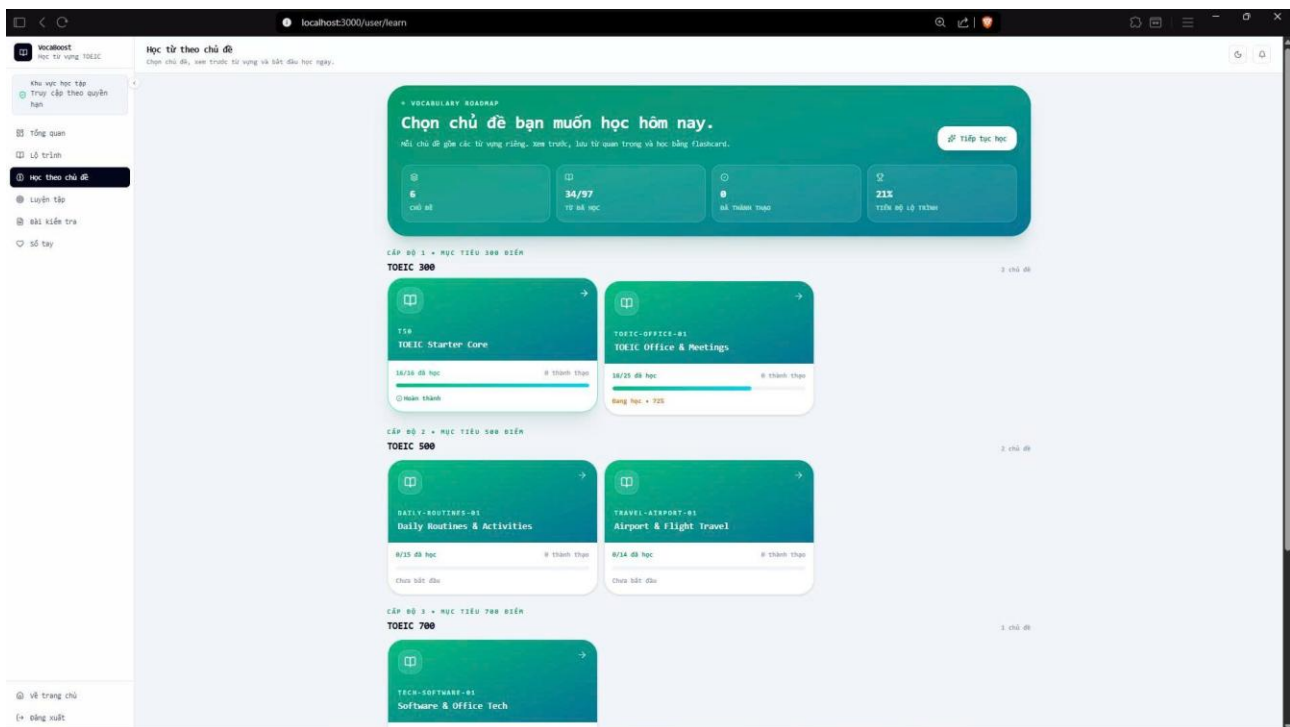
Hình 6.1.2: Giao diện bảng điều khiển học tập với mục tiêu ngày, XP, chuỗi học, hoạt động và lộ trình tiếp theo.

.....41



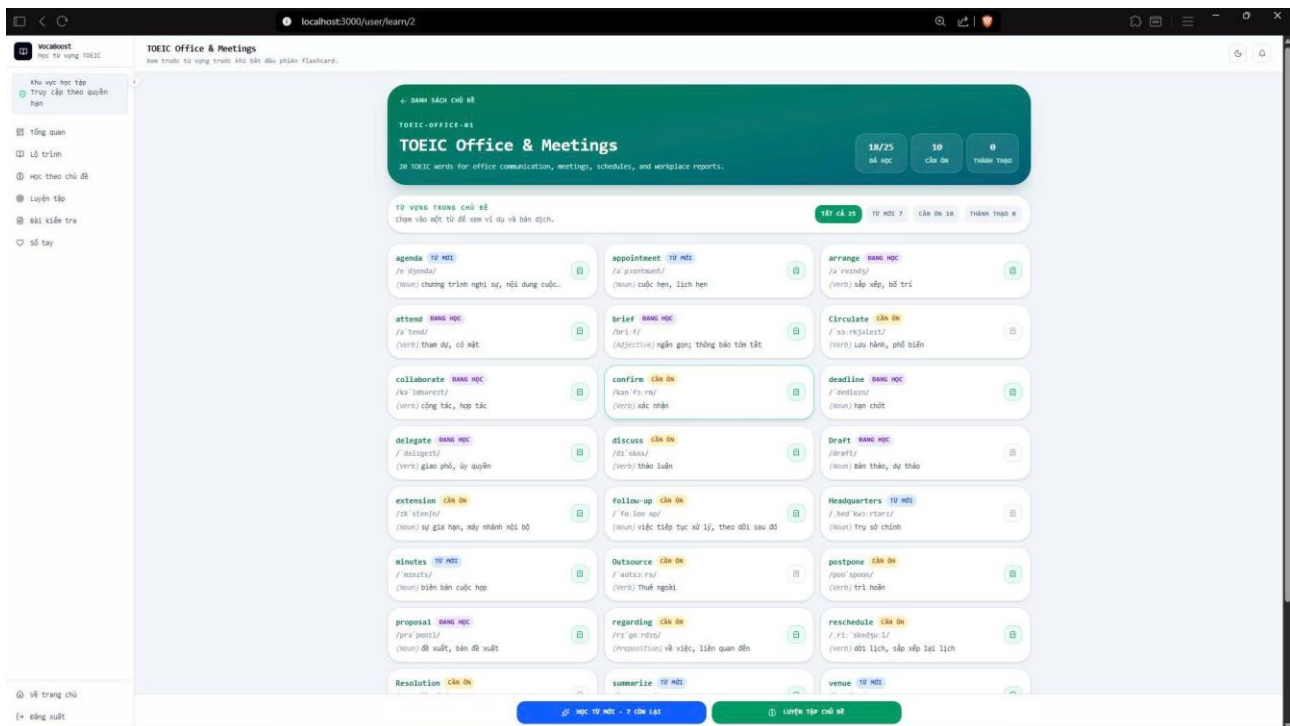
..... 41

Hình 6.1.3: Giao diện danh sách lộ trình/chủ đề giúp học viên chọn nội dung theo thứ tự học phù hợp. 41



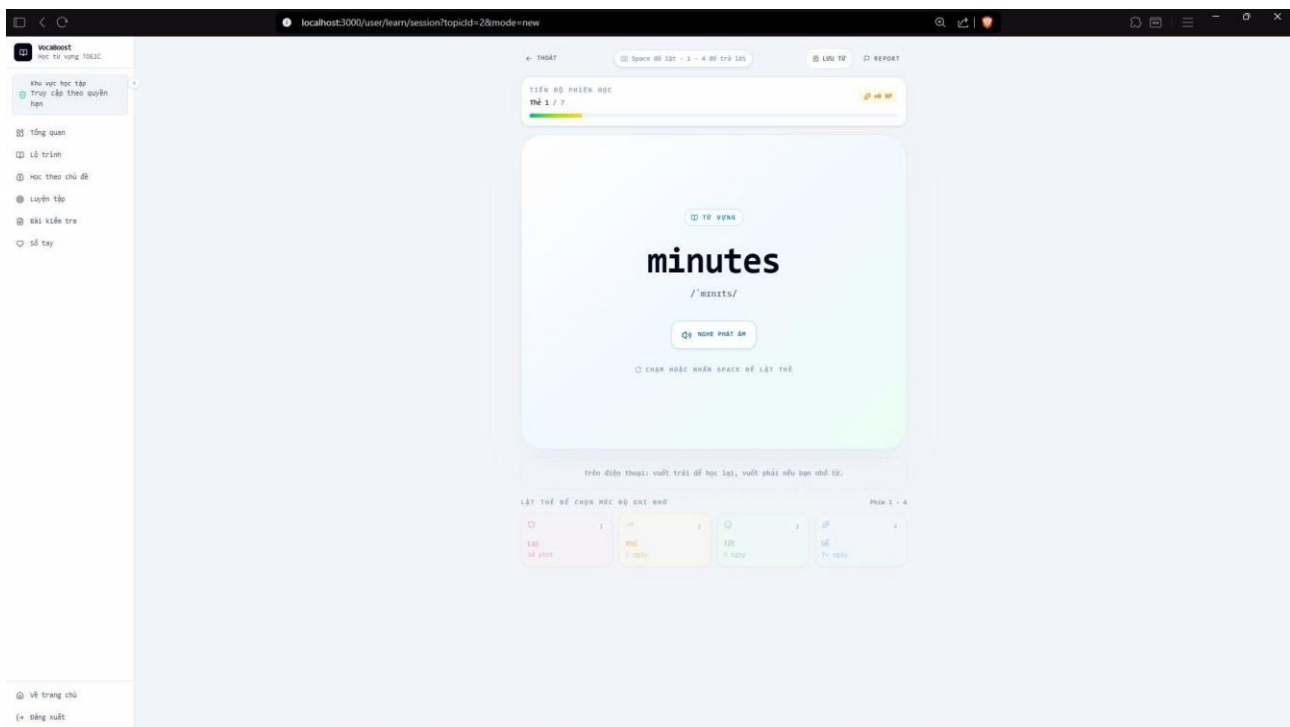
..... 42

Hình 6.1.4: Khu vực chọn chủ đề và bắt đầu phiên học từ vựng bằng flashcard..... 42

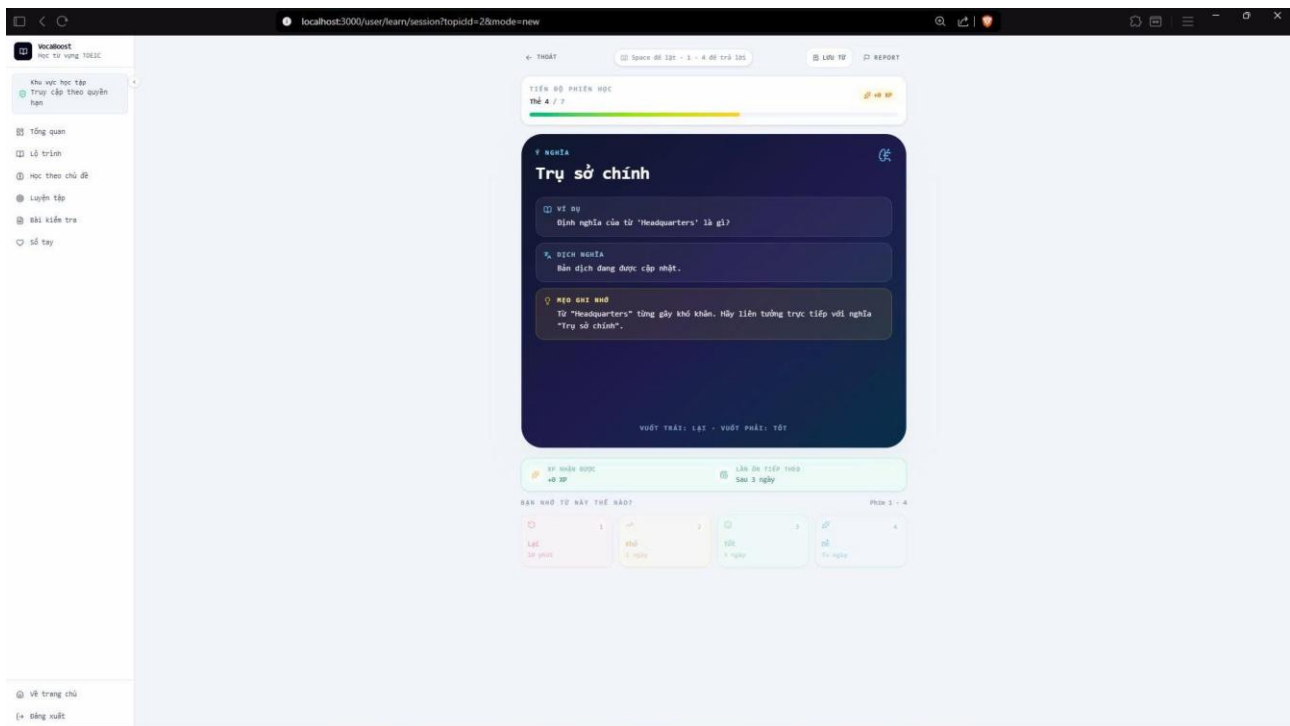


.....42

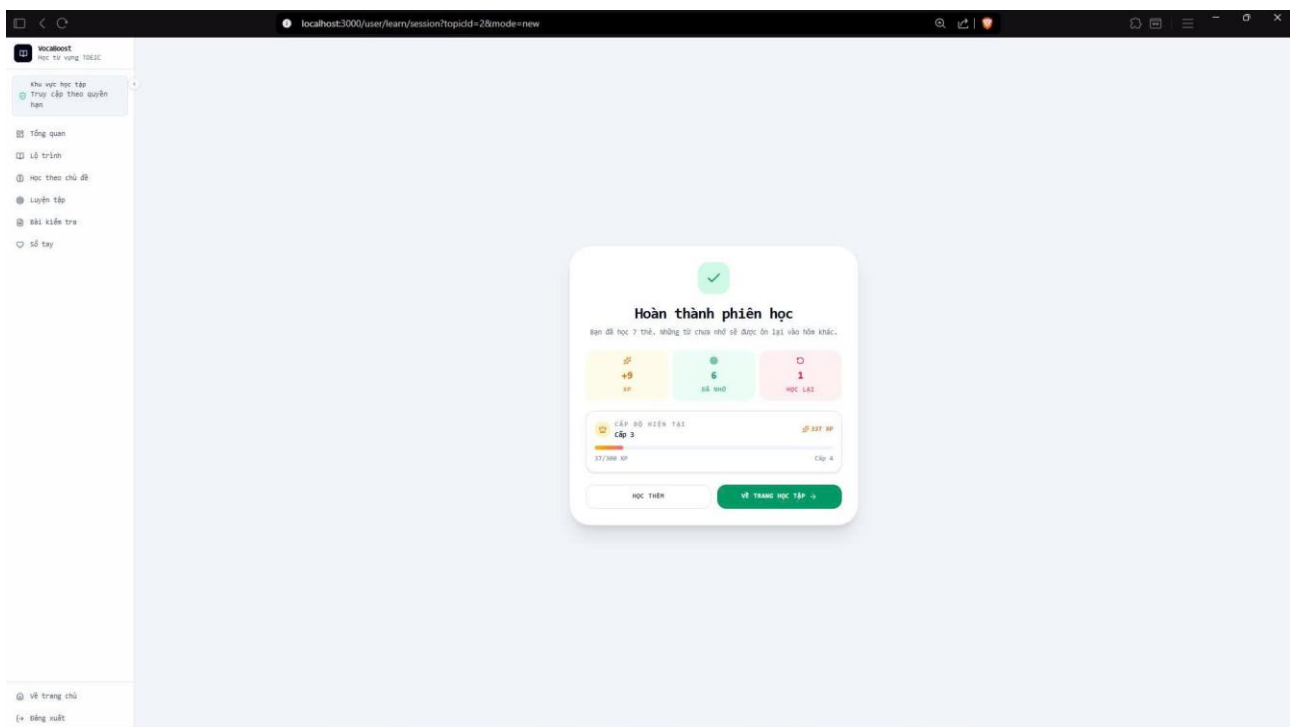
Hình 6.1.5: Giao diện xem các từ vựng trong chủ đề42



.....43

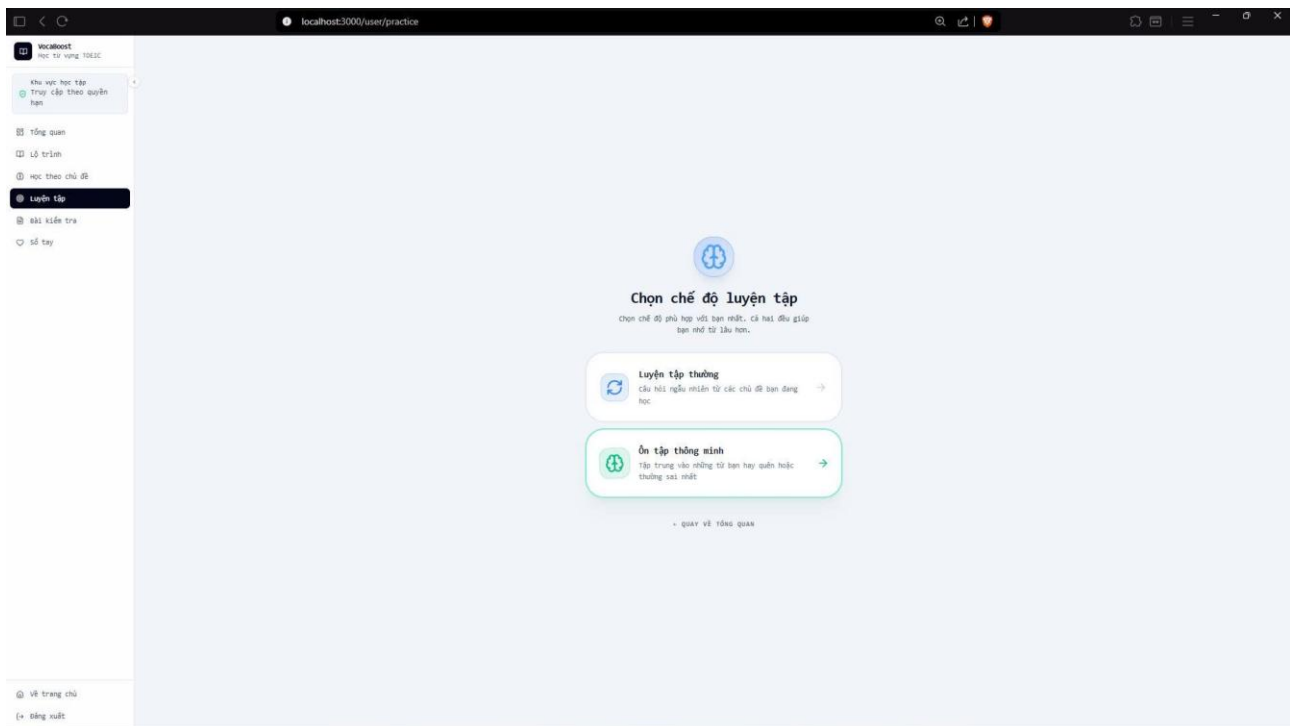


.....43

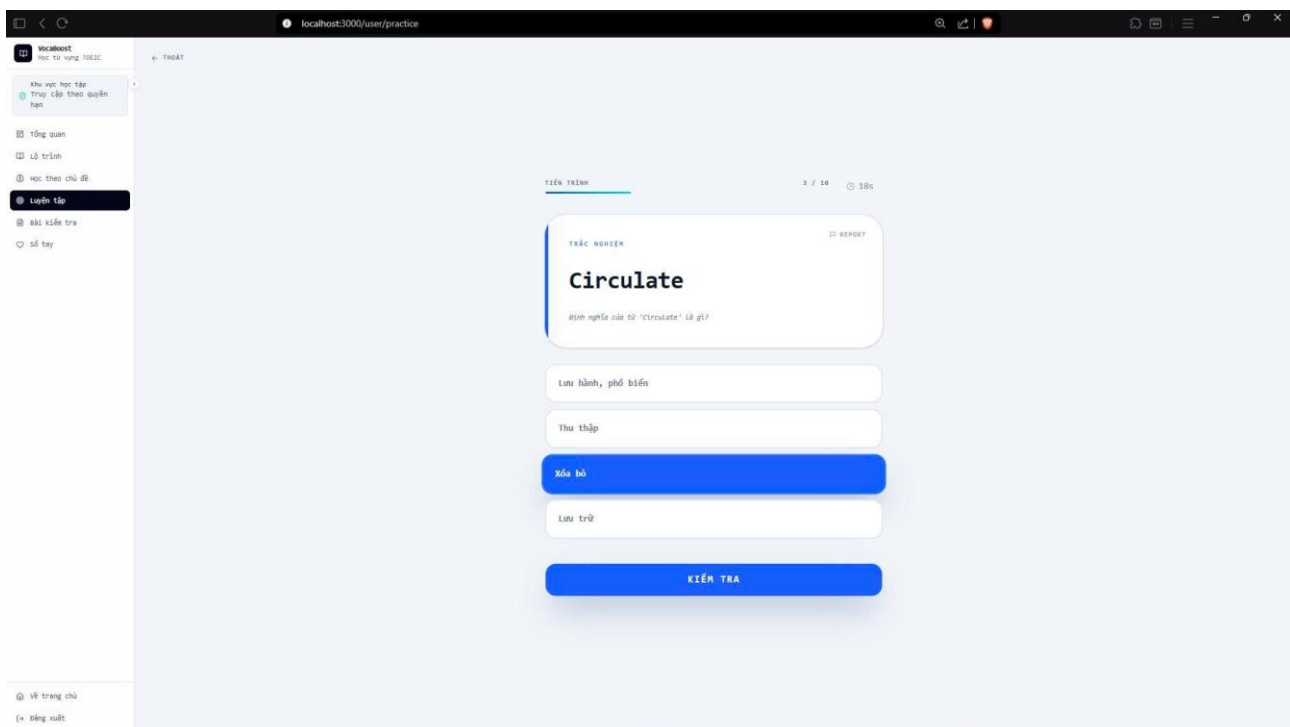


.....44

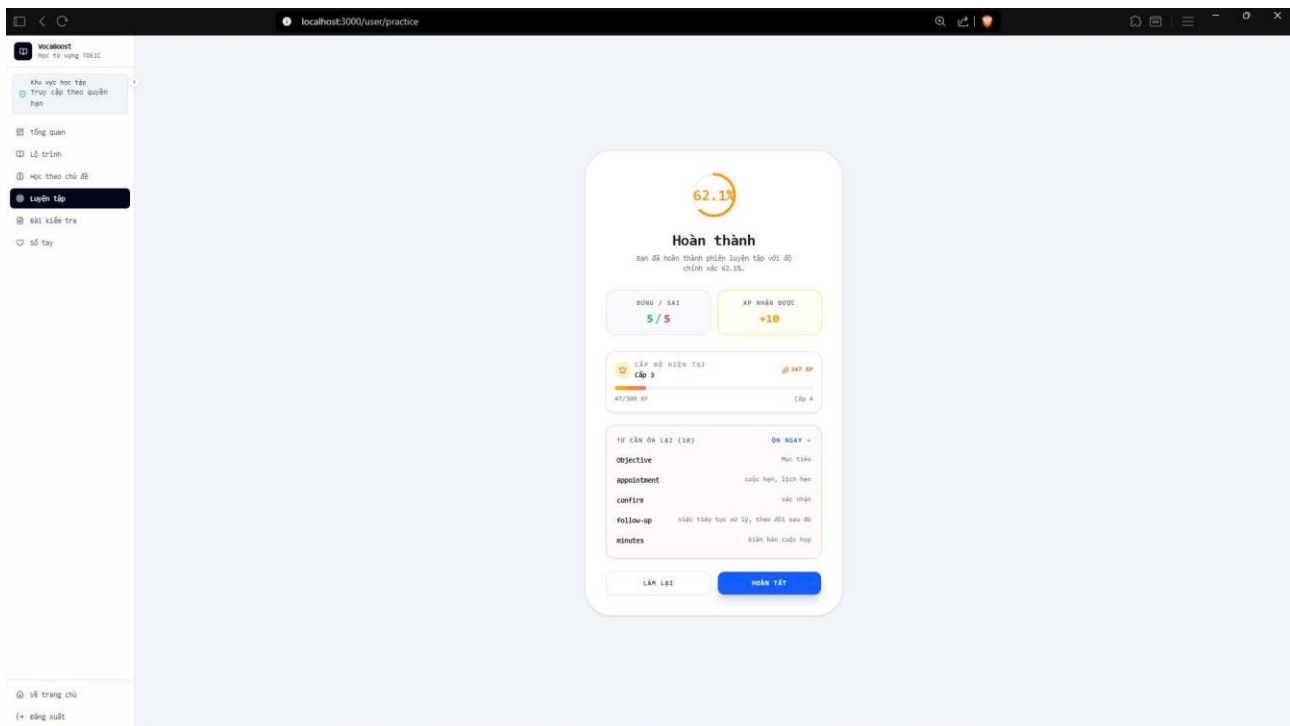
Hình 6.1.6: Giao diện học bằng flashcard.....44



44

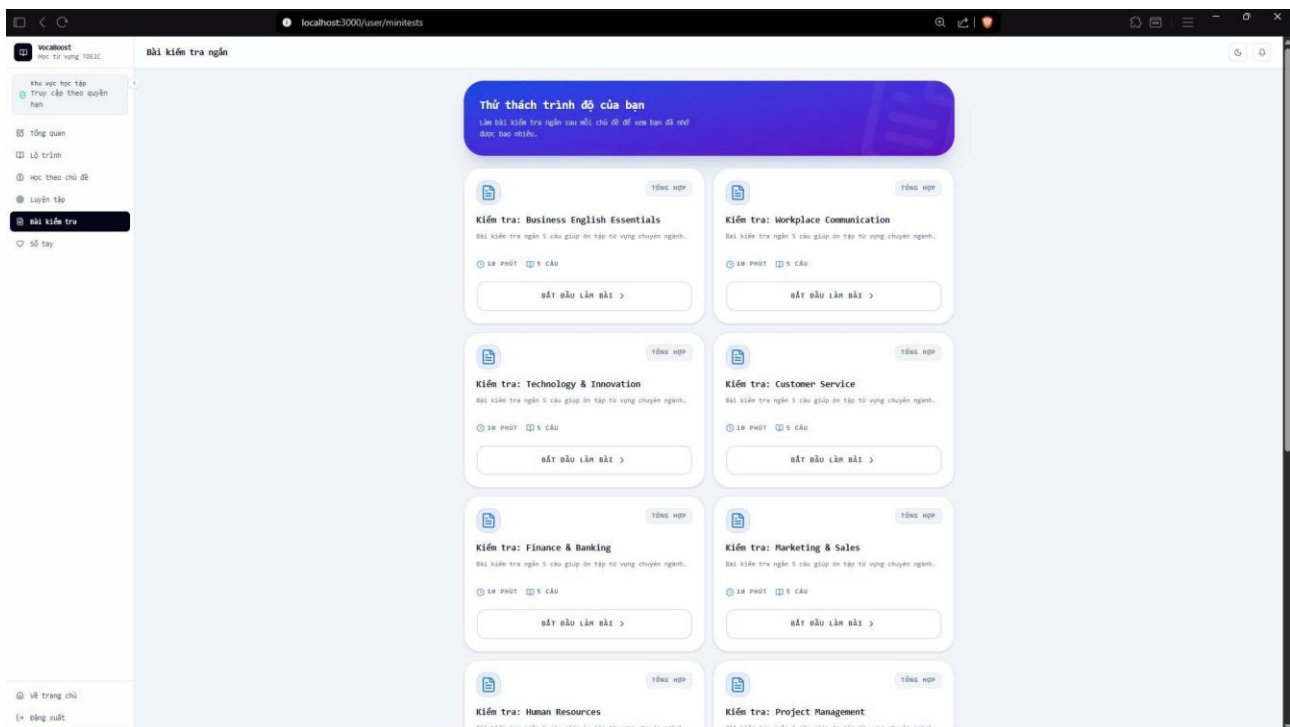


45



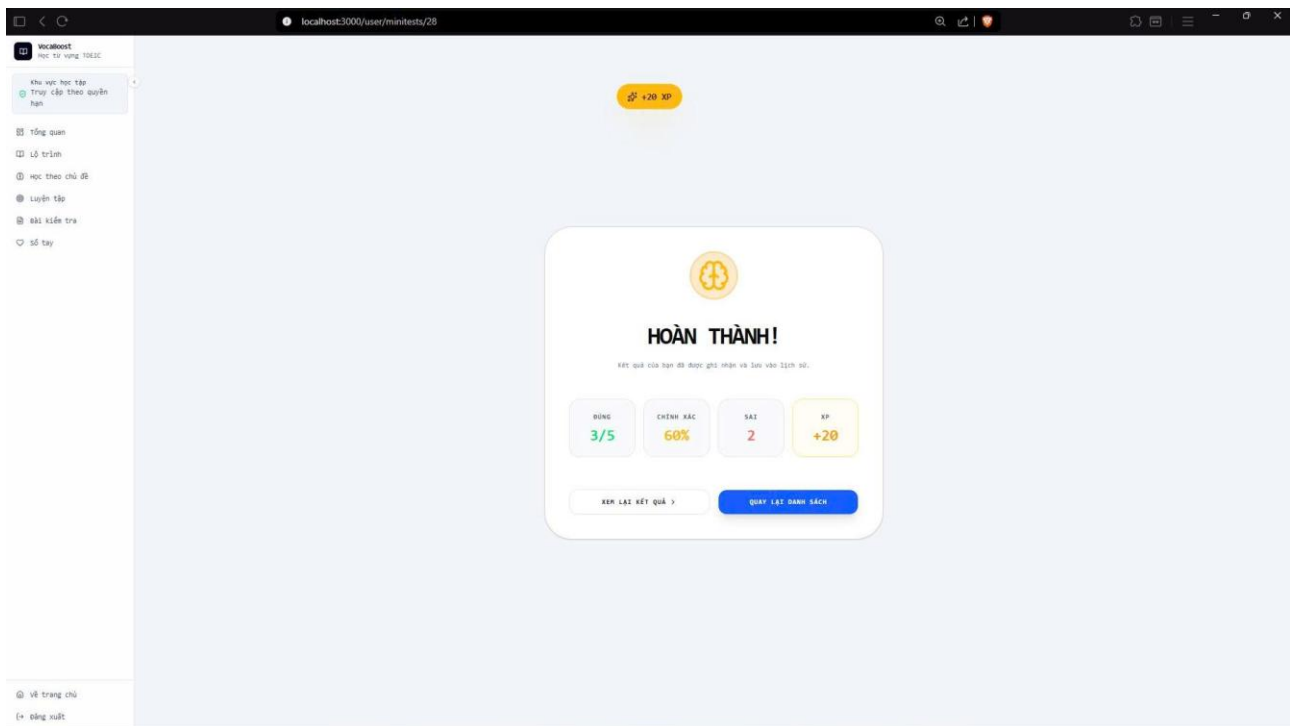
..... 45

Hình 6.1.7: Màn hình chọn chế độ luyện tập và ôn tập thông minh theo dữ liệu SRS. 45



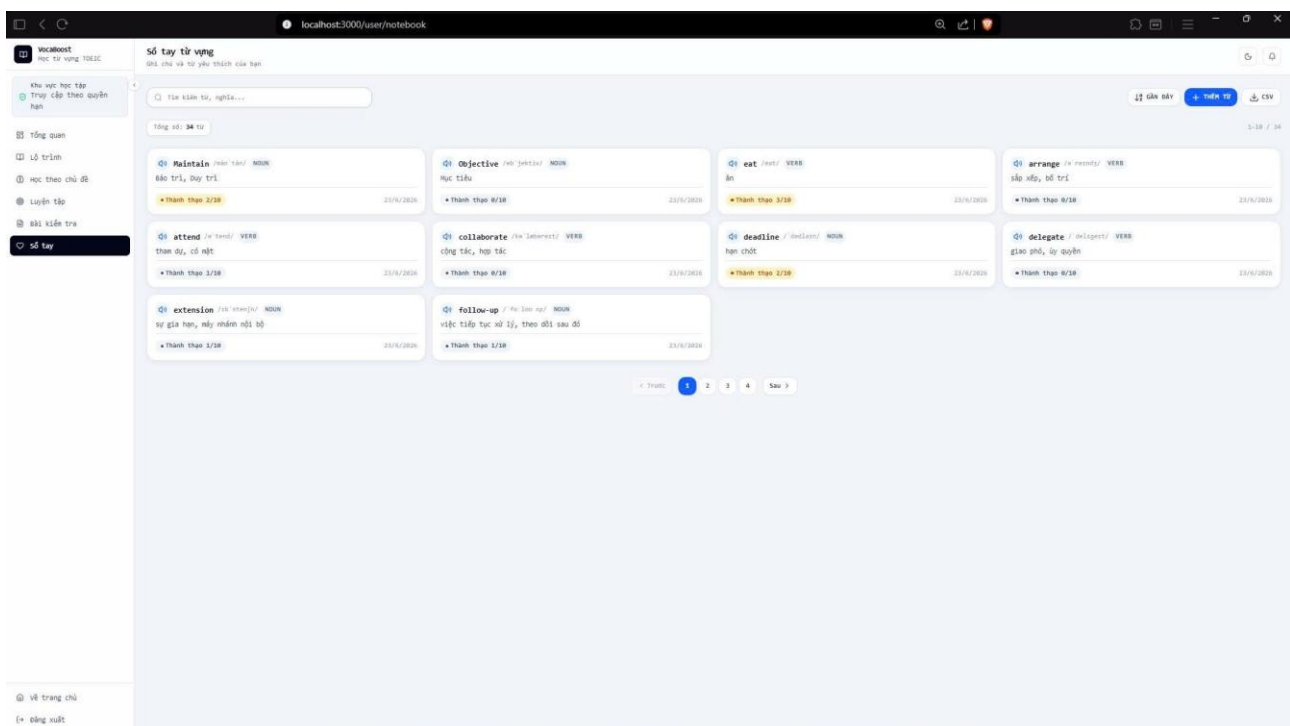
..... 46

Hình 6.1.8: Danh sách mini test để đánh giá mức độ ghi nhớ và theo dõi kết quả. 46



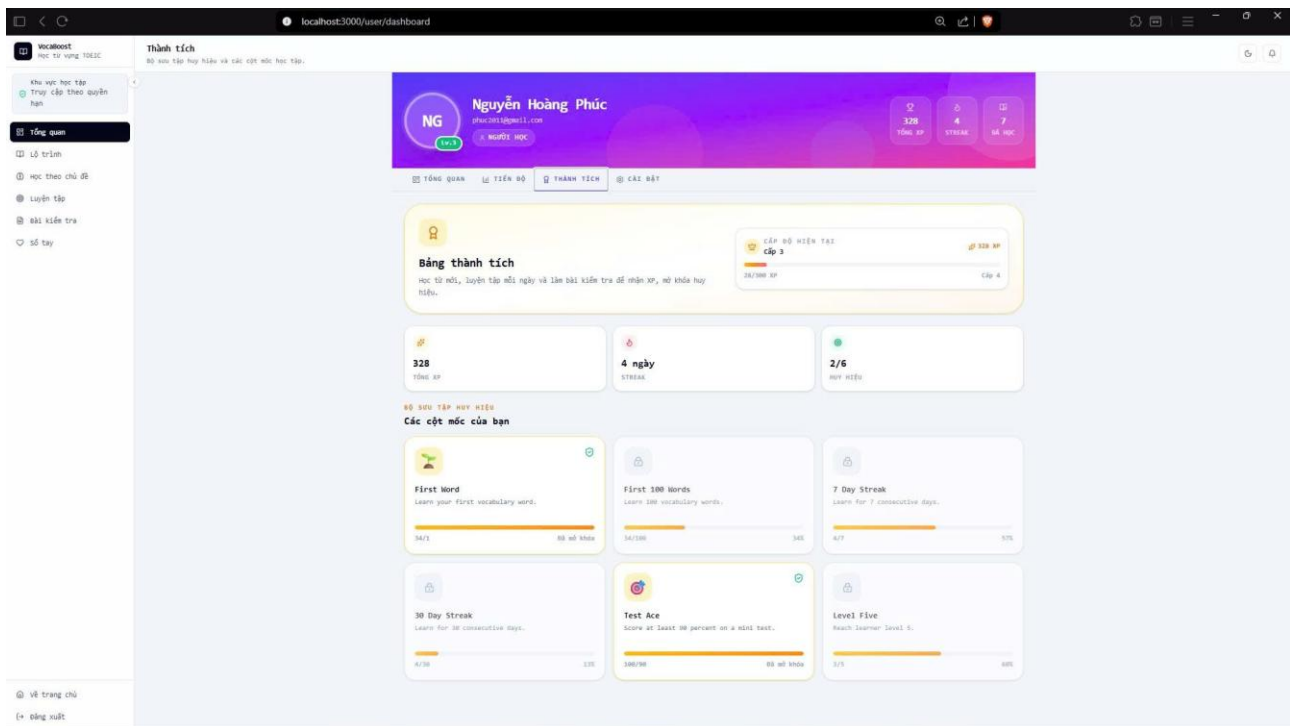
.....47

Hình 6.1.9: Giao diện luyện tập minitest47



.....47

Hình 6.1.10: Nơi lưu, tìm kiếm và quản lý những từ vựng học viên muốn ghi nhớ riêng.47

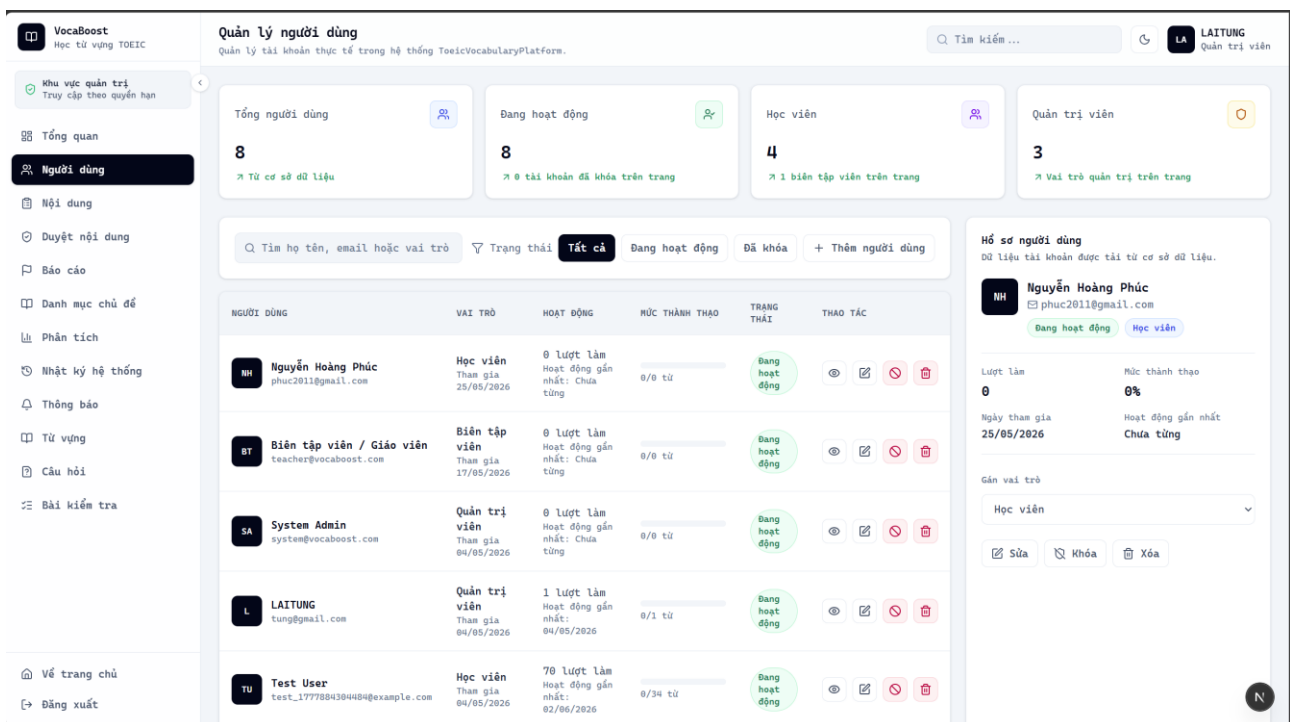


..... 48

Hình 6.1.11: Các huy hiệu, cột mốc và tiến độ gamification của học viên. 48

6.2 Giao diện dành cho quản trị (admin) 48

Hình 6.2.1: Dashboard hệ thống với KPI người dùng, nội dung, hoạt động học và trạng thái dịch vụ. 48



..... 49

Hình 6.2.2: Danh sách tài khoản, vai trò, trạng thái hoạt động và các thao tác quản trị. 49

Hình 6.2.3: Giao diện tổng hợp chủ đề và nội dung học để quản trị trạng thái xuất bản. 50

Hình 6.2.4: Giao diện hàng đợi kiểm duyệt nội dung do biên tập viên gửi lên. 50

Hình 6.2.5: Giao diện danh sách phản hồi/báo lỗi của học viên và quy trình xử lý. 51

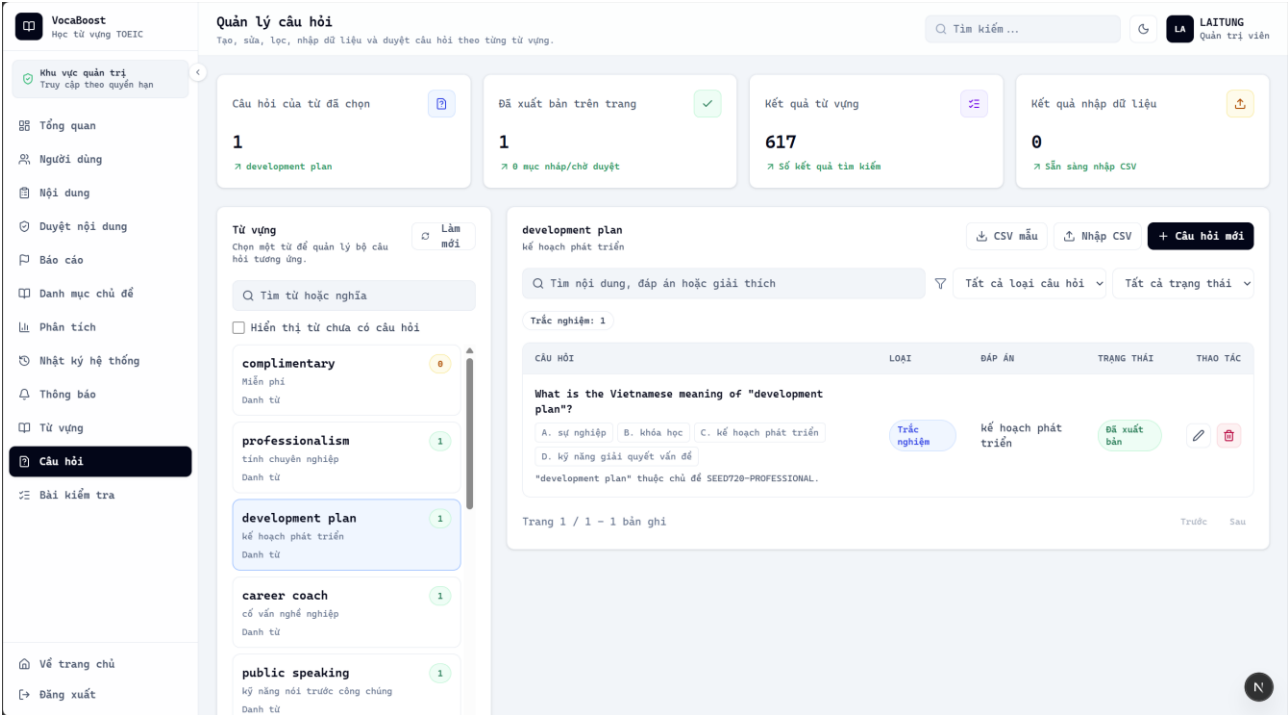
Hình 6.2.6: Giao diện Quản lý các nhóm danh mục dùng để phân loại chủ đề học. 51

Hình 6.2.7: Biểu đồ phân tích người dùng, học tập và dữ liệu nội dung toàn hệ thống. 52

Hình 6.2.8: Giao diện theo dõi các hành động quản trị và thay đổi dữ liệu quan trọng. 52

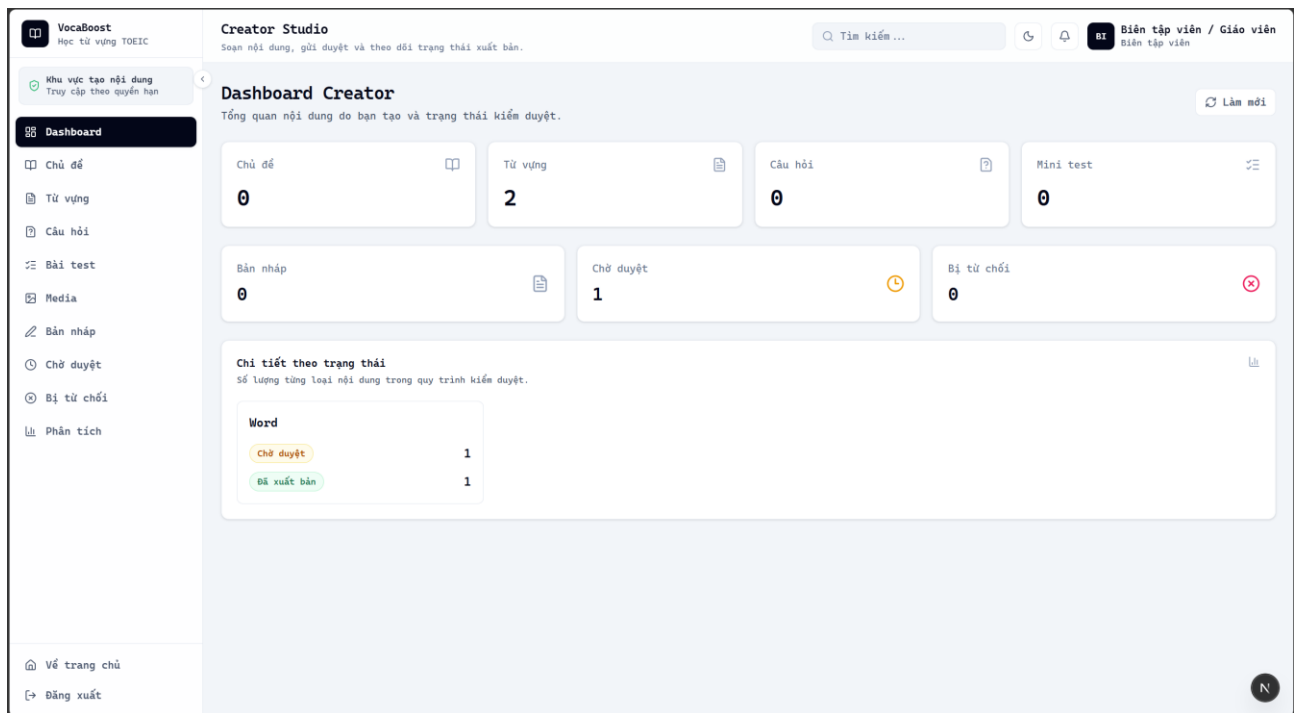
Hình 6.2.9: Giao diện gửi thông báo chung và tạo lời nhắc học hằng ngày..... 53

Hình 6.2.10: Giao diện bảng dữ liệu từ vựng với tìm kiếm, lọc, nhập hàng loạt và CRUD. 54



..... 54

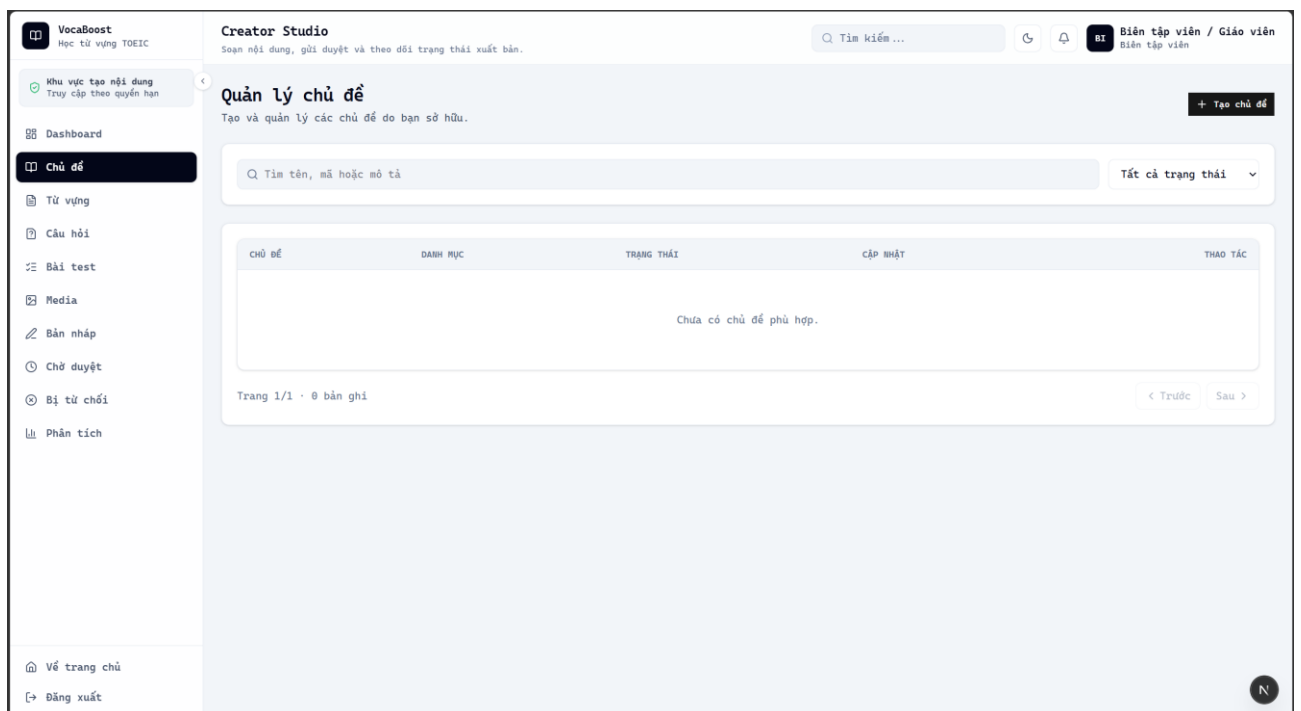
6.3 Giao diện dành cho Creator 55



55

Hình 6.3.1: Tổng quan số lượng chủ đề, từ vựng, câu hỏi, bài test và trạng thái nội dung.

55



55

Hình 6.3.9: Các chỉ số giúp đánh giá quy mô và hiệu quả nội dung đã tạo.

59

1. GIỚI THIỆU

1.1. Mục đích

Tài liệu này cung cấp thiết kế chi tiết về kiến trúc, dữ liệu cũng như các thành phần của hệ thống học từ vựng TOEIC thông minh VocaBoost.

Tài liệu này được sử dụng để:

Nhân viên kiểm thử: Lập kịch bản test và số liệu kiểm thử.

Thành viên đội phát triển: Xây dựng các phân hệ nghiệp vụ chức năng.

Nhân viên thiết kế UI/UX: Tham khảo để thiết kế giao diện.

Giảng viên hướng dẫn: Xác nhận tính đúng đắn của thiết kế.

1.2. Phạm vi

Tài liệu này phục vụ cho việc phát triển, bảo trì dự án VocaBoost.

1.3. Vai trò và trách nhiệm

Tài liệu này miêu tả chi tiết các quy trình phát triển các module và sự liên hệ giữa các module trong hệ thống VocaBoost.

Miêu tả chi tiết sự ràng buộc giữa các bảng trong CSDL, vai trò và mục đích các bảng được sử dụng để phát triển hệ thống VocaBoost.

Miêu tả quy trình thực hiện của hệ thống.

1.4. Tài liệu tham khảo

Liệt kê tất cả các tài liệu tham khảo như: các tài liệu khác của hệ thống, hoặc các bài báo kỹ thuật, ...

Loại tài liệu	Tiêu đề

1.5. Từ và thuật ngữ

Từ/thuật ngữ	Ý nghĩa	Diễn giải
<i>API</i>	<i>Application Programming Interface</i>	<i>Giao diện Lập trình Ứng dụng</i>

2. TỔNG QUAN HỆ THỐNG

VocaBoost là hệ thống học từ vựng TOEIC trực tuyến với hơn 720 từ vựng theo 4 cấp độ (300-500-700-900). Người học tiếp nhận và ôn tập từ vựng thông qua phương pháp SRS cùng cơ chế gamification (XP, Level, Streak, Badge).

Hệ thống sử dụng kiến trúc Frontend-Backend-Database với Next.js 16, React 19, Express.js 5 và SQL Server. Authentication dùng JWT kết hợp Bcrypt, phân quyền dựa trên Role-Based Access Control (RBAC) với 3 vai trò: Admin, Learner, ContentCreator.

Hệ thống VocaBoost cung cấp hơn 720 từ vựng TOEIC chia làm 4 cấp độ (300-500-700-900), học theo phương pháp Spaced Repetition System (SRS) với 4 mức đánh giá: Again (lại), Hard (khó), Good (tốt), Easy (dễ). Cơ chế gamification với XP, Level, Streak, Badge khuyến khích học tập hàng ngày.

Hệ thống hỗ trợ ba vai trò người dùng: Người học (Learner) học flashcard SRS, làm mini test, notebook từ vựng. Quản trị viên (Admin) quản lý Words, Questions, Topics, MiniTests, Students, Content Review. Biên tập viên (Content Creator) tạo nội dung và gửi duyệt.

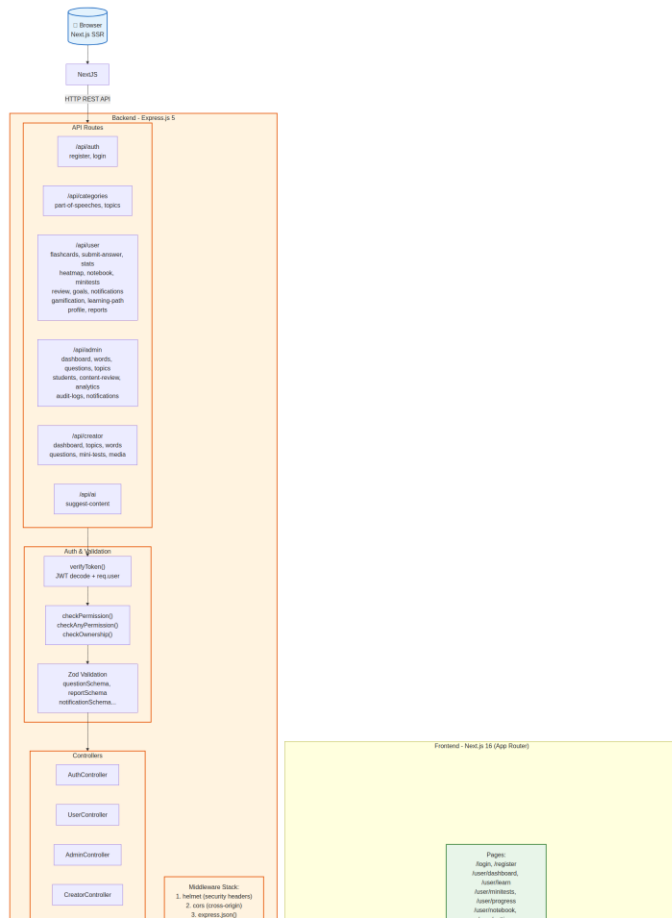
- +) Ảnh hưởng đến chất lượng dịch vụ học tập tại học tập của VocaBoost
- +) + Người học có thể luyện tập từ vựng mới và ôn tập từ đã học theo lịch trình SRS thông minh
- +) + Quản trị viên có thể quản lý từ vựng, câu hỏi, mini test, và duyệt nội dung từ biên tập viên
- + Biên tập viên có thể tạo nội dung từ vựng, câu hỏi, mini test và gửi cho admin duyệt (Draft > PendingReview > Published/Rejected)
- +) Dashboard admin cung cấp thống kê tổng quan: số lượng user, words, questions, mini tests, lượt làm bài, accuracy, và biểu đồ phân tích chi tiết.

2.1. Tổng quan

2.2. Các yêu cầu chức năng

3. KIẾN TRÚC HỆ THỐNG

3.1. Mô hình kiến trúc



Hình 5: Sơ đồ các thành phần của phần mềm

- Client (Frontend)

Giao diện người dùng được xây dựng bằng Next.js 16 (App Router), React 19, TypeScript và Tailwind CSS 4. Giao tiếp với backend thông qua REST API (Axios).

- Backend

Thành phần logic xử lý nghiệp vụ của hệ thống (Express.js 5), bao gồm:

- API Routes (Express Router)

Các route API tiếp nhận yêu cầu từ Client, xác thực JWT và điều hướng đến Controller tương ứng. Xác thực người dùng dựa trên JWT với phân quyền động.

- Auth Controller & Auth Service

Auth Controller & Auth Service xử lý đăng ký và đăng nhập người dùng. AuthService.register() tạo user mới, hash password bằng Bcrypt. AuthService.login() xác thực thông tin và trả về JWT token chứa userId, email, role, permissions.

- **AuthController**

Thành phần Controller xử lý xác thực người dùng. AuthController cung cấp endpoint POST /api/auth/register (đăng ký với fullName, email, password) và POST /api/auth/login (đăng nhập, trả về JWT token). Sử dụng AuthService để hash password bằng Bcrypt, tạo JWT token chứa userId, email, role, permissions với thời hạn 1 ngày. Sau đăng nhập thành công, tự động awardDailyLogin XP cho người học (Learner).

- **AdminController & AdminService**

Admin Controller & Admin Service xử lý logic quản trị: CRUD từ vựng (Words), câu hỏi (Questions), chủ đề (Topics), mini test (MiniTests); quản lý người dùng (Students); duyệt nội dung (Content Review); thống kê dashboard; import CSV; audit logs.

- **Creator Controller & Creator Service**

Xử lý logic cho biên tập viên: CRUD nội dung (từ vựng, câu hỏi, mini test), gửi duyệt nội dung (submit-review), xem thống kê (content-summary, analytics). Nội dung tạo ở trạng thái Draft, gửi admin duyệt qua PendingReview.

- **GamificationService**

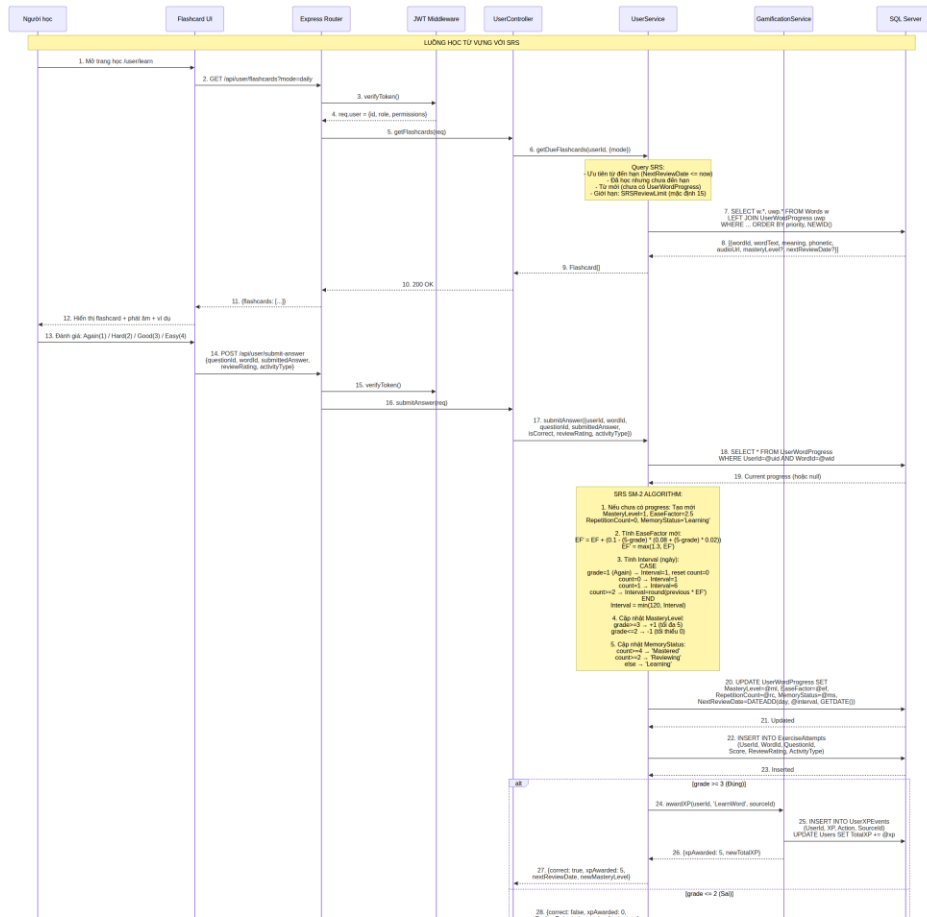
Dịch vụ gamification xử lý XP, Level, Streak, Achievement. Định nghĩa XP_REWARDS cho các hành động: LearnWord (10 XP), PracticeComplete (50 XP), MiniTestComplete (100 XP), DailyLogin (20 XP). Cung cấp getLevelState(totalXP) để tính cấp độ hiện tại ($\text{Level} = \text{floor}(\text{totalXP}/100) + 1$) và tiến độ XP trong cấp độ. Hàm awardXP(userId, action, sourceId) ghi nhận sự kiện XP. Theo dõi streak (số ngày học liên tiếp) và achievements (huy hiệu).

- **LearningPathService**

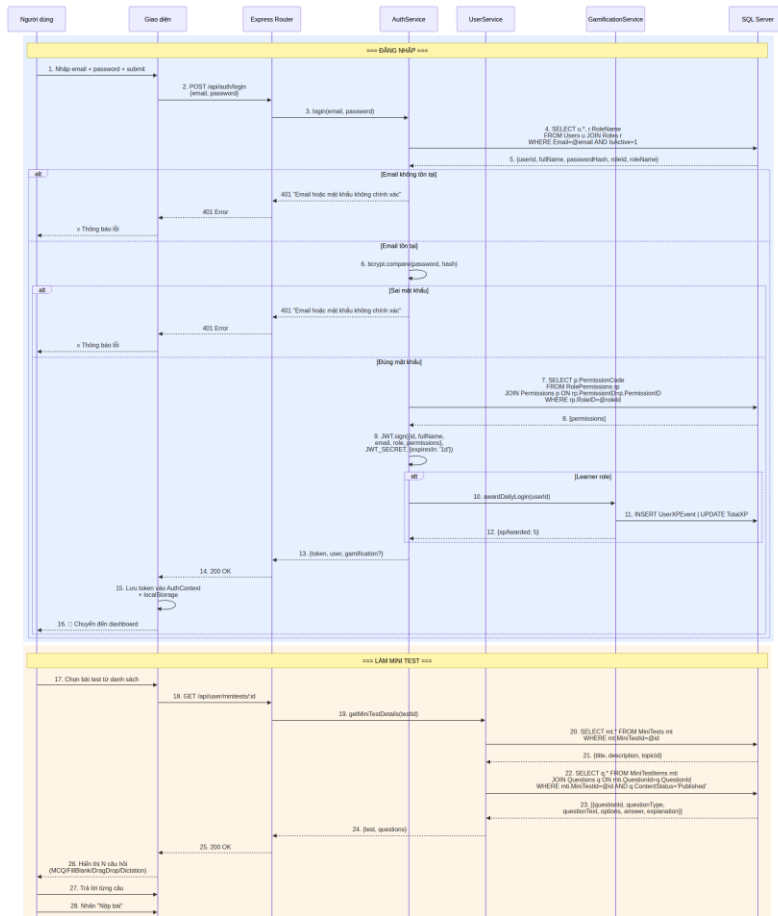
Dịch vụ quản lý lộ trình học tập (Learning Path). Tự động tạo schema với 2 bảng: dbo.LearningPathLevels (LevelId, Name, DisplayOrder) và dbo.LearningPathTopics (liên kết Level với Topics). Định nghĩa 4 cấp độ TOEIC: TOEIC 300, TOEIC 500, TOEIC 700, TOEIC 900. Mỗi cấp độ chứa nhiều chủ đề (Topics) tương ứng. getLearningPath(userId, levelId) trả về lộ trình cá nhân hóa với tiến độ học tập của người dùng tại từng chủ đề.

- **SQL Server Database (ToeicVocabularyPlatform)**

Cơ sở dữ liệu SQL Server (ToeicVocabularyPlatform) sử dụng mssql/msnodesqlv8 driver, hỗ trợ Windows Authentication và SQL Authentication. Tất cả queries được tham số hóa (parameterized queries) để chống SQL injection.



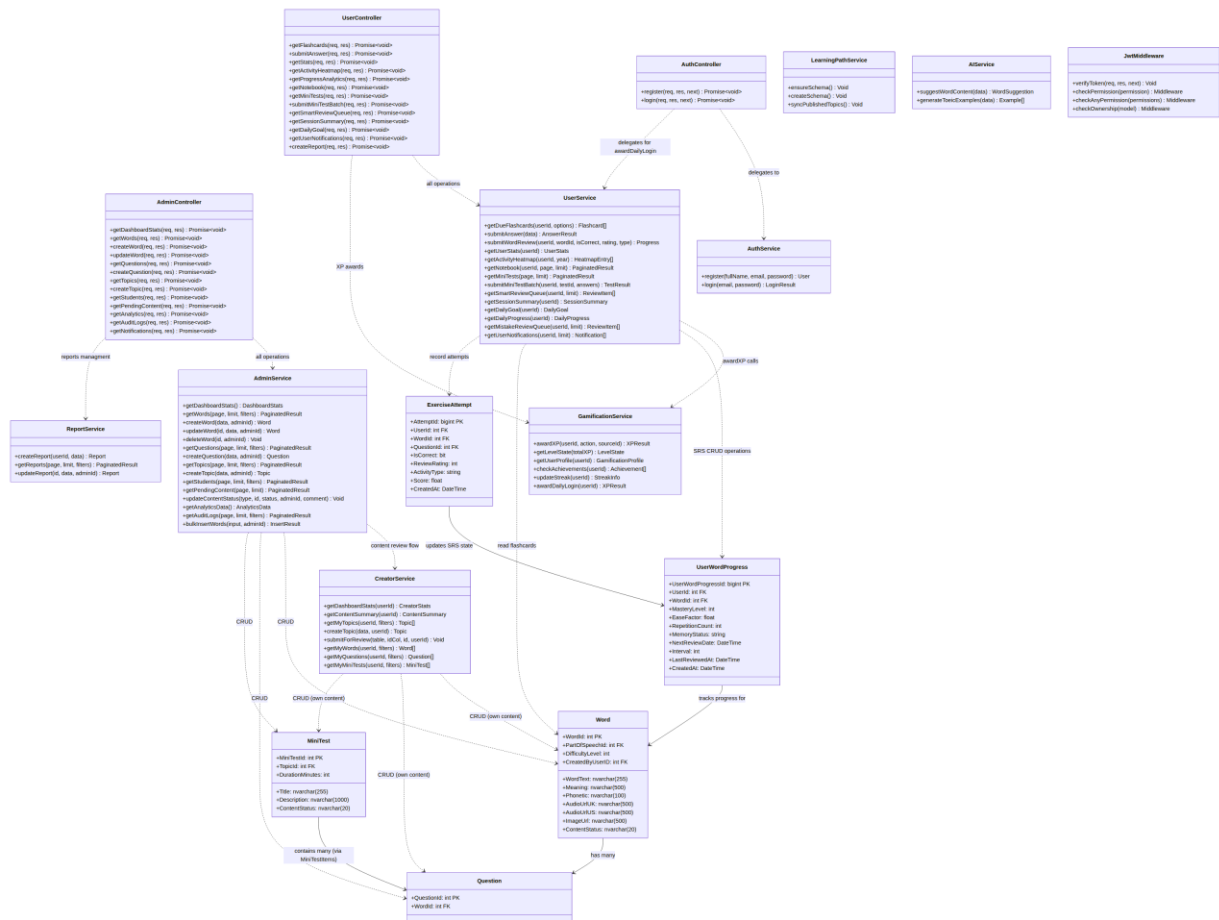
Hình 6: Sequence Diagram cho chức năng học từ vựng (SubmitAnswer)



Hình 7: Sequence Diagram cho chức năng đăng nhập và tạo Mini Test

4. THIẾT KẾ LỚP

4.1. Class Diagram



Hình 8: Biểu đồ lớp cho hệ thống VocaBoost

Các module backend được tách theo lớp Controller và Service. Controller chịu trách nhiệm nhận request, đọc params/body/query và trả response; Service chịu trách nhiệm xử lý nghiệp vụ, truy vấn CSDL, transaction, cập nhật trạng thái và gọi các service liên quan. Mỗi hàm dưới đây đều có luồng xử lý và giả mã.

1. Module Auth - Xác thực và phân quyền

Mục đích	Xử lý đăng ký, đăng nhập, phát hành JWT và kiểm soát quyền truy cập API.
Thành phần	auth.routes.js; AuthController; AuthService; validate middleware; auth middleware; bảng Users, Roles, Permissions.
Luồng module	Client -> auth.routes -> validate(schema) -> AuthController -> AuthService -> SQL Server -> response. Với API nội bộ: Client -> verifyToken -> checkPermission/checkAnyPermission -> Controller.

Controller

Hàm Controller	Luồng xử lý	Giả mã
AuthController.register(req,res,next) <i>controllers/auth.controller.js</i> Mục đích: Nhận request đăng ký và trả HTTP 201.	Client POST /api/auth/register -> validate schemas.register -> controller lấy fullName,email,password -> gọi AuthService.register -> trả user mới -> lỗi chuyển next(error)	FUNCTION register(req,res,next) TRY Lấy fullName,email,password từ req.body user = AuthService.register(fullName,email,password)

		<pre> res.status(201).json(user) CATCH error next(error) END TRY END FUNCTION </pre>
AuthController.login(req,res,next) <i>controllers/auth.controller.js</i> Mục đích: Nhận request đăng nhập và trả token.	Client POST /api/auth/login -> validate schemas.login -> controller lấy email,password -> gọi AuthService.login -> trả token,user,gamification nếu có	<pre> FUNCTION login(req,res,next) TRY Lấy email,password từ req.body result = AuthService.login(email,password) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
AuthService.register(fullName, email,password) <i>services/auth.service.js</i> Mục đích: Tạo user mới với role Learner.	Service nhận dữ liệu từ controller -> query kiểm tra email -> hash password bằng bcrypt -> lấy RoleID Learner -> INSERT Users -> trả user object không chứa passwordHash	<pre> FUNCTION register(fullName,email,password) pool = mở kết nối CSDL existing = SELECT Users WHERE Email=email IF existing THEN throw 400 passwordHash = bcrypt.hash(password) role = SELECT RoleID WHERE RoleName='Learner' INSERT Users(fullName,email,passwordHash,role) RETURN user mới END FUNCTION </pre>
AuthService.login(email,password) <i>services/auth.service.js</i> Mục đích: Xác thực tài khoản và sinh JWT.	Service query user active -> so sánh mật khẩu -> query permissions -> jwt.sign payload -> nếu Learner thì thưởng daily login -> trả token + user	<pre> FUNCTION login(email,password) user = SELECT active user by email IF user không tồn tại THEN throw 401 valid = bcrypt.compare(password,user.PasswordHash) IF valid=false THEN throw 401 permissions = query permission theo RoleID token = jwt.sign({id,role,permissions}) IF role='Learner' THEN awardDailyLogin(userId) RETURN {token,user} END FUNCTION </pre>
verifyToken checkPermission <i>middlewares/auth.js</i> Mục đích: Bảo vệ các module sau đăng nhập.	Route gọi middleware -> verifyToken đọc Bearer token -> gán req.user -> checkPermission kiểm tra permissions -> hợp lệ thì next()	<pre> FUNCTION verifyToken(req,res,next) token = lấy Bearer token IF thiếu token THEN return 401 payload = jwt.verify(token) req.user = payload next() END FUNCTION FUNCTION checkPermission(permission) IF req.user.permissions không chứa permission return 403 next() END FUNCTION </pre>

2. Module Categories - Danh mục và chủ đề học

Mục đích	Cung cấp loại từ, topic học và tiến độ topic cho frontend.
Thành phần	categories.routes.js; CategoriesController; CategoriesService; bảng PartOfSpeeches, Topics, WordTopics, UserWordProgress.
Luồng module	Client -> categories.routes -> CategoriesController -> CategoriesService -> SQL Server -> JSON danh mục/chủ đề.

Controller

Hàm Controller	Luồng xử lý	Giả mã
----------------	-------------	--------

CategoriesController.getPartOfSpeeches <i>controllers/categories.controller.js</i> Mục đích: Trả danh sách loại từ.	GET /part-of-speeches -> controller gọi service -> nhận recordset -> trả HTTP 200	<pre> FUNCTION getPartOfSpeeches (req, res, next) TRY data = CategoriesService.getPartOfSpeeches () res.status(200).json(data) CATCH error next(error) END TRY END FUNCTION </pre>
CategoriesController.getTopics <i>controllers/categories.controller.js</i> Mục đích: Trả danh sách topic học.	GET /topics -> lấy userId nếu có -> gọi CategoriesService.getTopics -> trả topics kèm progress	<pre> FUNCTION getTopics (req, res, next) TRY userId = req.user?.id hoặc null topics = CategoriesService.getTopics(userId) res.status(200).json(topics) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giải mã
CategoriesService.getPartOfSpeeches <i>services/categories.service.js</i> Mục đích: Query bảng loại từ.	Kết nối DB -> SELECT PartOfSpeeches -> ORDER BY tên -> return recordset	<pre> FUNCTION getPartOfSpeeches () pool = mở kết nối CSDL result = SELECT * FROM PartOfSpeeches ORDER BY name RETURN result.recordset END FUNCTION </pre>
CategoriesService.getTopics(userId) <i>services/categories.service.js</i> Mục đích: Tính topic kèm wordCount/progress.	Query Topics Published -> JOIN WordTopics/Words -> nếu có userId thì JOIN UserWordProgress -> tính learned/mastered/duel/progress -> return topics	<pre> FUNCTION getTopics(userId) Query danh sách Topics Published FOR mỗi topic wordCount = đếm Words thuộc topic IF userId tồn tại THEN learnedCount = đếm progress của user masteredCount = đếm từ mastered dueCount = đếm từ đến hạn ôn progressPercent = learnedCount/wordCount END IF END FOR RETURN topics END FUNCTION </pre>

3. Module User Learning - Học từ vựng, SRS, mini-test, notebook

Mục đích	Nhiệm vụ chính của Learner: học flashcard, ôn SRS, làm mini-test, xem tiến độ và quản lý notebook.
Thành phần	user.routes.js; UserController; UserService; GamificationService; bảng Words, Questions, UserWordProgress, ExerciseAttempts, MiniTestAttempts, UserVocabularyNotebook.
Luồng module	Learner -> user.routes -> verifyToken -> UserController -> UserService -> SQL Server. Khi hoàn thành hoạt động học, UserService cập nhật progress và gọi GamificationService cộng XP.

Controller

Hàm Controller	Luồng xử lý	Giải mã
UserController.getFlashcards <i>controllers/user.controller.js</i> Mục đích: Nhận yêu cầu lấy flashcards.	GET /flashcards -> lấy userId, topicId, mode -> gọi UserService.getDueFlashcards -> trả danh sách thẻ học	<pre> FUNCTION getFlashcards (req, res, next) TRY userId = req.user.id options = {topicId:req.query.topicId, mode:req.query.mode} flashcards = UserService.getDueFlashcards(userId, options) res.status(200).json(flashcards) CATCH error next(error) END TRY END FUNCTION </pre>
UserController.submitAnswer <i>controllers/user.controller.js</i>	POST /submit-answer -> lấy body answer/rating -> gọi	<pre> FUNCTION submitAnswer (req, res, next) TRY payload = req.body + userId từ token </pre>

Mục đích: Nhận đáp án luyện tập.	UserService.submitAnswer -> trả kết quả đúng/sai, XP, nextReviewDate	<pre> result = UserService.submitAnswer(payload) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>
UserController.submitMiniTest <i>controllers/user.controller.js</i> Mục đích: Nhận bài làm mini-test.	POST /minitests/:id/submit -> lấy testId và answers -> gọi UserService.submitMiniTestBatch -> trả điểm và chi tiết	<pre> FUNCTION submitMiniTest(req,res,next) TRY testId = req.params.id answers = req.body.answers result = UserService.submitMiniTestBatch(userId, testId, answers) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>
UserController.getProgressAnalytics <i>controllers/user.controller.js</i> Mục đích: Trả dữ liệu biểu đồ tiến độ.	GET /progress/analytics -> lấy userId -> gọi UserService.getProgressAnalytics -> trả analytics	<pre> FUNCTION getProgressAnalytics(req,res,next) TRY analytics = UserService.getProgressAnalytics(req.userId) res.status(200).json(analytics) CATCH error next(error) END TRY END FUNCTION </pre>
Notebook controllers <i>controllers/user.controller.js</i> Mục đích: CRUD sổ tay từ vựng.	GET/POST/PUT/DELETE /notebook -> controller đọc params/body -> gọi service tương ứng -> trả entry/list/success	<pre> FUNCTION notebookController(req,res,next) TRY IF GET THEN getNotebook(userId,page,pageSize) IF POST THEN addNotebookEntry(userId,wordId,note) IF PUT THEN updateNotebookEntry(id,userId,data) IF DELETE THEN deleteNotebookEntry(id,userId) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
UserService.getDueFlashcards <i>services/user.service.js</i> Mục đích: Chọn từ cần học/ôn theo SRS.	Chuẩn hóa topic/mode -> lấy limit SRS của user -> query Words Published + UserWordProgress -> ưu tiên từ due/sai nhiều/từ mới -> trả flashcards	<pre> FUNCTION getDueFlashcards(userId,options) limit = lấy SRSReviewLimit của user Query Words Published LEFT JOIN UserWordProgress IF mode='new' THEN lọc từ chưa học IF mode='daily' THEN lọc NextReviewDate <= now ORDER BY độ ưu tiên SRS RETURN flashcards END FUNCTION </pre>
UserService.submitAnswer <i>services/user.service.js</i> Mục đích: Ghi attempt và cập nhật feedback SRS.	Gọi stored procedure chấm câu hỏi -> lấy canonicalWordId -> nếu có reviewRating thì cập nhật EaseFactor/NextReviewDate -> nếu LearnWord thì awardXP -> trả result	<pre> FUNCTION submitAnswer(payload) result = EXEC usp_SubmitQuestionAttempt wordId = result.WordID IF wordId rỗng THEN throw error IF reviewRating in Again/Hard/Good/Easy THEN UPDATE UserWordProgress SET EaseFactor, NextReviewDate theo rating END IF IF activityType='LearnWord' THEN awardXP RETURN result + reviewFeedback + xp END FUNCTION </pre>
UserService.submitWordReview <i>services/user.service.js</i> Mục đích: Cập nhật tiến độ một từ sau flashcard.	MERGE UserWordProgress -> đúng tăng mastery, sai giảm mastery -> tính nextReviewDate theo rating -> cập nhật MemoryStatus -> cộng XP nếu cần	<pre> FUNCTION submitWordReview(payload) MERGE progress by userId,wordId IF matched THEN UPDATE mastery/repetition/streak ELSE INSERT progress mới NextReviewDate = tính theo Again/Hard/Good/Easy MemoryStatus = Learning/Reviewing/Mastered/Lapsed </pre>

		RETURN progress mới END FUNCTION
UserService.getSmartReviewQueue <i>services/user.service.js</i> Mục đích: Tạo hàng đợi ôn thông minh.	Query progress có NextReviewDate trong 7 ngày -> tính priorityScore -> từ sai nhiều/quá hạn có điểm cao -> trả TOP limit	FUNCTION getSmartReviewQueue(userId,limit) limit = clamp(1,50) SELECT Words JOIN UserWordProgress WHERE NextReviewDate <= now + 7 days priorityScore = overdueHours * wrongMultiplier ORDER BY priorityScore DESC, MasteryLevel ASC RETURN queue END FUNCTION
UserService.submitMiniTestBatch <i>services/user.service.js</i> Mục đích: Chấm mini-test theo transaction.	Begin transaction -> lấy câu hỏi của test -> validate answers -> loop từng answer: insert attempt, merge progress -> insert MiniTestAttempts -> commit, awardXP	FUNCTION submitMiniTestBatch(userId,testId,answers) BEGIN TRANSACTION questions = SELECT questions by testId IF answers.length != questions.length THEN throw error FOR answer IN answers isCorrect = compare(answer,CorrectAnswer) INSERT ExerciseAttempts MERGE UserWordProgress END FOR INSERT MiniTestAttempts(score) COMMIT awardXP(MiniTestComplete) RETURN score/details END FUNCTION
UserService.getProgressAnalytics <i>services/user.service.js</i> Mục đích: Tổng hợp dashboard học tập.	Query progress, attempts, topic mastery -> tính accuracy, mastery distribution, activity timeline -> trả object cho chart	FUNCTION getProgressAnalytics(userId) learned = count UserWordProgress mastered = count MemoryStatus='Mastered' accuracy = avg ExerciseAttempts.IsCorrect topicProgress = group by topic timeline = group activity by date RETURN analytics END FUNCTION

4. Module Learning Path & Progress

Mục đích	Sinh lộ trình học TOEIC và thống kê tiến độ tổng quan.
Thành phần	LearningPathController; LearningPathService; ProgressController; ProgressService; LearningPathLevels, LearningPathTopics, UserWordProgress.
Luồng module	Client -> learning-path/progress routes -> Controller -> Service -> SQL Server -> roadmap/progress response.

Controller

Hàm Controller	Luồng xử lý	Giải mã
LearningPathController.getRoadmap <i>controllers/learning-path.controller.js</i> Mục đích: Trả roadmap cho learner.	GET /learning-path -> lấy userId -> gọi LearningPathService.getRoadmap -> trả roadmap	FUNCTION getRoadmap(req,res,next) TRY roadmap = LearningPathService.getRoadmap(req.user.id) res.status(200).json(roadmap) CATCH error next(error) END TRY END FUNCTION
ProgressController.getProgressStats <i>controllers/progress.controller.js</i> Mục đích: Trả tiến độ và thống kê.	GET /progress hoặc /progress/stats -> lấy userId -> gọi ProgressService -> trả JSON	FUNCTION progressController(req,res,next) TRY IF route="/" THEN result = ProgressService.getProgress(userId) IF route='/stats' THEN result = ProgressService.getStats(userId) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
LearningPathService.getRoadmap <i>services/learning-path.service.js</i> Mục đích: Lấy roadmap theo level/topic.	ensureSchema -> syncPublishedTopics -> query levels/topics -> query user progress -> buildRoadmap	<pre> FUNCTION getRoadmap(userId) ensureSchema() syncPublishedTopics() levels = SELECT LearningPathLevels topics = SELECT LearningPathTopics JOIN Topics progress = SELECT UserWordProgress by userId RETURN buildRoadmap(levels,topics,progress) END FUNCTION </pre>
LearningPathService.buildRoadmap <i>services/learning-path.service.js</i> Mục đích: Định dạng roadmap cho UI.	Duyệt từng level -> gán topics thuộc level -> tính progressPercent -> xác định trạng thái completed/available/locked	<pre> FUNCTION buildRoadmap(levels,topics,progress) FOR level IN levels levelTopics = topics.filter(level) FOR topic IN levelTopics topic.progress = tính theo UserWordProgress topic.status = completed/available/locked END FOR END FOR RETURN roadmap END FUNCTION </pre>
ProgressService.getProgress getStats <i>services/progress.service.js</i> Mục đích: Tính tiến độ học.	Query UserWordProgress/Attempts -> đếm learned/mastered/reviewing -> tính accuracy/streak -> trả summary	<pre> FUNCTION getProgress(userId) progress = SELECT UserWordProgress WHERE userId RETURN counts by MemoryStatus END FUNCTION FUNCTION getStats(userId) attempts = SELECT ExerciseAttempts WHERE userId accuracy = avg isCorrect streak = tính ngày học liên tiếp RETURN stats END FUNCTION </pre>

5. Module Gamification - XP, Level, Achievement

Mục đích	Cộng XP, tính level/streak và mở khóa huy hiệu cho learner.
Thành phần	GamificationController; GamificationService; UserXPEvents; Achievements; UserAchievements.
Luồng module	Hoạt động học -> Controller/UserService -> GamificationService -> ghi XP event -> cập nhật user -> kiểm tra achievement -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
GamificationController.getProfile <i>controllers/gamification.controller.js</i> Mục đích: Trả hồ sơ gamification.	GET /gamification/profile -> lấy userId -> gọi service.getProfile -> trả XP/level/achievements	<pre> FUNCTION getProfile(req,res,next) TRY profile = GamificationService.getProfile(req.user. id) res.status(200).json(profile) CATCH error next(error) END TRY END FUNCTION </pre>
completePractice markAchievementsSeen <i>controllers/gamification.controller.js</i> Mục đích: Cộng XP phiên luyện tập và đánh dấu huy hiệu đã xem.	Practice complete -> awardXP Achievements seen -> markAchievementsSeen Controller trả success/result	<pre> FUNCTION gamificationActions(req,res,next) TRY IF completePractice THEN awardXP(userId,PracticeComplete) IF markSeen THEN markAchievementsSeen(userId,ids) res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
GamificationService.awardXP <i>services/gamification.service.js</i> Mục đích: Cộng XP theo sự kiện.	ensureSchema -> xác định XP REWARDS -> chống cộng trùng bằng sourceKey -> insert UserXPEvents -> update Users.TotalXP -> checkAchievements	<pre> FUNCTION awardXP(userId,event) xp = XP_REWARDS[eventType] IF sourceKey đã tồn tại THEN return xpGained=0 INSERT UserXPEvents UPDATE Users SET TotalXP += xp levelState = getLevelState(totalXP) achievements = checkAchievements(userId) RETURN xpGained,totalXP,levelState,achievements END FUNCTION </pre>
GamificationService.checkAchievements <i>services/gamification.service.js</i> Mục đích: Mở khóa achievement.	Lấy metrics -> duyệt achievements -> kiểm tra điều kiện -> insert UserAchievements nếu đạt	<pre> FUNCTION checkAchievements(userId) metrics = getMetrics(userId) achievements = getAchievements() FOR achievement IN achievements IF user đạt điều kiện AND chưa unlock THEN INSERT UserAchievements END IF END FOR RETURN newlyUnlocked END FUNCTION </pre>
GamificationService.getProfile <i>services/gamification.service.js</i> Mục đích: Tổng hợp profile.	ensureSchema -> getMetrics -> getAchievements -> getLevelState -> trả profile	<pre> FUNCTION getProfile(userId) ensureSchema() metrics = getMetrics(userId) achievements = getAchievements(userId,metrics) levelState = getLevelState(metrics.totalXP) RETURN {metrics,achievements,levelState} END FUNCTION </pre>

6. Module Admin - Quản trị hệ thống và nội dung

Mục đích	Quản trị toàn bộ nội dung, tài khoản, kiểm duyệt, thống kê, báo cáo, audit log và thông báo.
Thành phần	admin.routes.js; AdminController; AdminService; ReportService; auth/validate middleware; các bảng Words, Topics, Questions, MiniTests, Users, AuditLogs, Reports, Notifications.
Luồng module	Admin -> /api/admin -> verifyToken -> checkPermission/checkAnyPermission -> validate -> AdminController -> AdminService/ReportService -> SQL Server -> audit/review log -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
AdminController.getTopics createTopic updateTopic deleteTopic <i>controllers/admin.controller.js</i> Mục đích: Điều phối CRUD chủ đề.	Route /topics -> kiểm tra quyền MANAGE_TOPICS/MANAGE_WORDS -> validate body nếu POST/PUT -> controller đọc params/query/body -> gọi AdminService	<pre> FUNCTION adminTopicController(req,res,next) TRY IF GET THEN result = AdminService.getTopics(page,limit,filter s) IF POST THEN result = AdminService.createTopic(req.body,adminI d) IF PUT THEN result = AdminService.updateTopic(id,body,adminId) IF DELETE THEN result = AdminService.deleteTopic(id,adminId) res.status(successCode).json(result) CATCH error next(error) END TRY END FUNCTION </pre>
AdminController.getWords createWord updateWord deleteWord import <i>controllers/admin.controller.js</i>	Route /words -> check MANAGE_WORDS -> controller đọc filter/body/csv text -> gọi service get/create/update/delete/previ ew/bulkImport -> trả JSON	<pre> FUNCTION adminWordController(req,res,next) TRY SWITCH route GET /words -> getWords(page,limit,filters) POST /words -> createWord(body,adminId) PUT /words/:id -> updateWord(id,body,adminId) </pre>

Mục đích: Điều phối quản lý từ vựng.		<pre> DELETE -> deleteWord(id,adminId) import -> previewWordImport/bulkInsertWords(text) END SWITCH res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>
AdminController question miniTest handlers <i>controllers/admin.controller.js</i> Mục đích: Điều phối câu hỏi và mini-test.	Route questions/minitests -> check MANAGE_QUESTIONS/MANAGE_TESTS -> validate body -> gọi service tương ứng -> trả danh sách/chi tiết/status	<pre> FUNCTION adminAssessmentController(req,res,next) TRY IF question route THEN call AdminService question method IF minitest route THEN call AdminService miniTest method res.status(200).json(result) CATCH error next(error) END TRY END FUNCTION </pre>
AdminController user review report notification handlers <i>controllers/admin.controller.js</i> Mục đích: Điều phối user, duyệt nội dung, báo cáo, thông báo.	Route students/review/reports/notifications -> check quyền phù hợp -> controller lấy params/body -> gọi AdminService hoặc ReportService -> trả kết quả	<pre> FUNCTION adminOperationController(req,res,next) TRY IF students THEN call user management service IF content-review THEN approve/reject/archive/logs IF reports THEN ReportService.get/update IF notifications THEN sendAnnouncement/createDailyReminders res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
AdminService.getTopics createTopic updateTopic deleteTopic <i>services/admin.service.js</i> Mục đích: Thực thi CRUD Topics.	get: build filter + paginate create/update: transaction topic + metadata delete: kiểm tra liên kết rồi xóa/ẩn audit log sau thay đổi	<pre> FUNCTION adminTopicService(action) IF get THEN query Topics + wordCount + miniTestCount IF create THEN INSERT Topics IF update THEN UPDATE Topics WHERE TopicID IF delete THEN kiểm tra WordTopics/MiniTests rồi DELETE/Archive logAdminAction(adminId,action,'Topic',id) RETURN result END FUNCTION </pre>
AdminService.getWords createWord updateWord deleteWord <i>services/admin.service.js</i> Mục đích: Quản lý Words, topics, examples.	get: query Words + relations create/update: transaction -> Words -> WordTopics -> ExampleSentences delete/archive theo quyền	<pre> FUNCTION adminWordService(action) BEGIN TRANSACTION nếu create/update IF create THEN INSERT Words IF update THEN UPDATE Words IF topicIds THEN replace WordTopics IF examples THEN replace ExampleSentences COMMIT logAdminAction RETURN word/result END FUNCTION </pre>
AdminService.previewWordImport bulkInsertWords <i>services/admin.service.js</i> Mục đích: Nhập hàng loạt từ vựng.	preview parse csv -> validate rows bulk parse -> transaction -> insert words/topics/examples -> commit/rollback	<pre> FUNCTION bulkInsertWords(input,adminId) rows = parseDelimitedImport(input) validRows,errors = validate rows BEGIN TRANSACTION FOR row IN validRows INSERT Words INSERT WordTopics INSERT ExampleSentences END FOR COMMIT RETURN {inserted,errors} END FUNCTION </pre>
AdminService question miniTest services <i>services/admin.service.js</i> Mục đích: Quản lý Questions và MiniTests.	Questions: CRUD theo WordID MiniTests: CRUD test + items publish/archive cập nhật status và logs	<pre> FUNCTION assessmentService(action) IF question THEN validate WordID, answer, options; INSERT/UPDATE/DELETE Questions IF miniTest THEN BEGIN TRANSACTION; INSERT/UPDATE MiniTests; replace MiniTestItems; COMMIT IF publish/archive THEN setMiniTestStatus </pre>

		<pre> RETURN result END FUNCTION </pre>
AdminService user content report notification services <i>services/admin.service.js;</i> <i>services/report.service.js</i> Mục đích: Vận hành hệ thống.	Users: query/update role/status Content: update status + ContentReviewLogs Reports: get/update status Notifications: insert announcement/reminder	<pre> FUNCTION adminOperationService(action) IF user THEN SELECT/UPDATE Users, Roles IF content THEN UPDATE entity ContentStatus; INSERT ContentReviewLogs IF report THEN ReportService.getReports/updateReport IF notification THEN INSERT Notifications logAdminAction RETURN result END FUNCTION </pre>

7. Module Creator - Biên tập nội dung học

Mục đích	Cho Content Creator tạo topic, word, question, mini-test, media và gửi duyệt.
Thành phần	creator.routes.js; CreatorController; CreatorService; validate/auth middleware; Topics, Words, Questions, MiniTests, Media.
Luồng module	Creator -> /api/creator -> verifyToken -> permission -> validate -> CreatorController -> CreatorService -> SQL Server. Nội dung tạo ở Draft, submit-review chuyển PendingReview.

Controller

Hàm Controller	Luồng xử lý	Giả mã
CreatorController dashboard handlers <i>controllers/creator.controller.js</i> Mục đích: Trả dashboard và analytics cho creator.	<pre> GET dashboard/summary/analytics -> lấy userId -> gọi CreatorService stats/analytics -> trả dữ liệu </pre>	<pre> FUNCTION creatorDashboardController(req,res,next) TRY userId = req.user.id result = CreatorService.getDashboardStats/getCont entSummary/getAnalytics(userId) res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>
CreatorController topic word question handlers <i>controllers/creator.controller.js</i> Mục đích: Điều phối CRUD nội dung cơ bản.	<pre> Route topics/words/questions -> check permission -> validate body -> controller gọi CreatorService -> trả entity/status </pre>	<pre> FUNCTION creatorContentController(req,res,next) TRY IF topic route THEN call topic service IF word route THEN call word service IF question route THEN call question service IF submit-review THEN call submitForReview wrapper res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>
CreatorController miniTest media handlers <i>controllers/creator.controller.js</i> Mục đích: Điều phối mini-test và media.	<pre> Route mini-tests/media -> check permission -> validate -> gọi CreatorService -> trả result </pre>	<pre> FUNCTION creatorExtraController(req,res,next) TRY IF miniTest route THEN call miniTest service IF media route THEN call media service res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
CreatorService.create update deleteTopic	Kiểm tra ownership -> insert/update/delete Topics -> status mặc định Draft -> trả topic	<pre> FUNCTION creatorTopicService(action) IF create THEN INSERT Topics(ContentStatus='Draft') IF update/delete THEN kiểm tra </pre>

<i>services/creator.service.js</i> Mục đích: Creator quản lý topic của mình.		CreatedByUserID=userId UPDATE/DELETE Topics RETURN topic/status END FUNCTION
CreatorService.createWord updateWord deleteWord <i>services/creator.service.js</i> Mục đích: Biên soạn từ vựng và ví dụ.	Begin transaction -> kiểm tra partOfSpeech/topic -> kiểm tra trùng từ -> insert/update Words -> insert WordTopics/ExampleSentences -> commit	FUNCTION creatorWordService(data,userId) BEGIN TRANSACTION Validate PartOfSpeechID Validate topic ownership/Published Check duplicate Term + PartOfSpeech INSERT/UPDATE Words status Draft Replace WordTopics and ExampleSentences COMMIT RETURN word END FUNCTION
CreatorService.createQuestion updateQuestion <i>services/creator.service.js</i> Mục đích: Tạo câu hỏi luyện tập.	Kiểm tra word được phép dùng -> validate question/options -> insert/update Questions -> status Draft	FUNCTION creatorQuestionService(data,userId) Validate WordID belongs to creator or Published Validate question text, correctAnswer, options INSERT/UPDATE Questions(ContentStatus='Draft') RETURN question END FUNCTION
CreatorService.createMiniTest updateMiniTest items <i>services/creator.service.js</i> Mục đích: Tạo bài test từ câu hỏi.	validateMiniTestReferences -> transaction MiniTests + MiniTestItems -> add/remove item kiểm tra ownership -> return miniTest	FUNCTION creatorMiniTestService(data,userId) BEGIN TRANSACTION validateMiniTestReferences(topicId,quest ionIds,userId) INSERT/UPDATE MiniTests(ContentStatus='Draft') INSERT/DELETE MiniTestItems COMMIT RETURN miniTest END FUNCTION
CreatorService.submitForReview <i>services/creator.service.js</i> Mục đích: Gửi nội dung sang admin duyệt.	Kiểm tra entity thuộc creator -> status phải Draft/Rejected -> update PendingReview -> trả entity	FUNCTION submitForReview(table,idColumn,id,userId) entity = SELECT WHERE id AND CreatedByUserID=userId IF không tồn tại THEN throw 404 IF status không thuộc Draft/Rejected THEN throw error UPDATE entity SET ContentStatus='PendingReview' RETURN updated entity END FUNCTION

8. Module Review - Kiểm duyệt nội dung

Mục đích	Reviewer/Admin duyệt, từ chối, lưu trữ và xem log nội dung.
Thành phần	review.routes.js; ReviewController; ReviewService; ContentReviewLogs; Topics, Words, Questions, MiniTests.
Luồng module	Reviewer -> /api/review -> verifyToken -> REVIEW_CONTENT permission -> ReviewController -> ReviewService -> SQL transaction -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
ReviewController.getPending <i>controllers/review.controller.js</i> Mục đích: Trả danh sách nội dung chờ duyệt.	GET /pending -> check REVIEW_CONTENT -> gọi ReviewService.getPendingContent t -> trả danh sách	FUNCTION getPending(req,res,next) TRY data = ReviewService.getPendingContent() res.status(200).json(data) CATCH error next(error) END TRY END FUNCTION
ReviewController.approve reject	POST/GET entityType/entityId -> lấy params + adminId -> gọi ReviewService -> trả success/logs	FUNCTION reviewActionController(req,res,next) TRY entityType = req.params.entityType

archive logs <i>controllers/review.controller.js</i> Mục đích: Điều phối hành động duyệt.		<pre> entityId = req.params.entityId IF approve THEN result = ReviewService.approve(entityType,entityId,adminId) IF reject THEN result = ReviewService.reject(entityType,entityId,adminId,reason) IF archive THEN result = ReviewService.archive(entityType,entityId,adminId) IF logs THEN result = ReviewService.getReviewLogs(entityType,entityId) res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>
---	--	---

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
ReviewService.getPendingContent <i>services/review.service.js</i> Mục đích: Gộp nội dung PendingReview.	<pre> UNION Topics/Words/Questions/MiniTests -> JOIN Users creator -> order by createdAt -> return list </pre>	<pre> FUNCTION getPendingContent() SELECT PendingReview FROM Topics UNION ALL SELECT PendingReview FROM Words UNION ALL SELECT PendingReview FROM Questions UNION ALL SELECT PendingReview FROM MiniTests ORDER BY createdAt ASC RETURN recordset END FUNCTION </pre>
ReviewService.approve reject archive <i>services/review.service.js</i> Mục đích: Cập nhật trạng thái nội dung.	<pre> Resolve entityType -> begin transaction -> update status -> update reviewed fields -> insert ContentReviewLogs -> commit </pre>	<pre> FUNCTION reviewStatusAction(entityType,entityId,adminId,action) e = resolveEntity(entityType) BEGIN TRANSACTION UPDATE e.table SET ContentStatus = actionStatus IF MiniTest THEN update IsPublished INSERT ContentReviewLogs(oldStatus,newStatus,comment) COMMIT RETURN true END FUNCTION </pre>
ReviewService.getReviewLogs <i>services/review.service.js</i> Mục đích: Lấy lịch sử kiểm duyệt.	<pre> Resolve entity -> query ContentReviewLogs -> join Users actionBy -> order desc </pre>	<pre> FUNCTION getReviewLogs(entityType,entityId) e = resolveEntity(entityType) logs = SELECT ContentReviewLogs WHERE EntityType=e.type AND EntityID=entityId JOIN Users reviewer ORDER BY CreatedAt DESC RETURN logs END FUNCTION </pre>

9. Module AI - Hỗ trợ soạn nội dung TOEIC

Mục đích	Gợi ý nghĩa, ví dụ, bản dịch và dữ liệu từ điển cho nội dung TOEIC.
Thành phần	ai.routes.js; AiController; AiService; AI provider/dictionary.
Luồng module	Admin/Creator -> /api/ai -> verifyToken/permission -> AiController -> AiService -> AI provider -> parse/normalize -> response.

Controller

Hàm Controller	Luồng xử lý	Giả mã
AiController.suggestWordContent <i>controllers/ai.controller.js</i> Mục đích: Nhận yêu cầu gợi ý từ vựng.	<pre> POST /suggest-word-content -> validate quyền/body -> gọi AiService.suggestWordContent -> trả suggestion </pre>	<pre> FUNCTION suggestWordContent(req,res,next) TRY payload = req.body result = AiService.suggestWordContent(payload) res.status(200).json(result) CATCH error </pre>

		<pre> next(error) END TRY END FUNCTION </pre>
--	--	---

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
AIService.suggestWordContent <i>services/ai.service.js</i> Mục đích: Sinh gợi ý đầy đủ cho một từ.	Clean input -> tạo prompt TOEIC -> gọi AI withTimeout -> extract response text -> parse JSON -> normalize examples	<pre> FUNCTION suggestWordContent(payload) term = cleanText(payload.term) prompt = build TOEIC prompt response = call AI provider with timeout text = extractResponseText(response) data = parseJsonText(text) data.examples = normalizeExamples(data.examples) RETURN data END FUNCTION </pre>
generateToEICExamples translate lookupDictionary <i>services/ai.service.js</i> Mục đích: Các hàm hỗ trợ AI.	generate: prompt ví dụ TOEIC translate: prompt dịch tiếng Việt lookup: tra cứu dictionary -> normalize output	<pre> FUNCTION aiHelpers(action,input) IF generateExamples THEN call AI and normalize examples IF translate THEN call AI translate and return Vietnamese text IF lookupDictionary THEN query dictionary source RETURN normalized result END FUNCTION </pre>

10. Module Report & Notification - Báo cáo và thông báo

Mục đích	Người học gửi báo cáo; admin xử lý báo cáo và gửi thông báo/lời nhắc học.
Thành phần	UserController.createReport; ReportService; AdminController report/notification handlers; AdminService notification methods; Notifications, Reports.
Luồng module	Learner -> createReport -> ReportService -> DB. Admin -> reports/notifications routes -> AdminController -> ReportService/AdminService -> DB -> User đọc notifications.

Controller

Hàm Controller	Luồng xử lý	Giả mã
UserController.createReport <i>controllers/user.controller.js</i> Mục đích: Nhận báo cáo từ learner.	POST report -> lấy userId + body -> gọi ReportService.createReport -> trả report	<pre> FUNCTION createReport(req,res,next) TRY report = ReportService.createReport(req.user.id,r eq.body) res.status(201).json(report) CATCH error next(error) END TRY END FUNCTION </pre>
AdminController.getReports updateReport <i>controllers/admin.controller.js</i> Mục đích: Admin xem/cập nhật report.	GET/PATCH /reports -> check MANAGE_REPORTS -> gọi ReportService -> trả danh sách/report mới	<pre> FUNCTION adminReportController(req,res,next) TRY IF GET THEN result = ReportService.getReports(page,limit,filt ers) IF PATCH THEN result = ReportService.updateReport(id,body,admin Id) res.json(result) CATCH error next(error) END TRY END FUNCTION </pre>
Notification controllers <i>controllers/admin.controller.js;</i> <i>controllers/user.controller.js</i> Mục đích: Admin gửi, user đọc thông báo.	Admin POST announcement/reminders User GET/PUT notifications -> gọi service tương ứng -> trả success/list	<pre> FUNCTION notificationController(req,res,next) TRY IF adminSend THEN sendAnnouncement(body) IF dailyReminder THEN createDailyReminders() IF userGet THEN getUserNotifications(userId) IF markRead THEN </pre>

		<pre>markNotificationRead(userId,id) res.json(result) CATCH error next(error) END TRY END FUNCTION</pre>
--	--	--

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
ReportService.createReport getReports updateReport <i>services/report.service.js</i> Mục đích: Lưu và xử lý báo cáo.	create: validate/infer entity -> insert get: filter/paginate update: đổi status/priority/note	<pre>FUNCTION reportService(action) ensureSchema() IF create THEN INSERT Report(userId,entityType,content,status) IF get THEN SELECT Reports WHERE filters OFFSET/FETCH IF update THEN UPDATE Reports SET status,note,adminId RETURN report/result END FUNCTION</pre>
AdminService.sendAnnouncement createDailyReminders <i>services/admin.service.js</i> Mục đích: Tạo thông báo hệ thống.	send: xác định audience -> insert notifications reminder: tìm user có từ due - > insert reminder -> audit log	<pre>FUNCTION notificationService(action) IF sendAnnouncement THEN users = query audience FOR user IN users INSERT Notifications IF createDailyReminders THEN learners = query active learners with due words FOR learner INSERT reminder notification logAdminAction RETURN count END FUNCTION</pre>
UserService notification methods <i>services/user.service.js</i> Mục đích: User đọc/đánh dấu thông báo.	Query notification theo userId -> update ReadAt cho một hoặc tất cả -> return list/success	<pre>FUNCTION userNotificationService(action,userId) IF get THEN SELECT Notifications WHERE UserID=userId IF markRead THEN UPDATE Notifications SET ReadAt=now WHERE id,userId IF markAll THEN UPDATE all unread WHERE userId RETURN result END FUNCTION</pre>

11. Module System/Common - Hạ tầng backend

Mục đích	Kết nối CSDL, validate request, xử lý lỗi, health-check, integration-test và shutdown an toàn.
Thành phần	config/db.js; validate.js; errorHandler.js; health-check.js; integration-test.js; index.js.
Luồng module	Express app -> middleware JSON/CORS -> routes -> middleware auth/validate -> controller -> service -> DB. Lỗi đi qua next(error) về errorHandler.

Controller

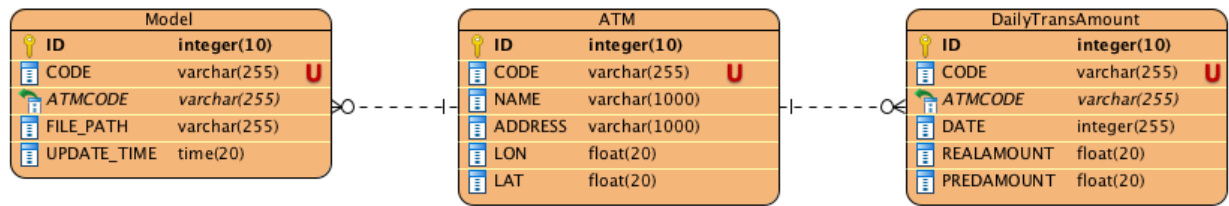
Hàm Controller	Luồng xử lý	Giả mã
Express route middleware chain <i>index.js; routes/*.js</i> Mục đích: Định tuyến request vào module.	Request vào Express -> CORS/JSON parser -> mount route theo prefix -> middleware auth/validate -> controller -> errorHandler nếu lỗi	<pre>FUNCTION requestPipeline(req,res) app.use(cors,json) app.use('/api/auth',authRoutes) app.use('/api/user',userRoutes) app.use('/api/admin',adminRoutes) IF error THEN errorHandler(err,req,res,next) END FUNCTION</pre>

Service / Middleware / Config

Hàm Service	Luồng xử lý	Giả mã
DB poolPromise <i>config/db.js</i>	Đọc biến môi trường -> tạo config mssql/msnodesqlv8 -> tạo poolPromise -> export sql,poolPromise	<pre>FUNCTION createDbPool() config = read env DB_SERVER, DB_NAME, DB_AUTH poolPromise = sql.connect(config)</pre>

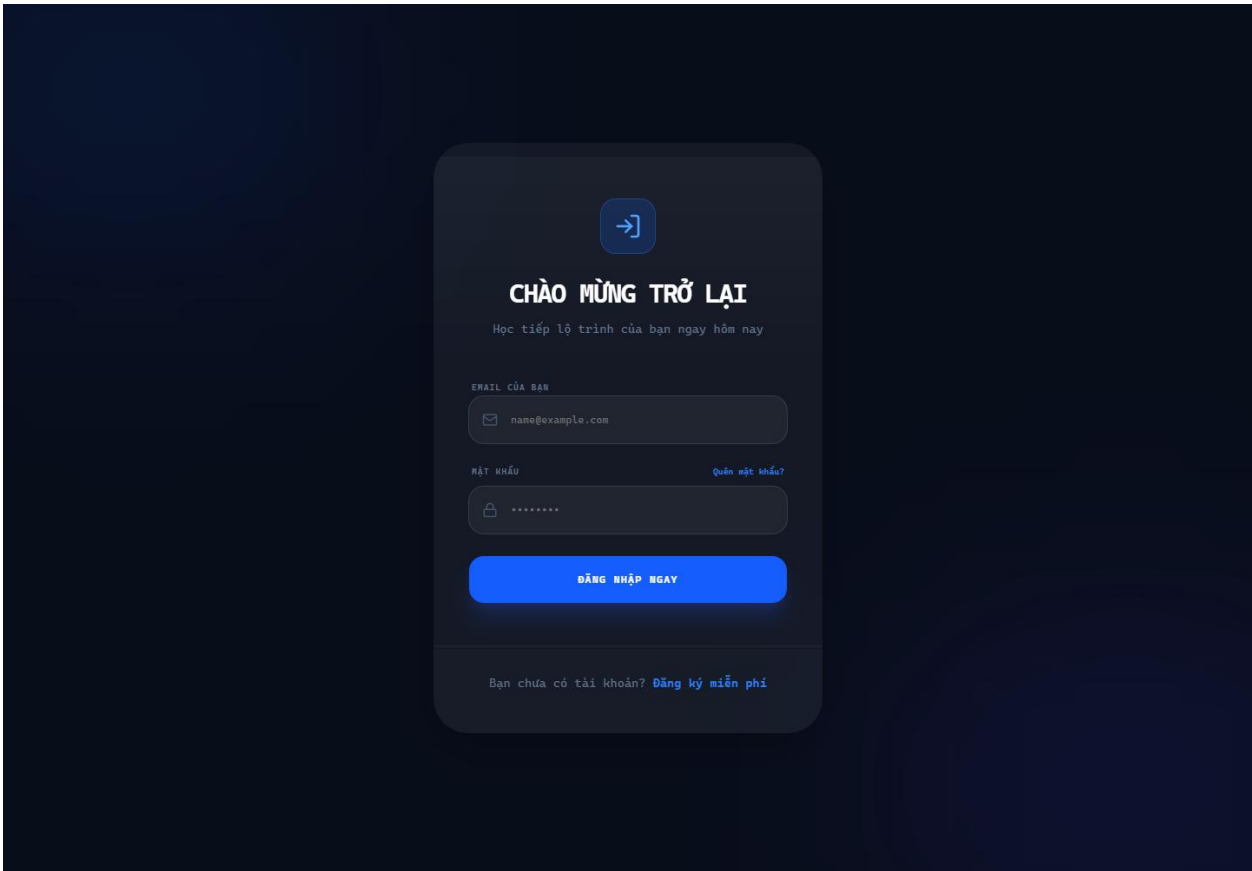
Mục đích: Chia sẻ kết nối SQL Server.		<pre>EXPORT {sql,poolPromise} END FUNCTION</pre>
validate(schema) <i>middlewares/validate.js</i> Mục đích: Validate dữ liệu đầu vào.	Middleware nhận schema -> validate req.body -> lỗi trả 400 -> hợp lệ gán value và next()	<pre>FUNCTION validate(schema) RETURN middleware(req,res,next) result = schema.validate(req.body) IF result.error THEN return 400 req.body = result.value next() END RETURN END FUNCTION</pre>
errorHandler <i>middlewares/errorHandler.js</i> Mục đích: Chuẩn hóa lỗi API.	Controller/service throw error -> next(error) -> errorHandler lấy status/message -> trả JSON lỗi	<pre>FUNCTION errorHandler(err,req,res,next) status = err.statusCode OR 500 message = err.message OR 'Internal Server Error' res.status(status).json({message}) END FUNCTION</pre>
health-check integration-test shutdown <i>config/health-check.js;</i> <i>config/integration-test.js; index.js</i> Mục đích: Kiểm tra và vận hành server.	Health: kiểm tra env + DB Integration: query schema/dữ liệu Shutdown: nhận signal -> close server	<pre>FUNCTION opsHelpers() runHealthCheck: check env, connect DB, query sys.tables runIntegrationTest: chạy các query kiểm tra tích hợp gracefulShutdown: server.close rồi process.exit END FUNCTION</pre>

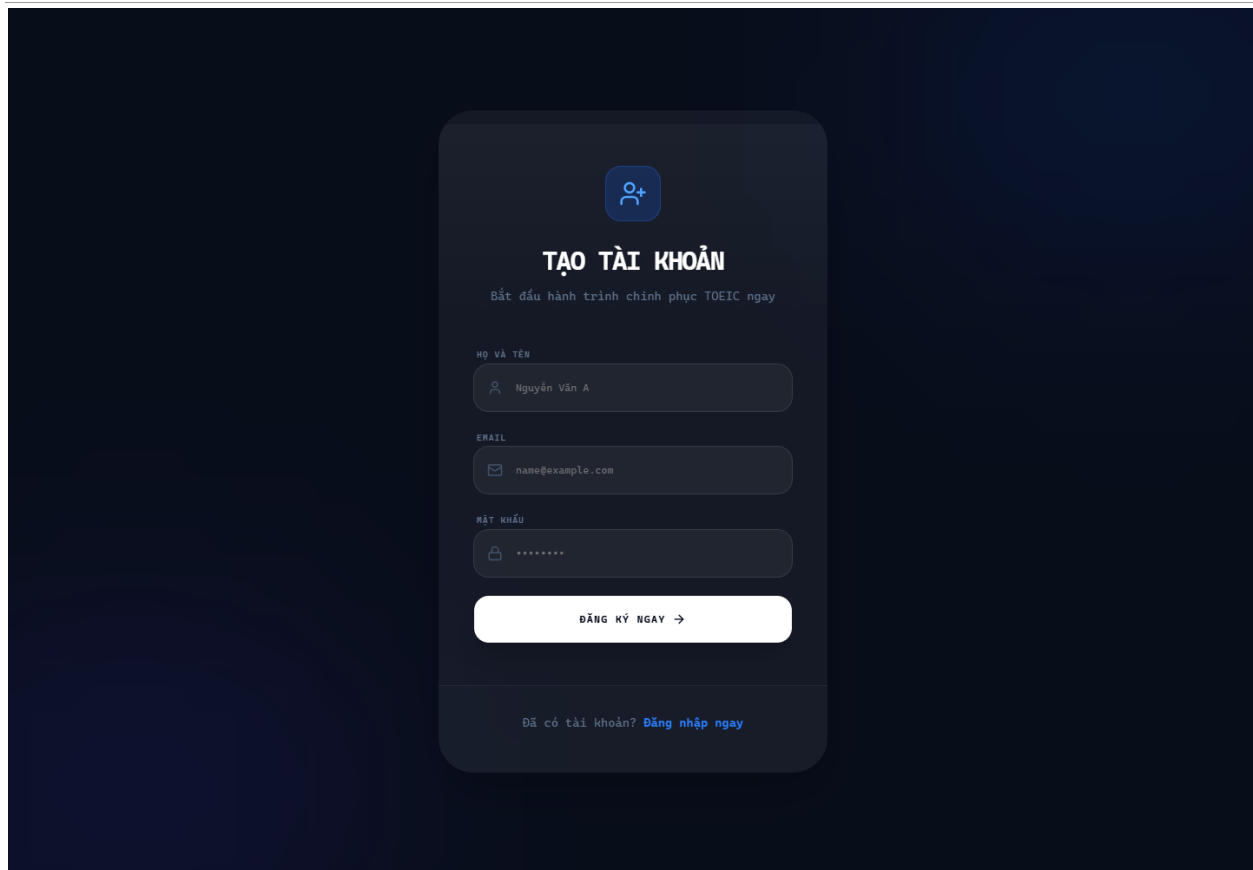
5. THIẾT KẾ DỮ LIỆU



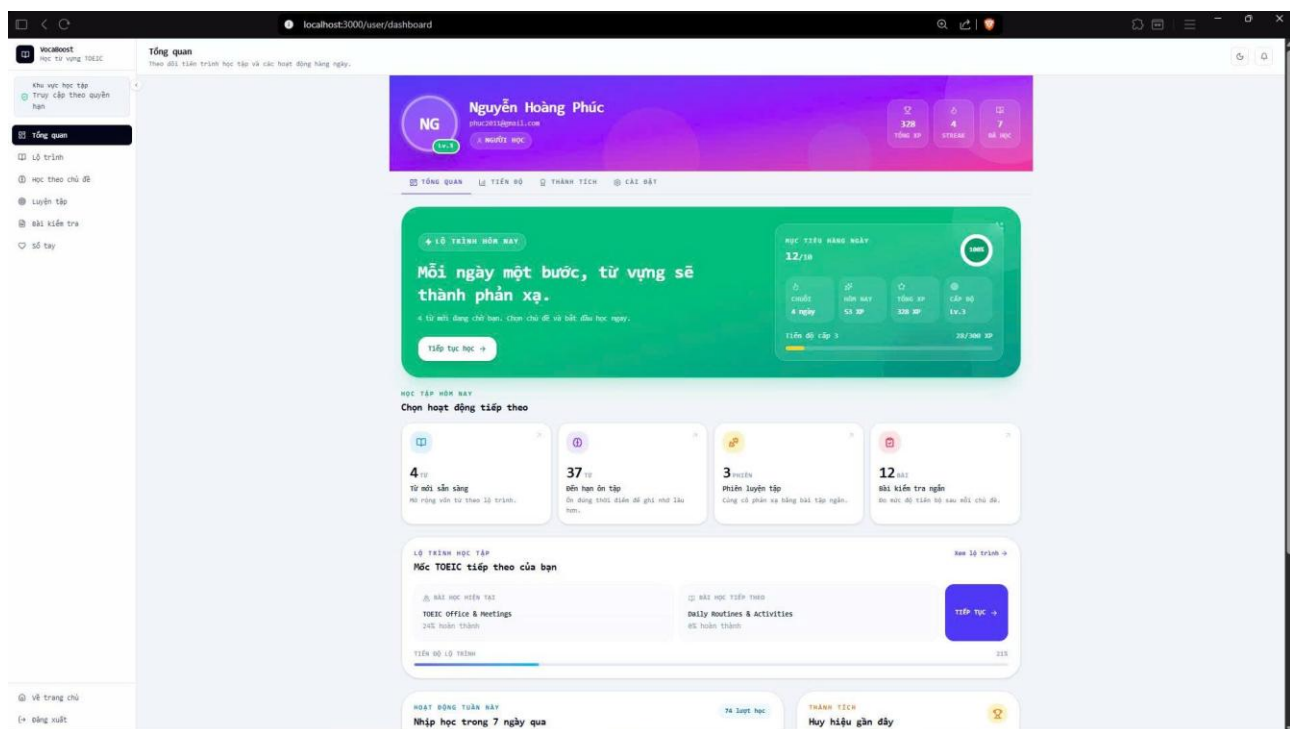
6. THIẾT KẾ GIAO DIỆN NGƯỜI DÙNG

6.1 Giao diện đăng nhập, người dùng

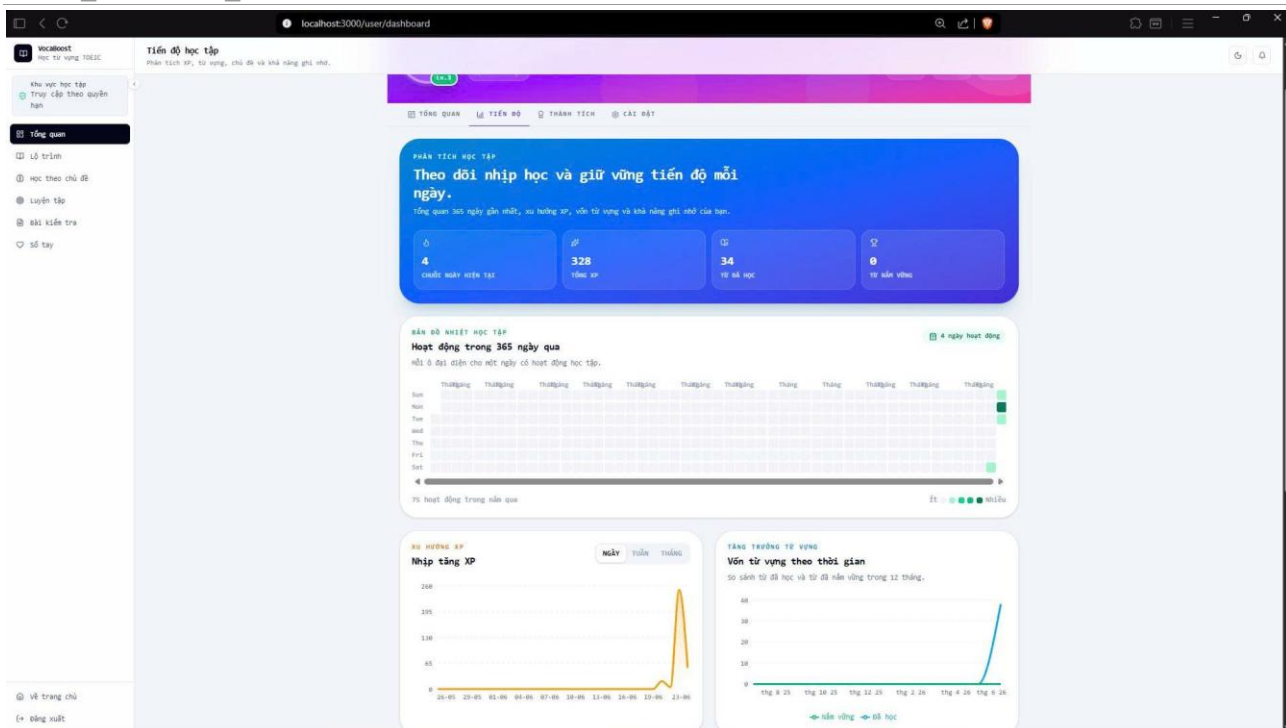




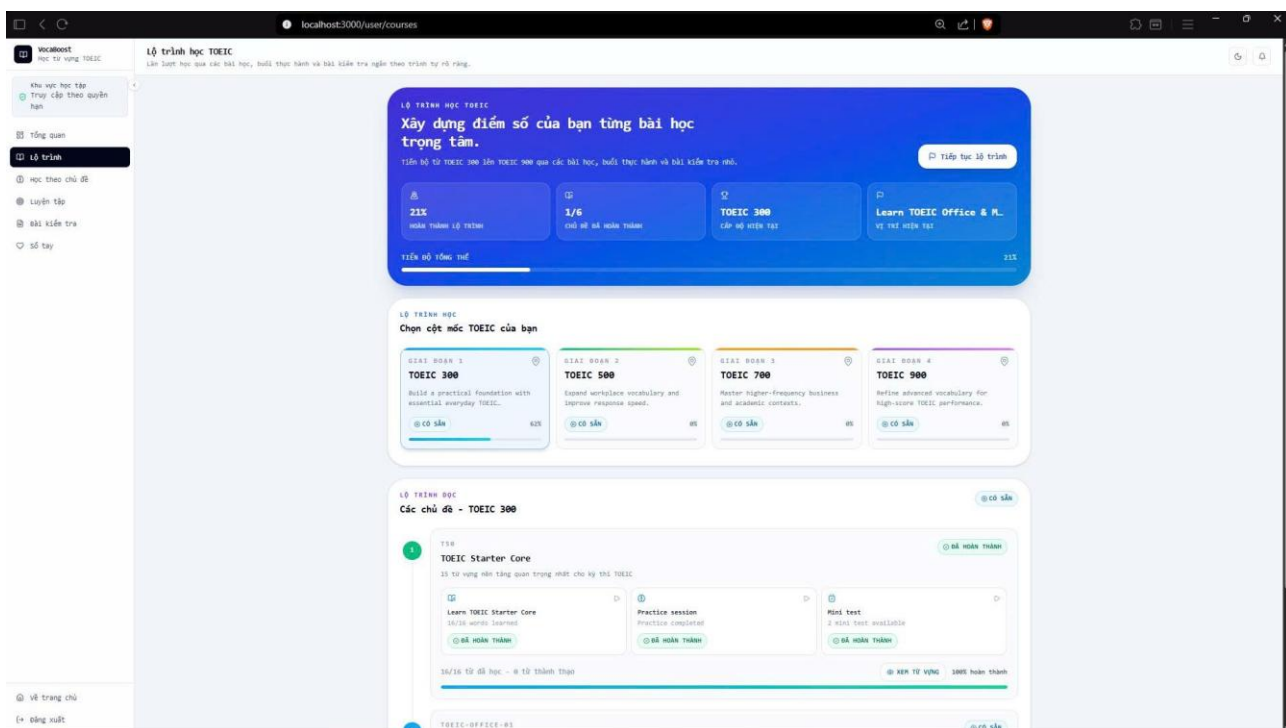
Hình 6.1.1: Giao diện đăng nhập, đăng kí hệ thống



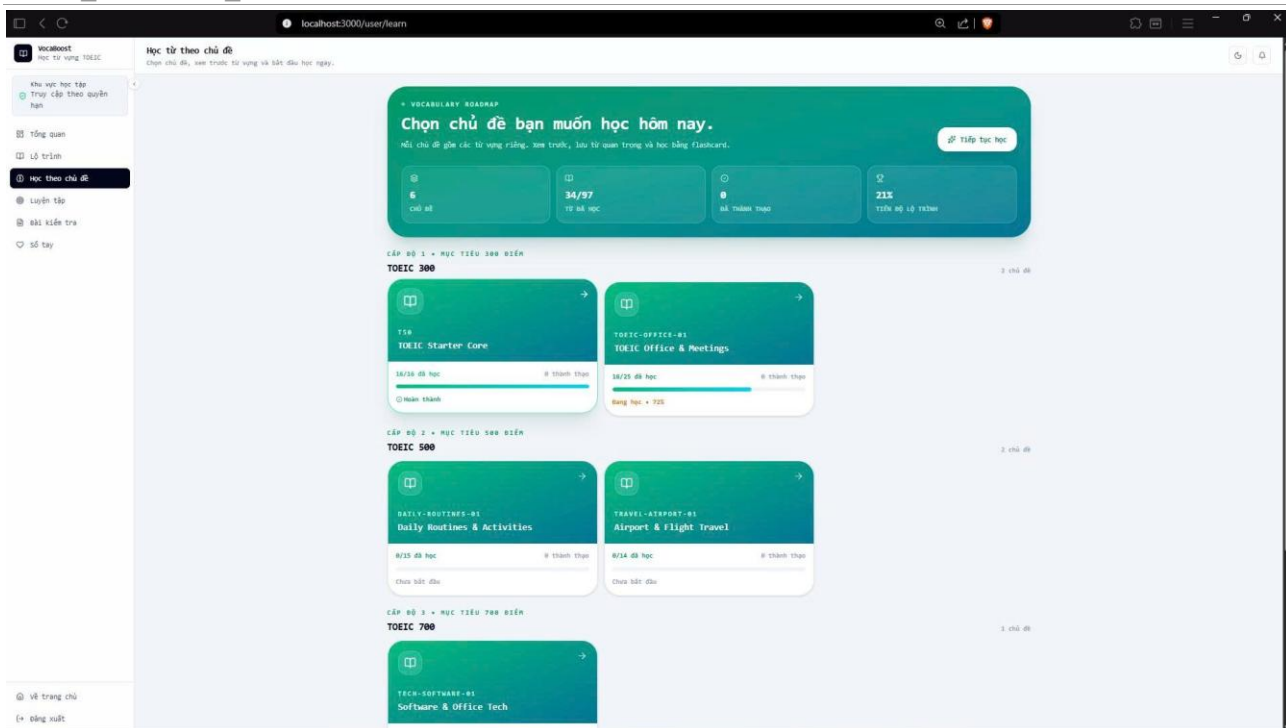
<2024_VocaBoost_VKU>: Thiết kế chi tiết



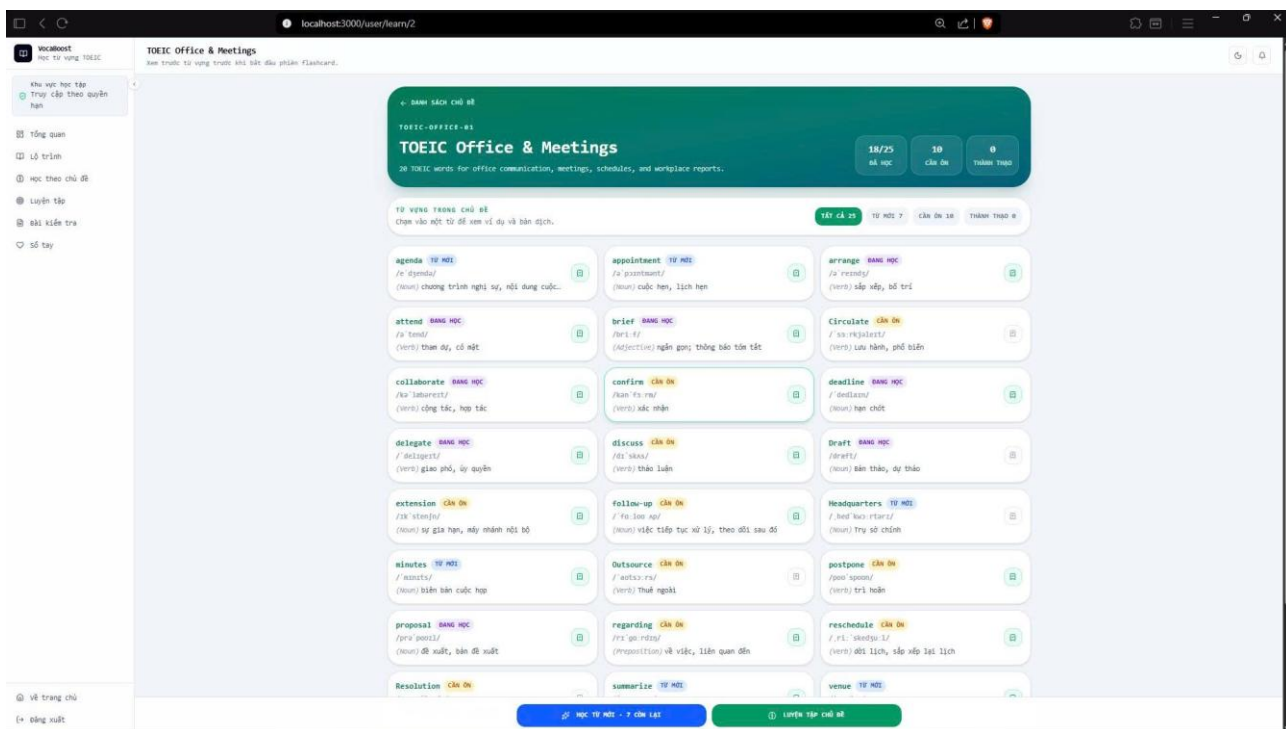
Hình 6.1.2: Giao diện bảng điều khiển học tập với mục tiêu ngày, XP, chuỗi học, hoạt động và lộ trình tiếp theo.



Hình 6.1.3: Giao diện danh sách lộ trình/chủ đề giúp học viên chọn nội dung theo thứ tự học phù hợp.

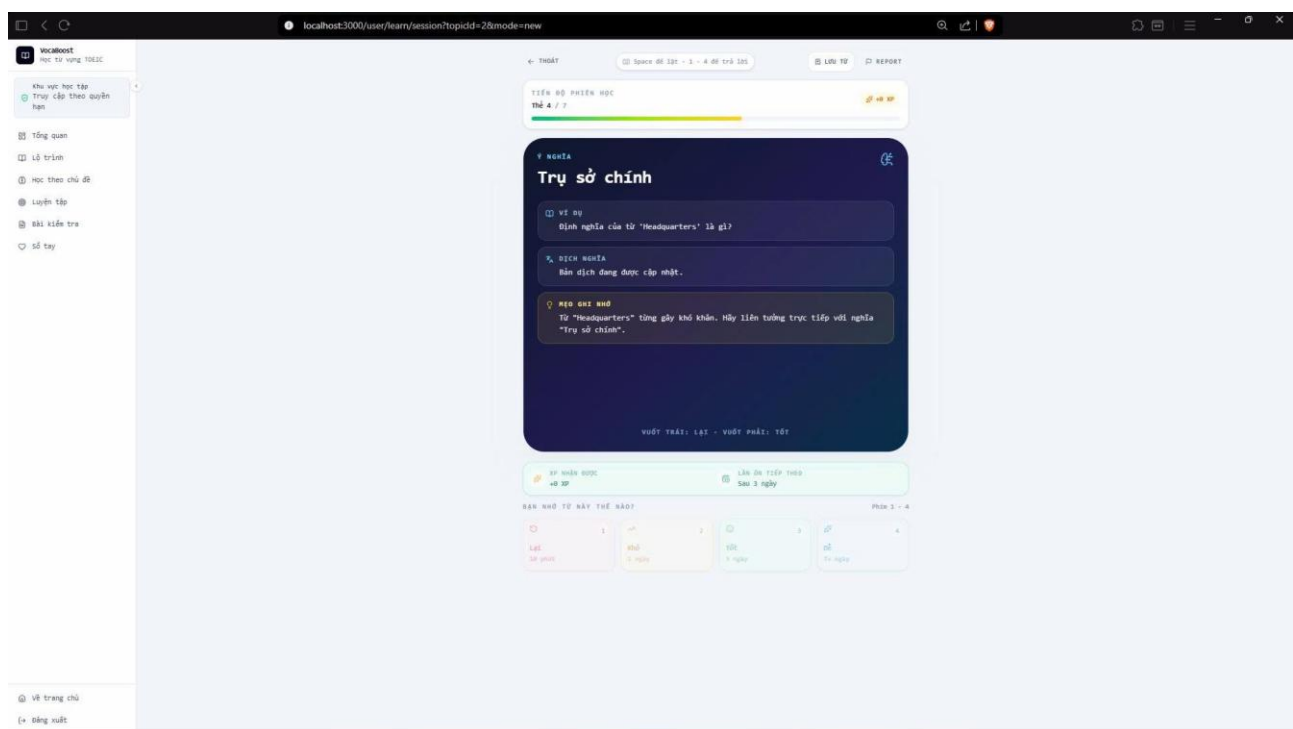
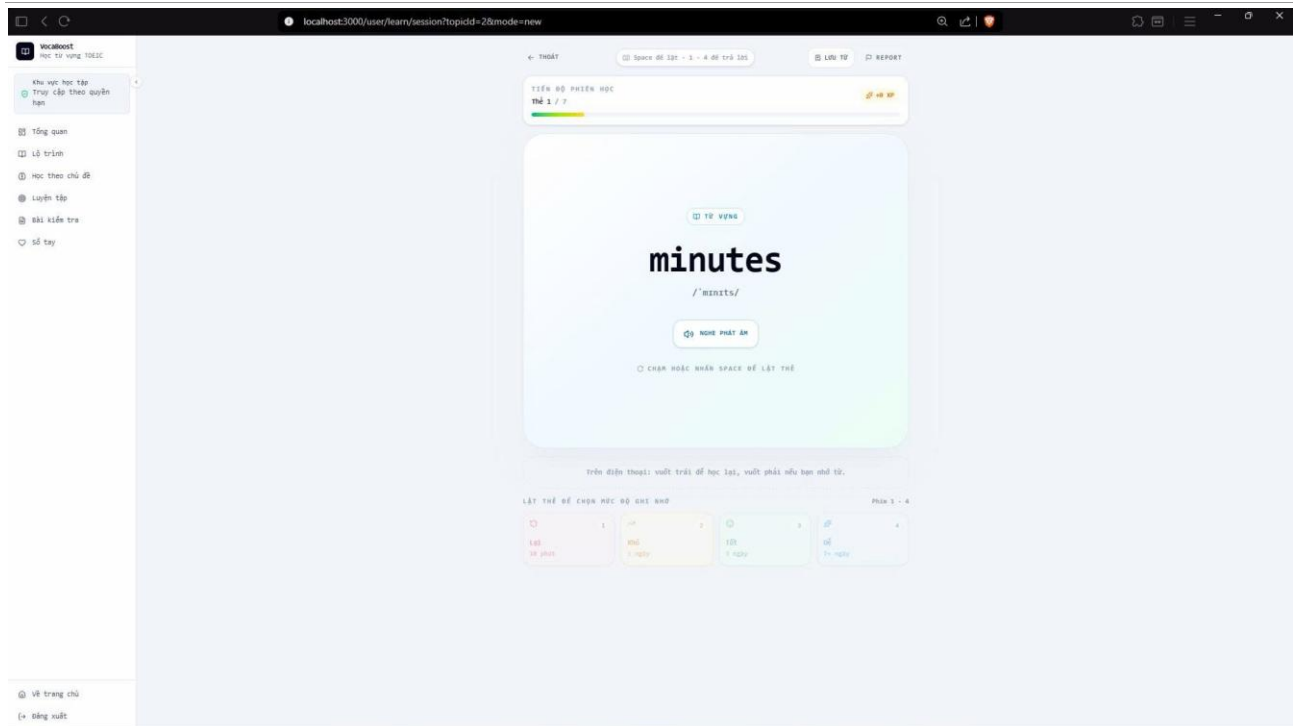


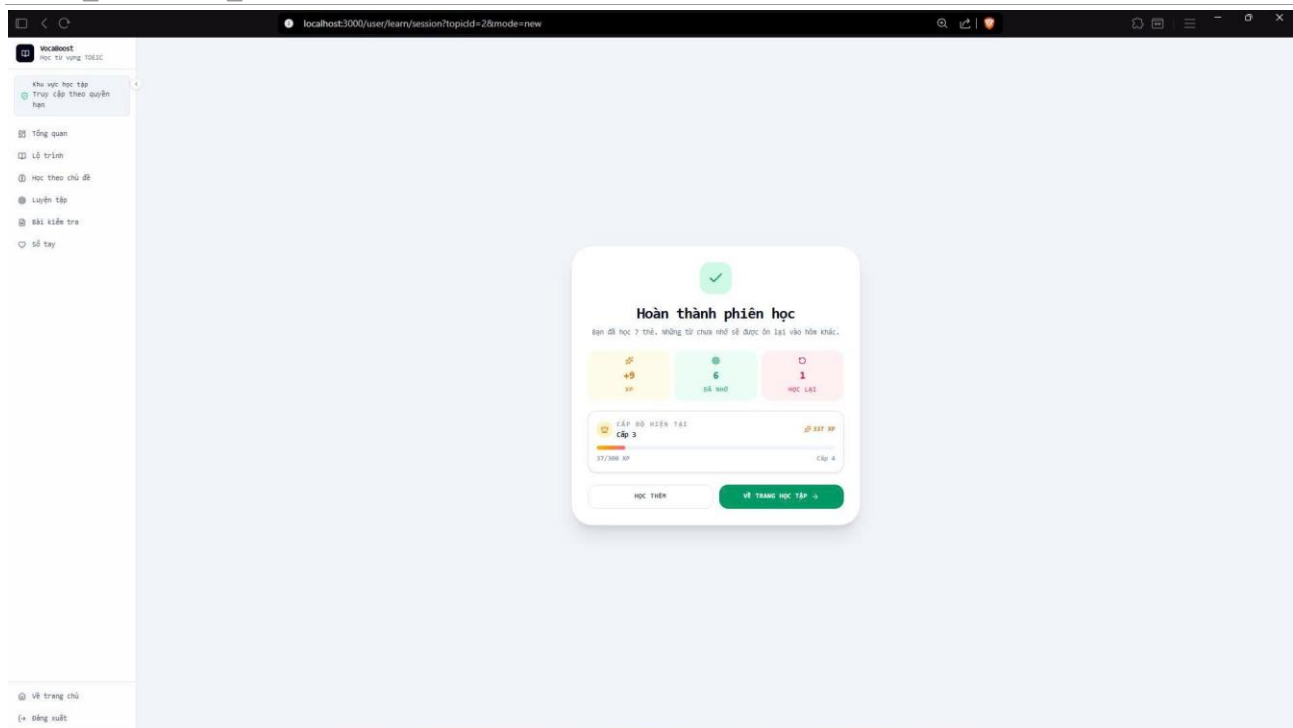
Hình 6.1.4: Khu vực chọn chủ đề và bắt đầu phiên học từ vựng bằng flashcard



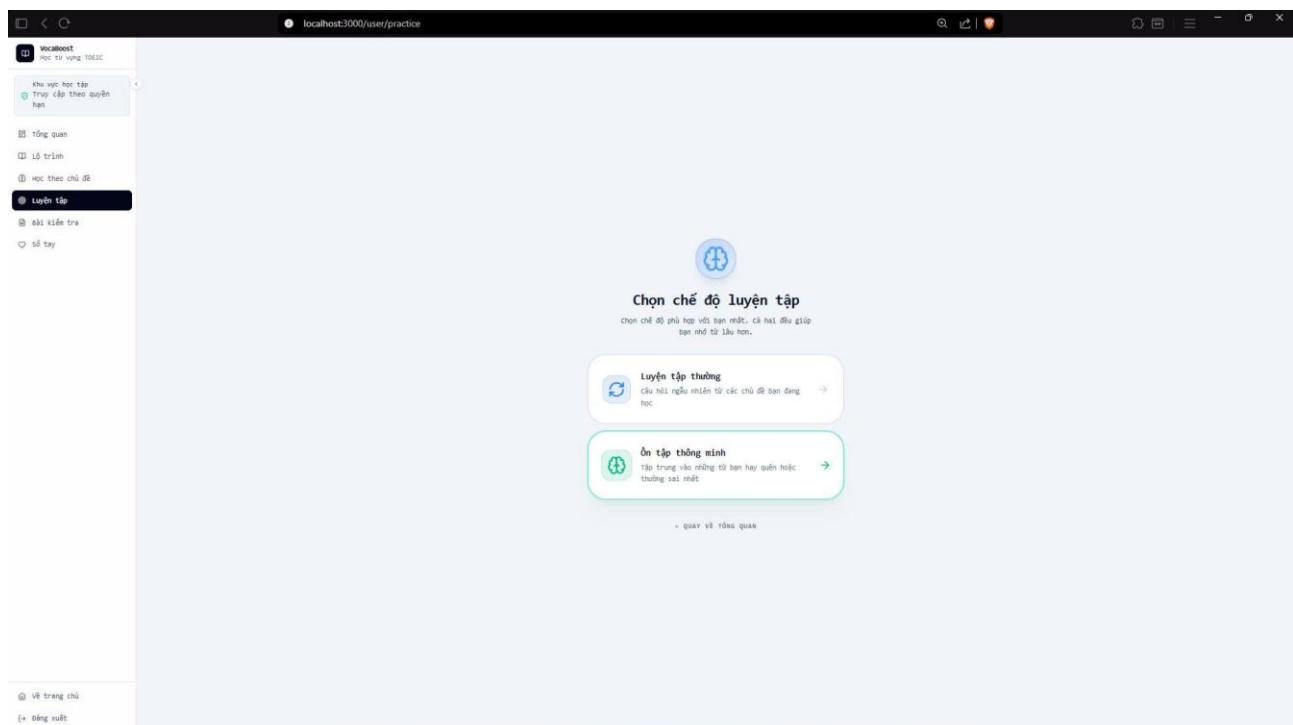
Hình 6.1.5: Giao diện xem các từ vựng trong chủ đề

<2024_VocaBoost_VKU>: Thiết kế chi tiết

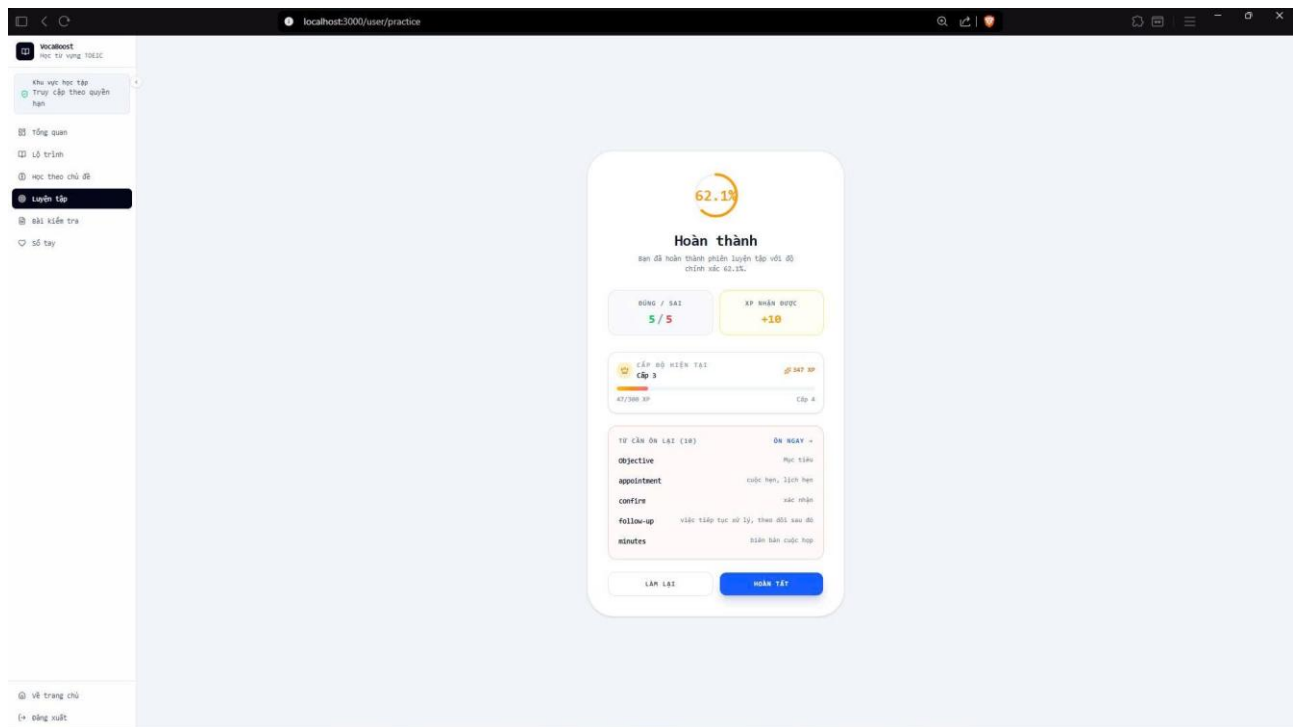
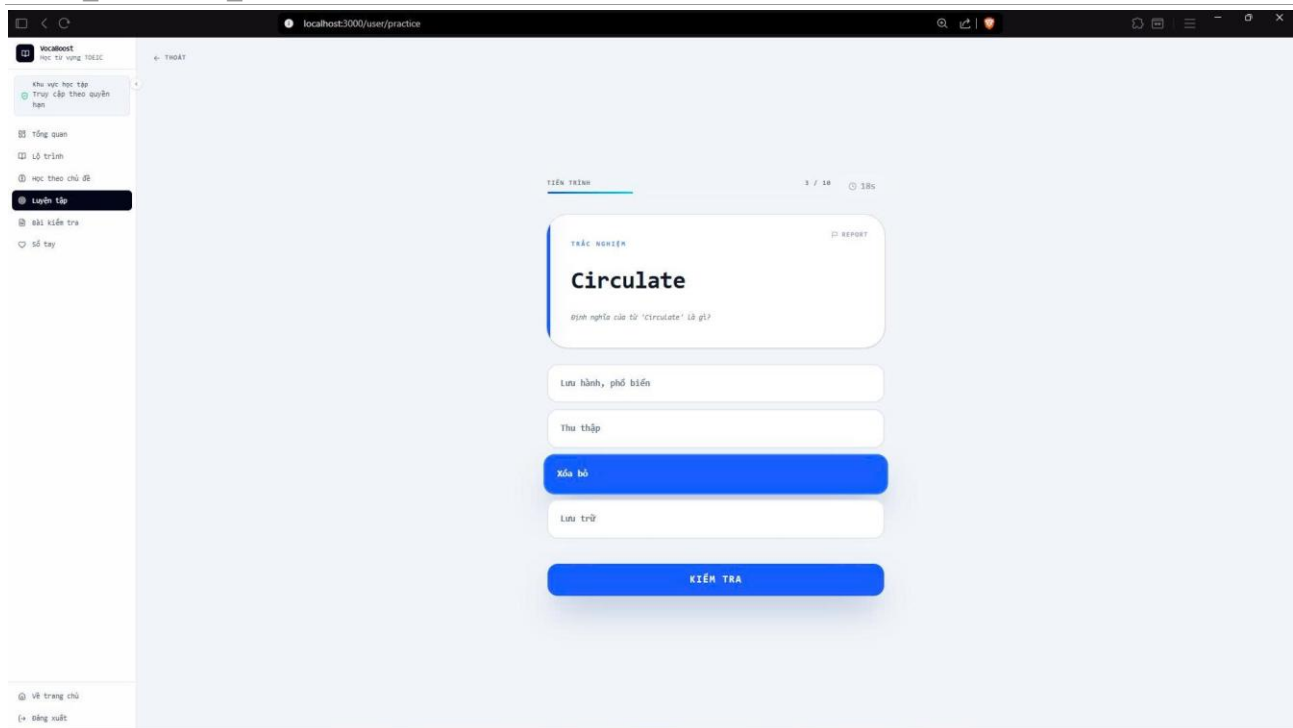




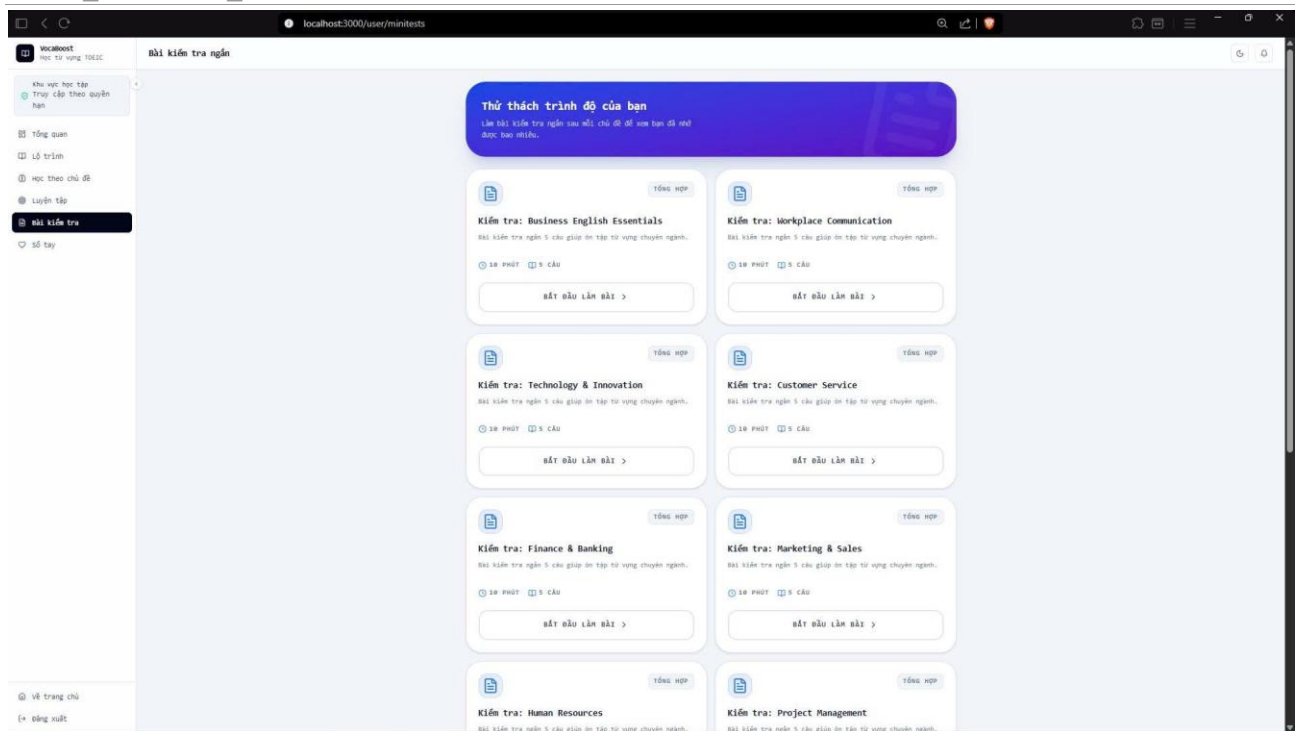
Hình 6.1.6: Giao diện học bằng flashcard



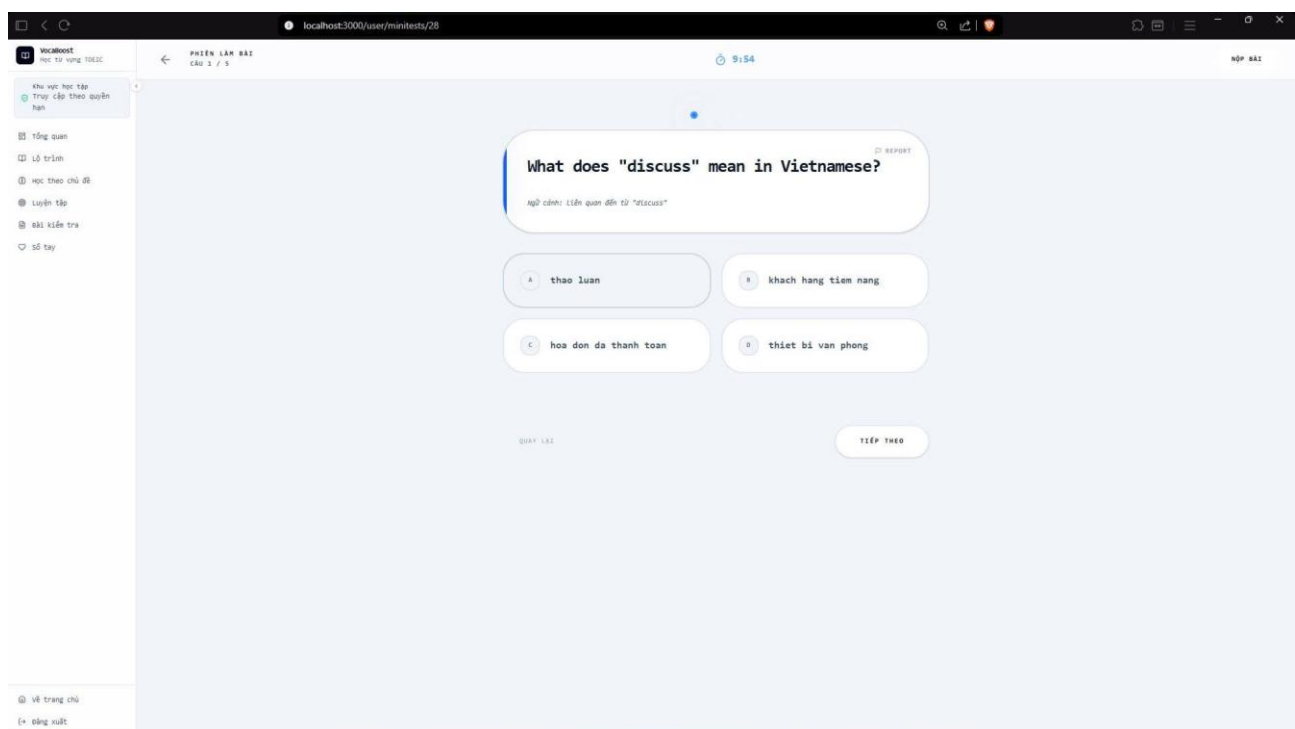
<2024_VocaBoost_VKU>: Thiết kế chi tiết



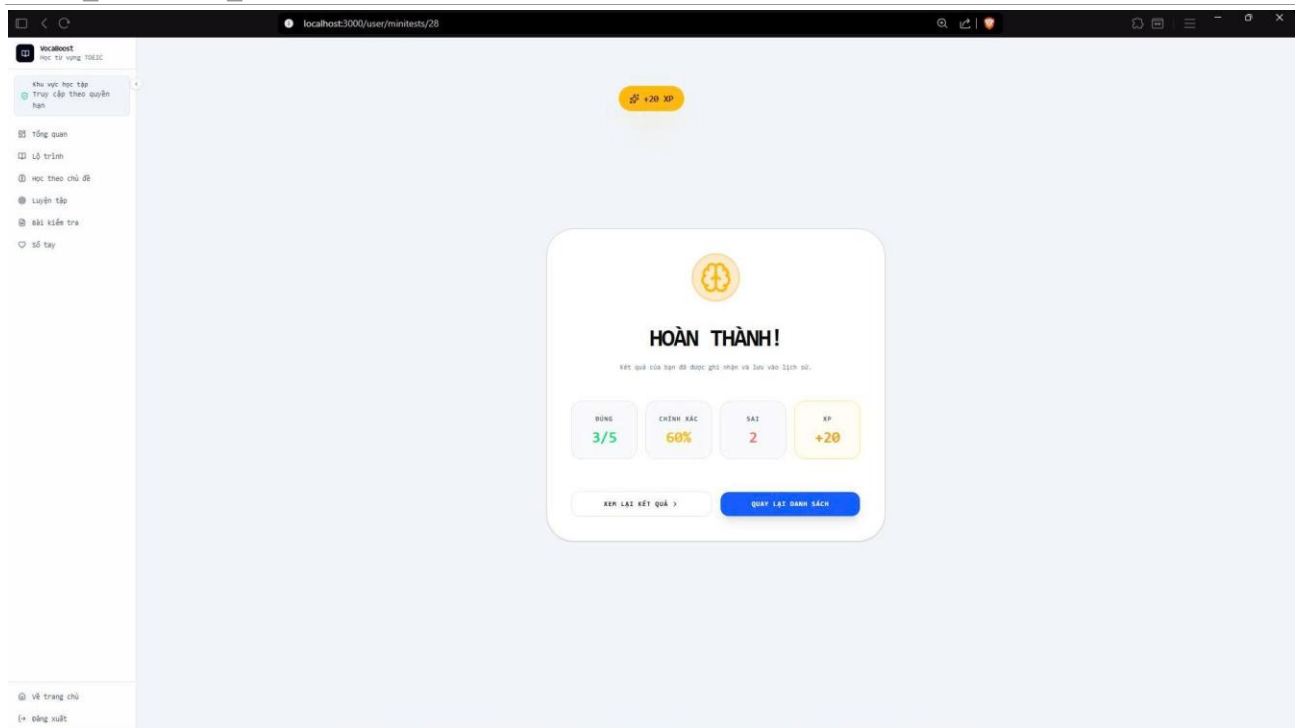
Hình 6.1.7: Màn hình chọn chế độ luyện tập và ôn tập thông minh theo dữ liệu SRS.



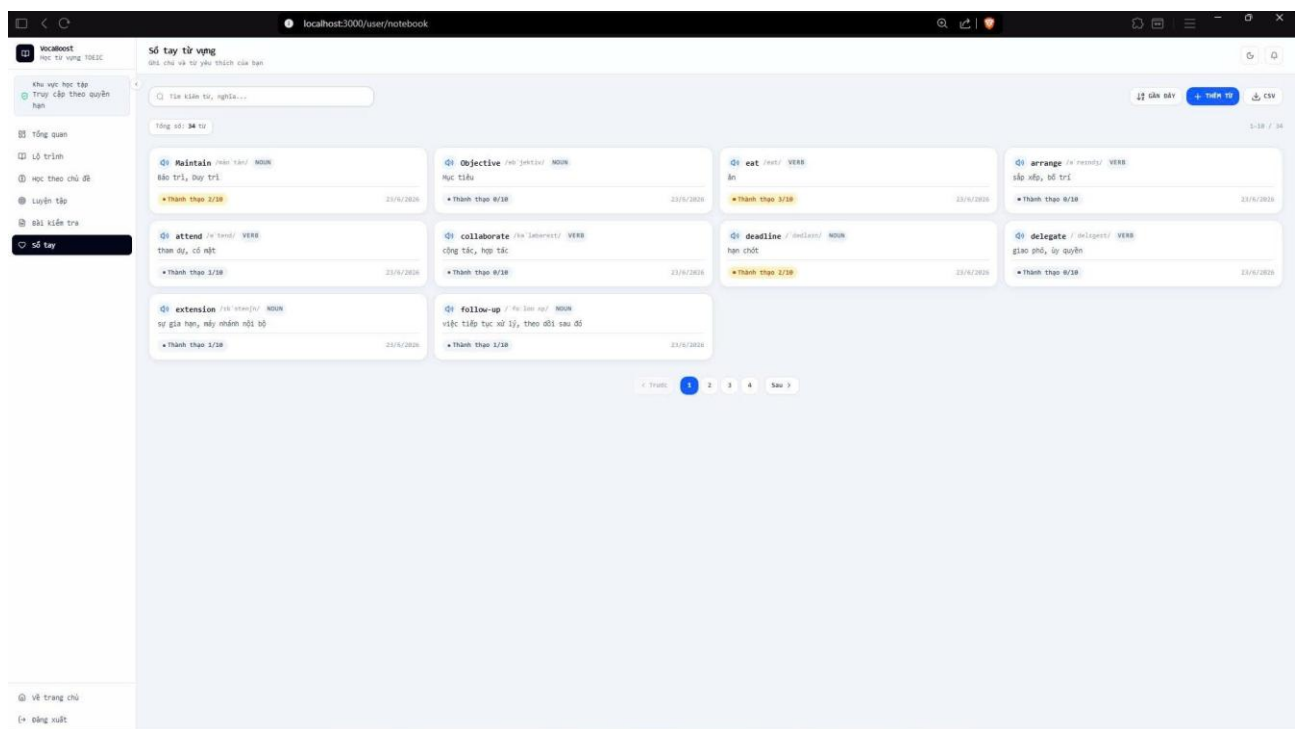
Hình 6.1.8: Danh sách mini test để đánh giá mức độ ghi nhớ và theo dõi kết quả.



<2024_VocaBoost_VKU>: Thiết kế chi tiết

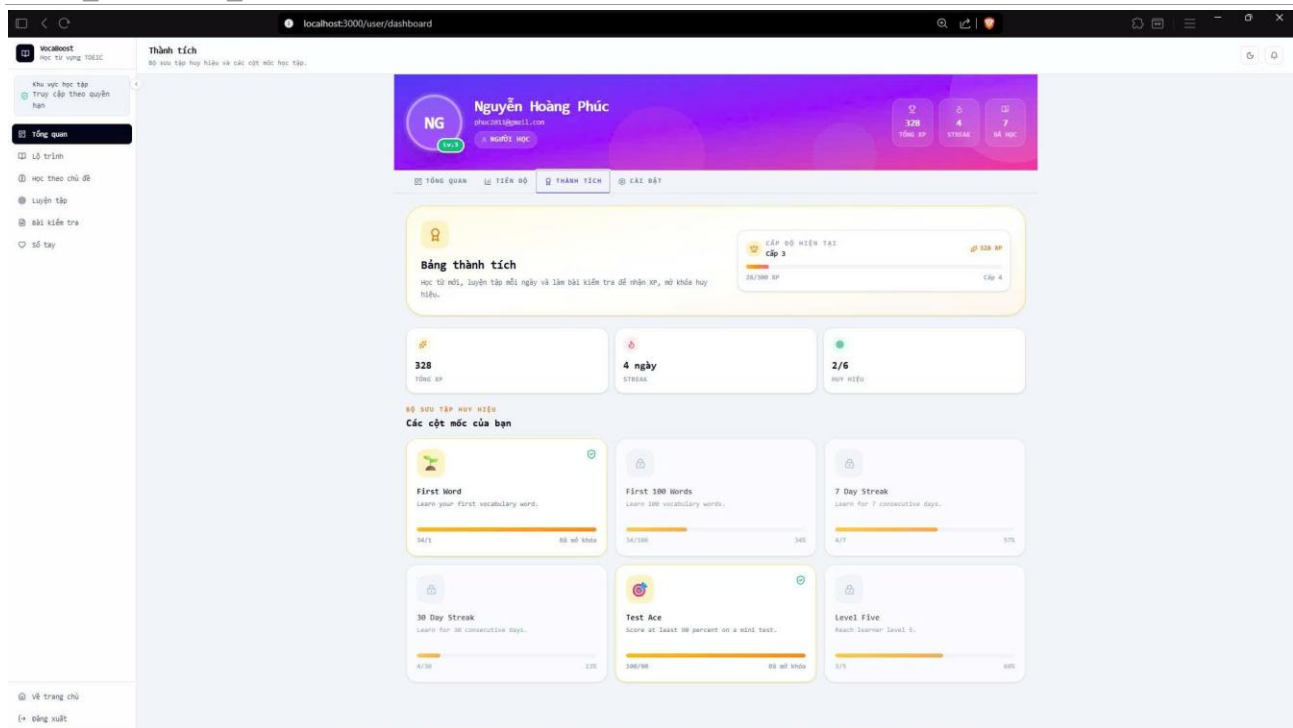


Hình 6.1.9: Giao diện luyện tập minitest



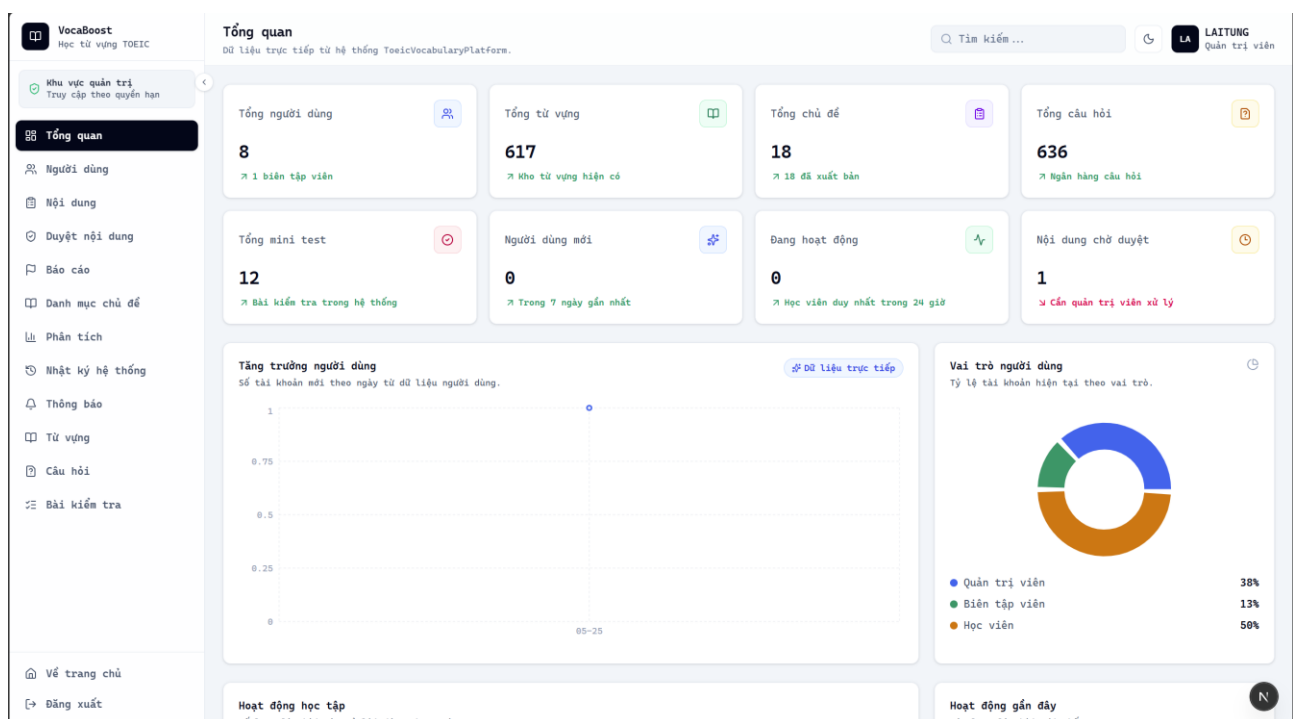
Hình 6.1.10: Nơi lưu, tìm kiếm và quản lý những từ vựng học viên muốn ghi nhớ riêng.

<2024_VocaBoost_VKU>: Thiết kế chi tiết



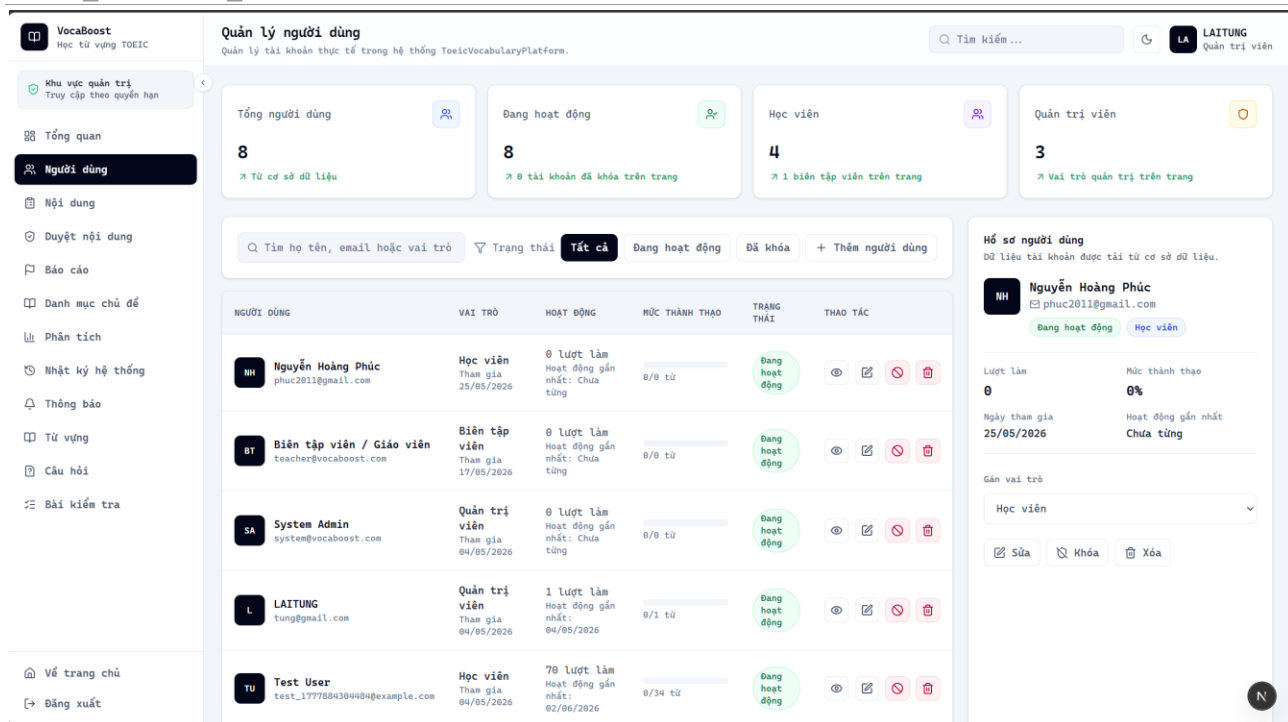
Hình 6.1.11: Các huy hiệu, cột mốc và tiến độ gamification của học viên.

6.2 Giao diện dành cho quản trị (admin)

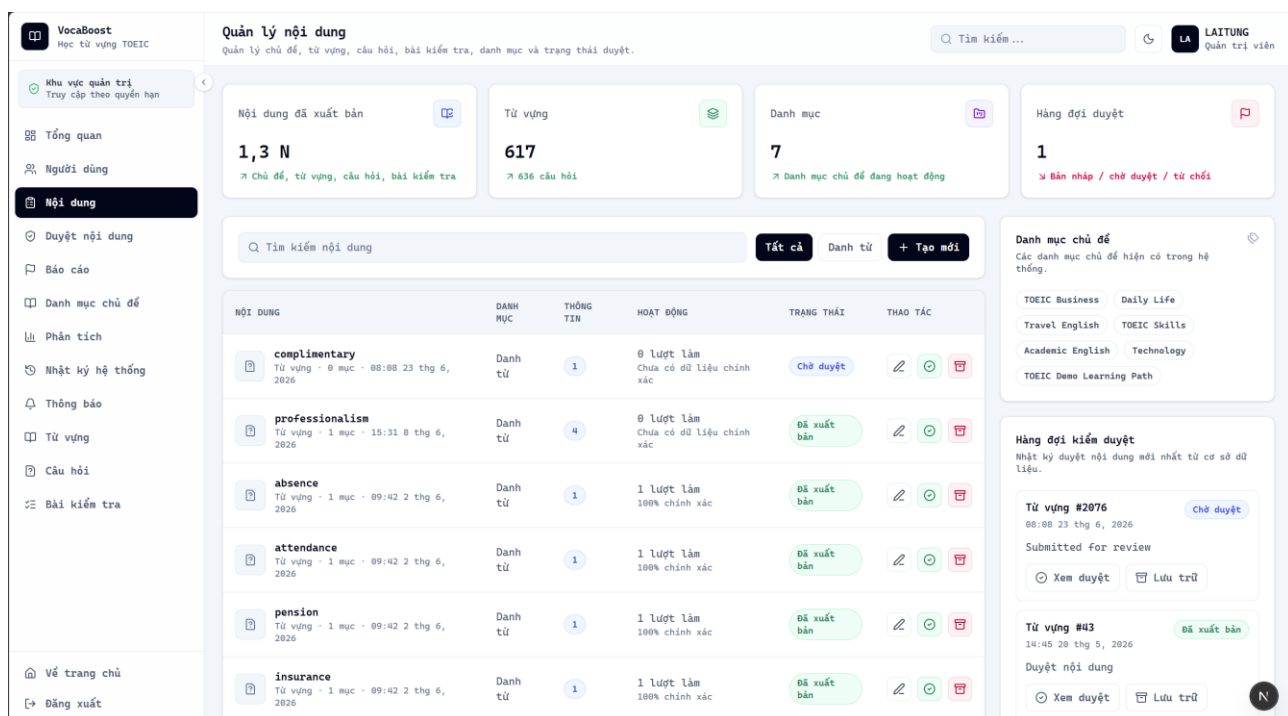


Hình 6.2.1: Dashboard hệ thống với KPI người dùng, nội dung, hoạt động học và trạng thái dịch vụ.

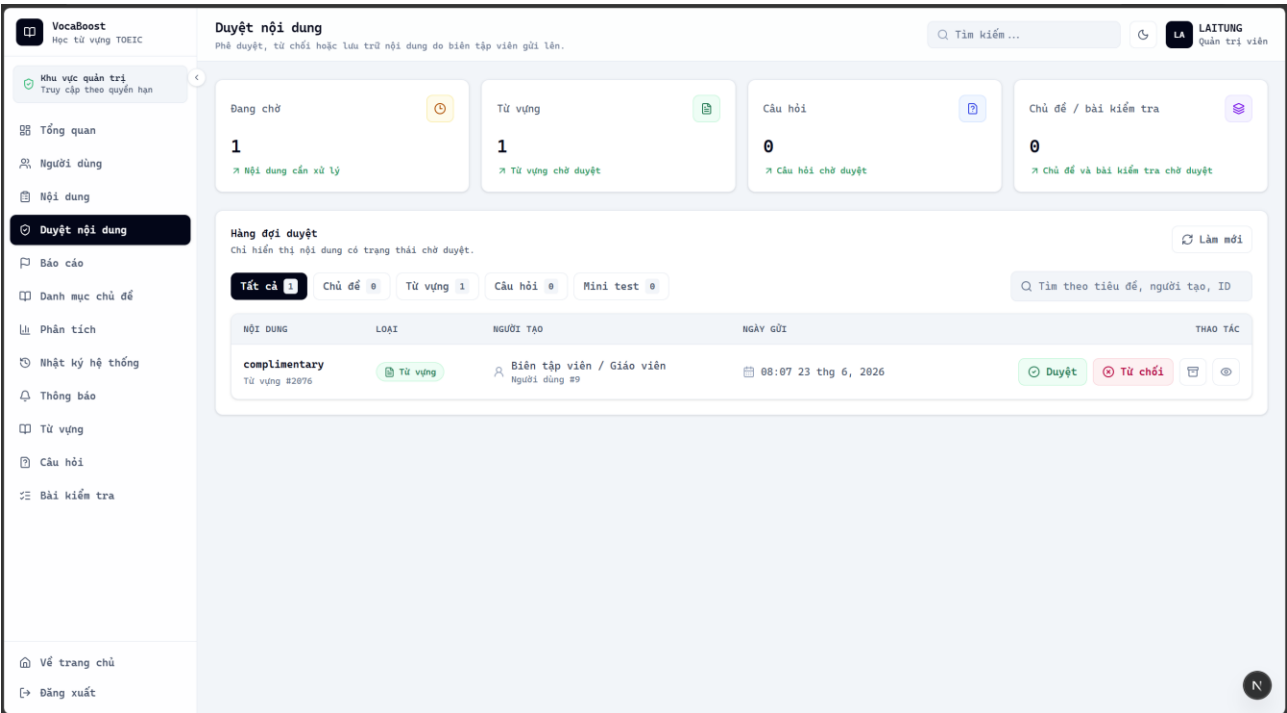
<2024_VocaBoost_VKU>: Thiết kế chi tiết



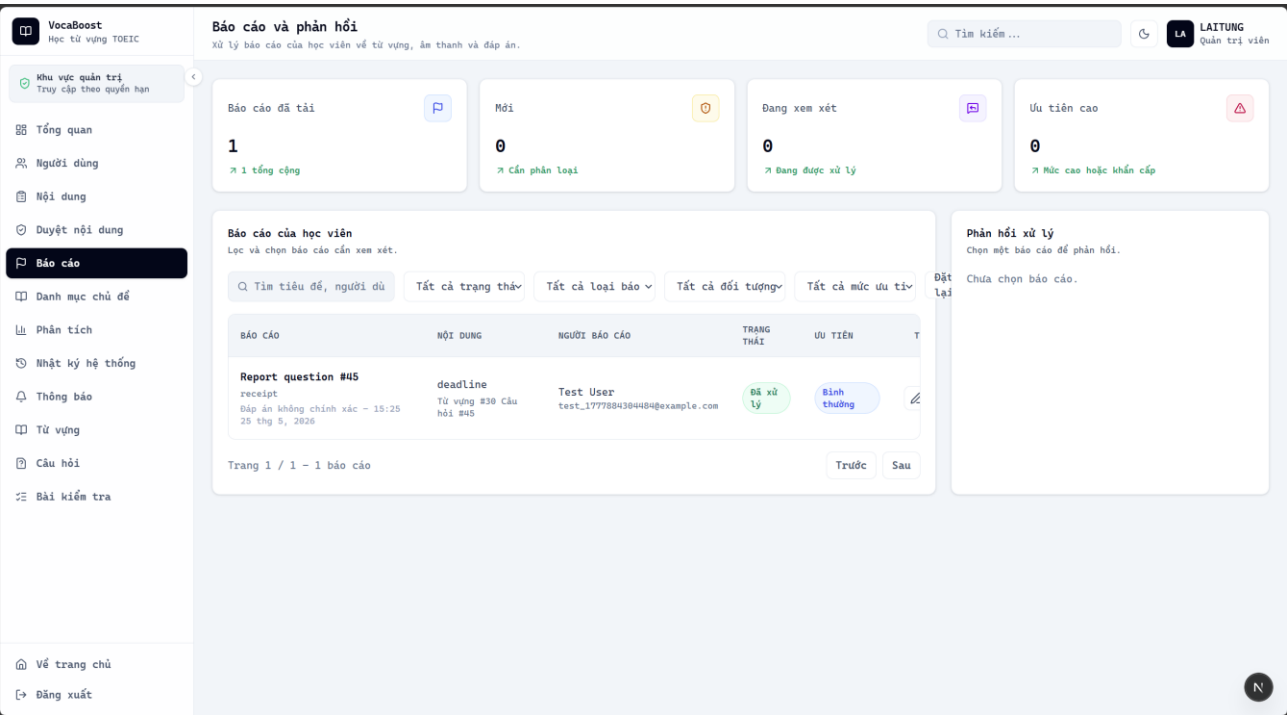
Hình 6.2.2: Danh sách tài khoản, vai trò, trạng thái hoạt động và các thao tác quản trị.



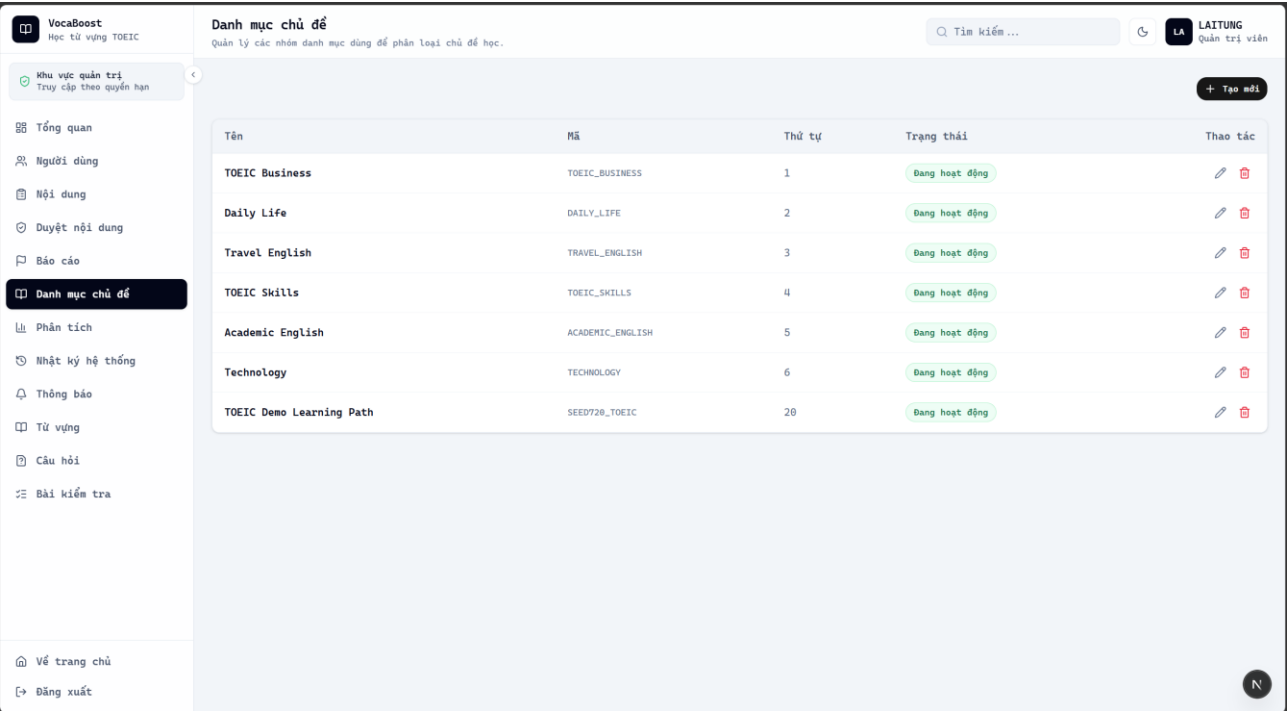
Hình 6.2.3: Giao diện tổng hợp chủ đề và nội dung học để quản trị trạng thái xuất bản.



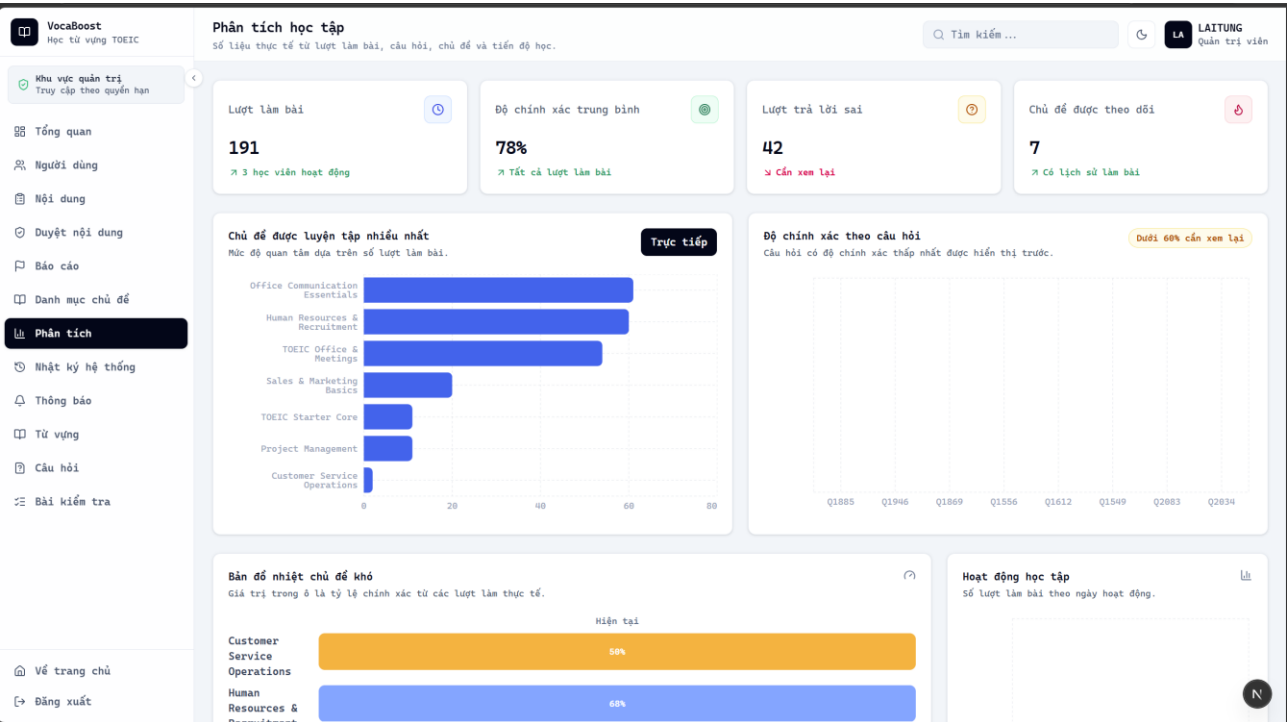
Hình 6.2.4: Giao diện hàng đợi kiểm duyệt nội dung do biên tập viên gửi lên.



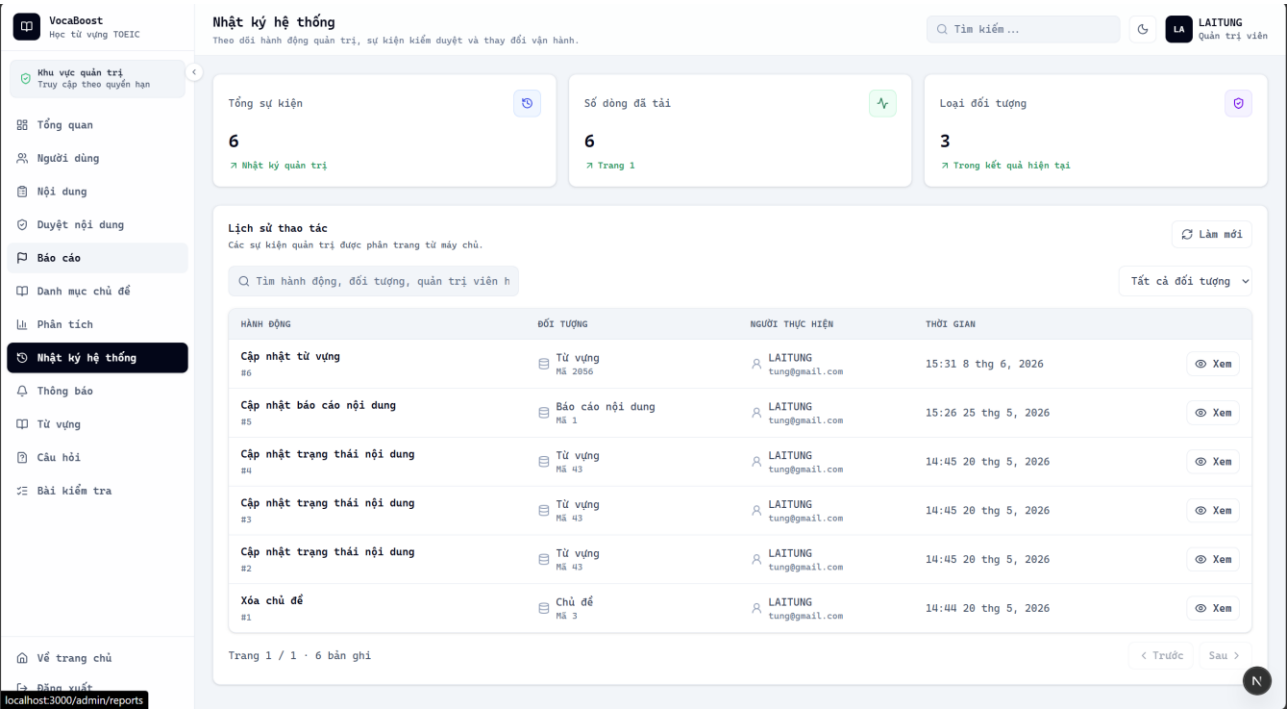
Hình 6.2.5: Giao diện danh sách phản hồi/báo lỗi của học viên và quy trình xử lý.



Hình 6.2.6: Giao diện Quản lý các nhóm danh mục dùng để phân loại chủ đề học.

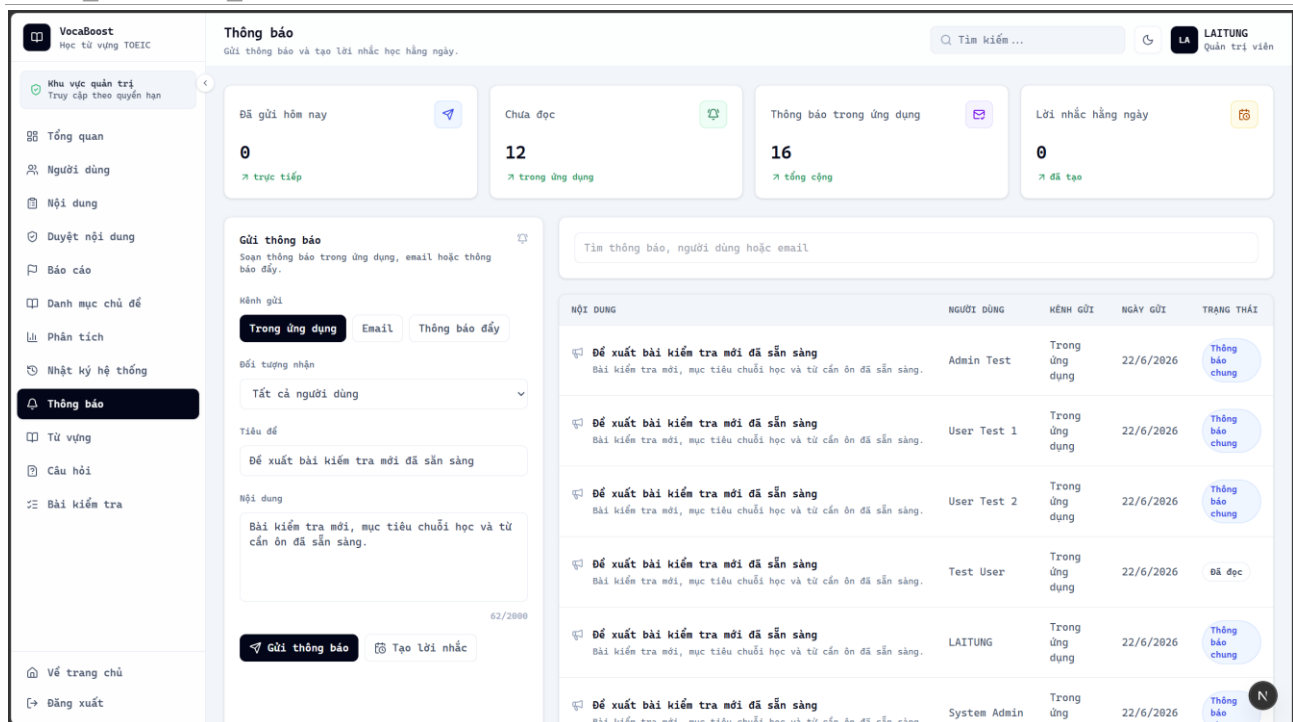


Hình 6.2.7: Biểu đồ phân tích người dùng, học tập và dữ liệu nội dung toàn hệ thống.

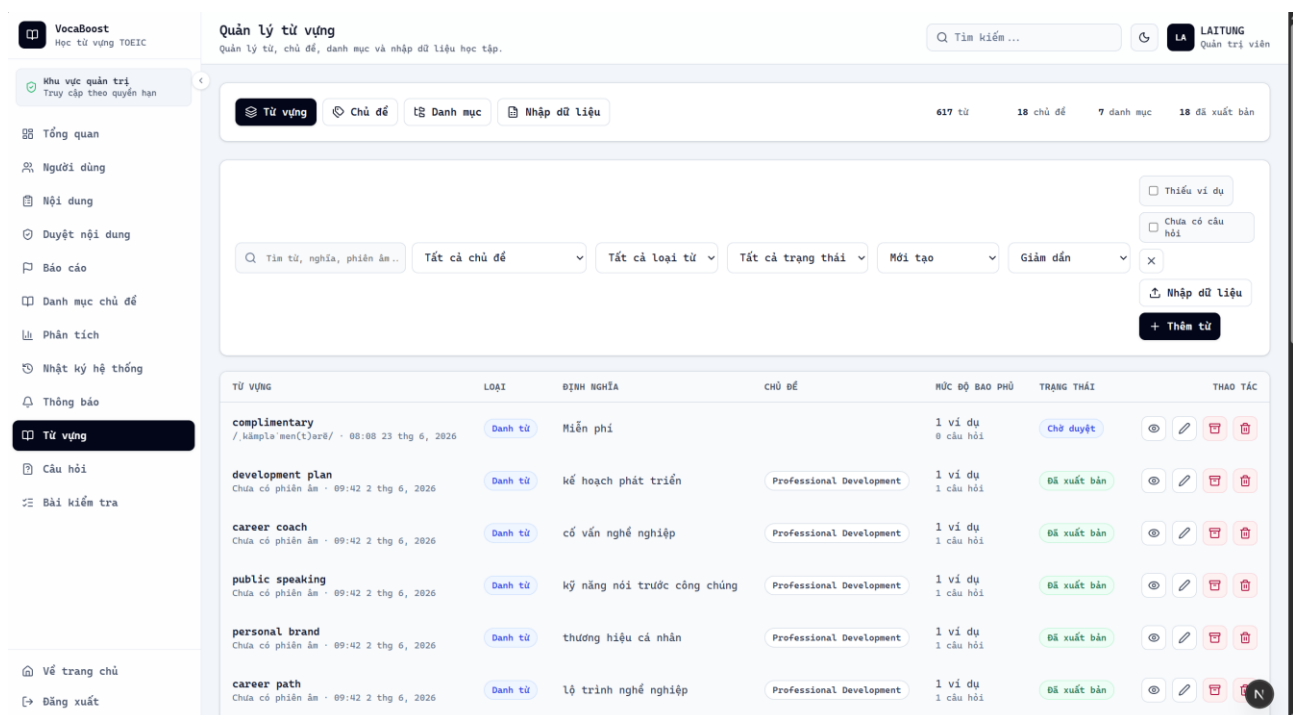


Hình 6.2.8: Giao diện theo dõi các hành động quản trị và thay đổi dữ liệu quan trọng.

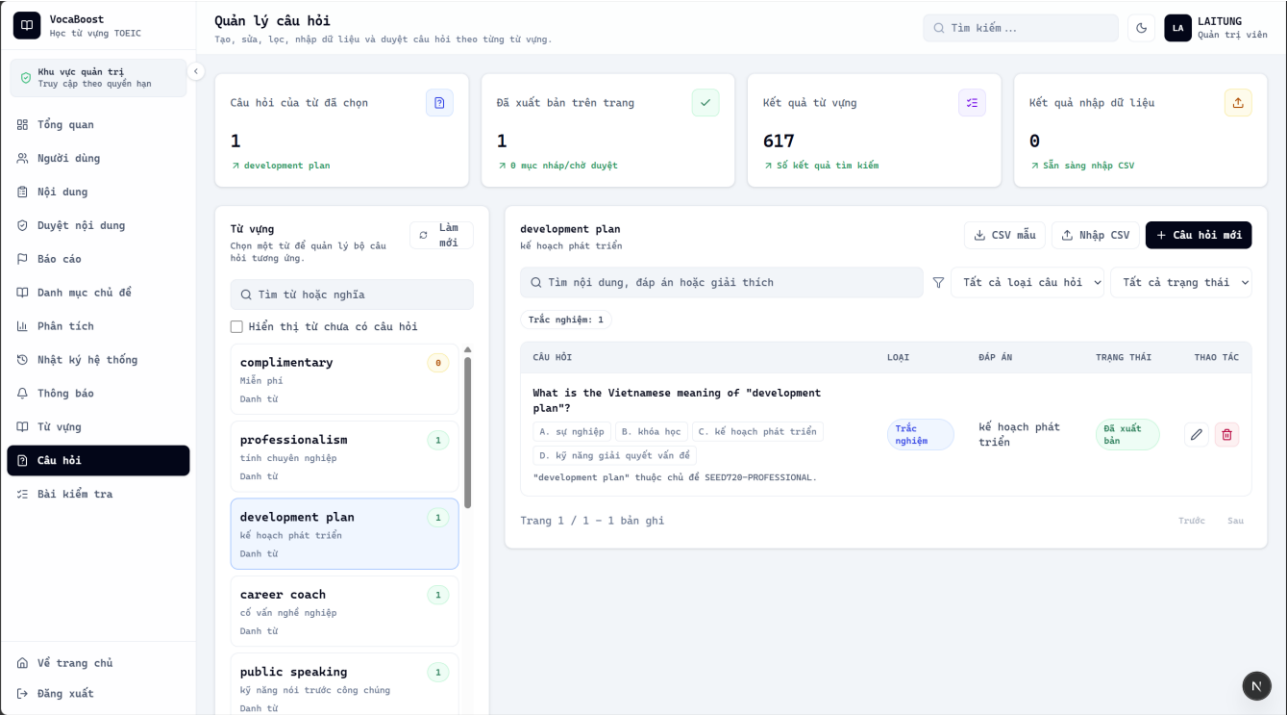
<2024_VocaBoost_VKU>: Thiết kế chi tiết



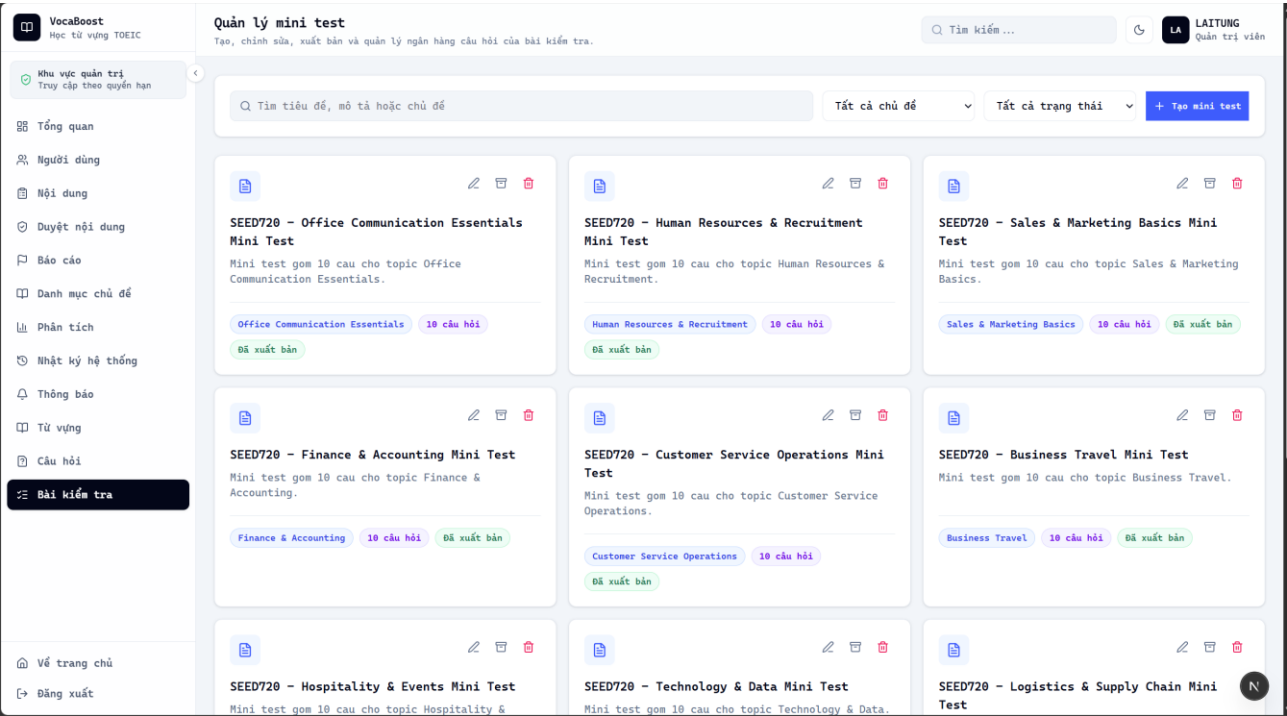
Hình 6.2.9: Giao diện gửi thông báo chung và tạo lời nhắc học hằng ngày.



Hình 6.2.10: Giao diện bảng dữ liệu từ vựng với tìm kiếm, lọc, nhập hàng loạt và CRUD.

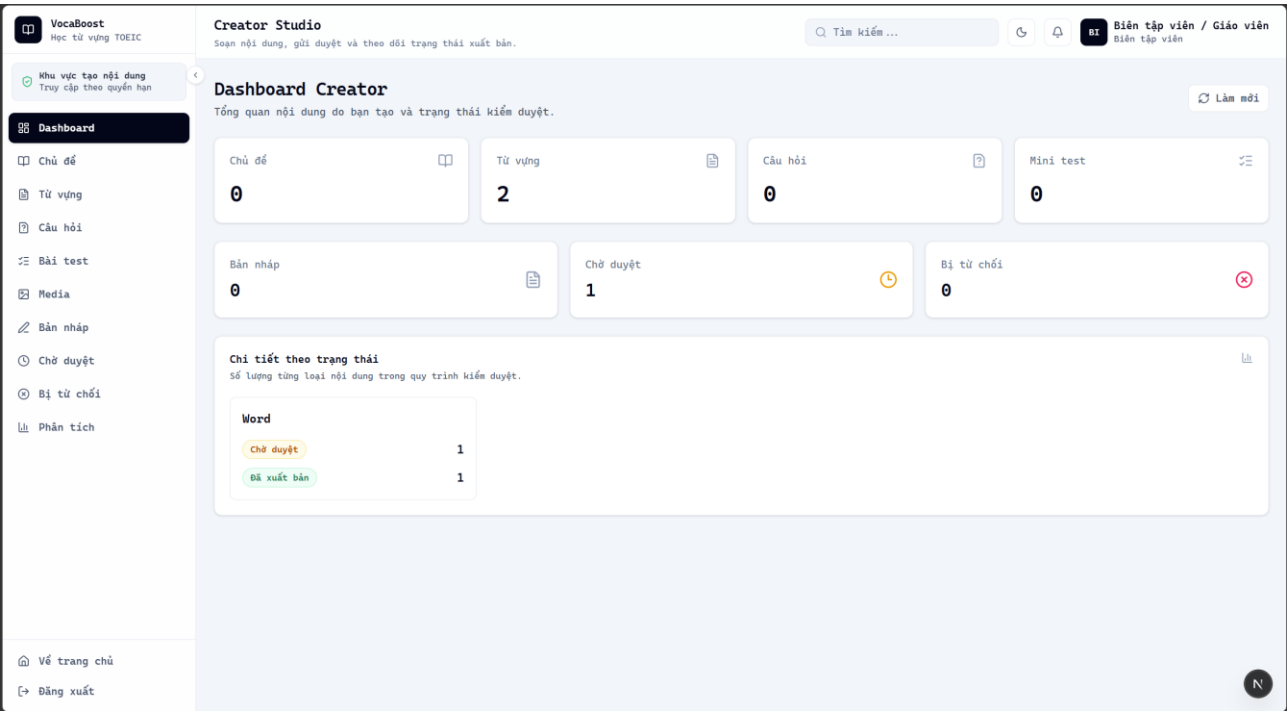


Hình 6.2.11: Giao diện quản lý câu hỏi luyện tập gắn với từ vựng và chủ đề.

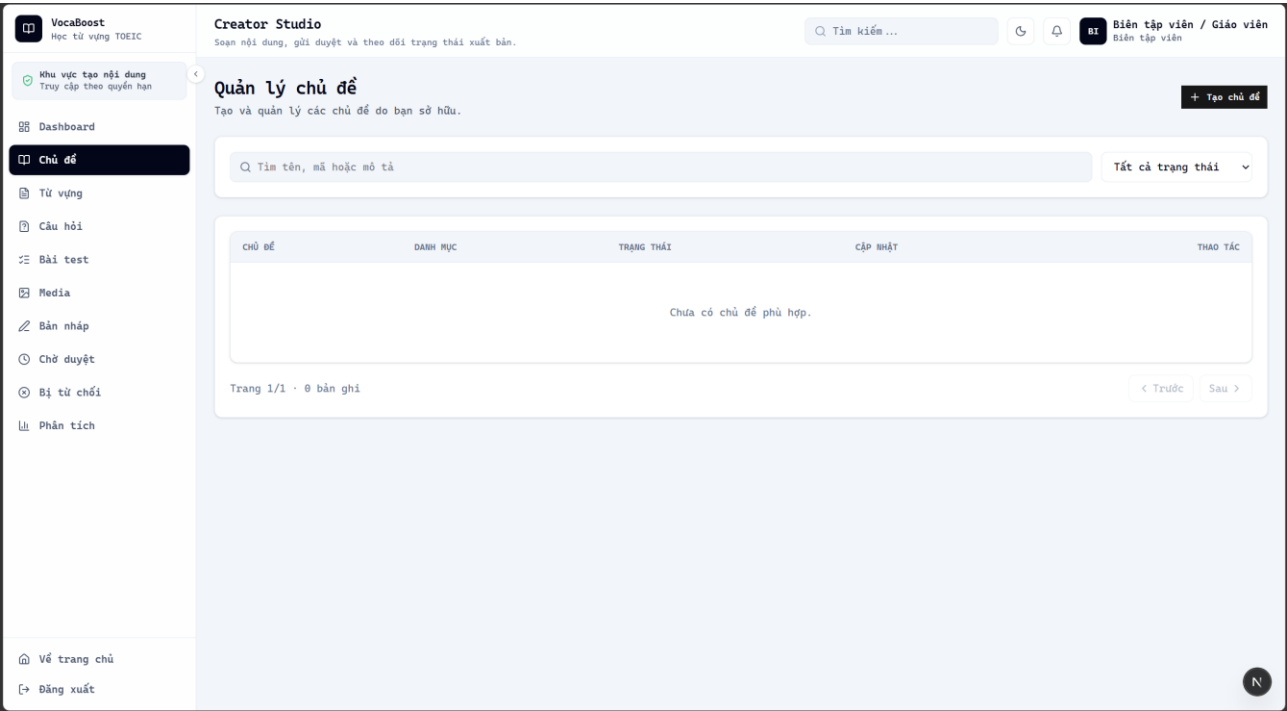


Hình 6.2.12: Giao diện tạo, chỉnh sửa, xuất bản và lưu trữ các bài kiểm tra ngắn.

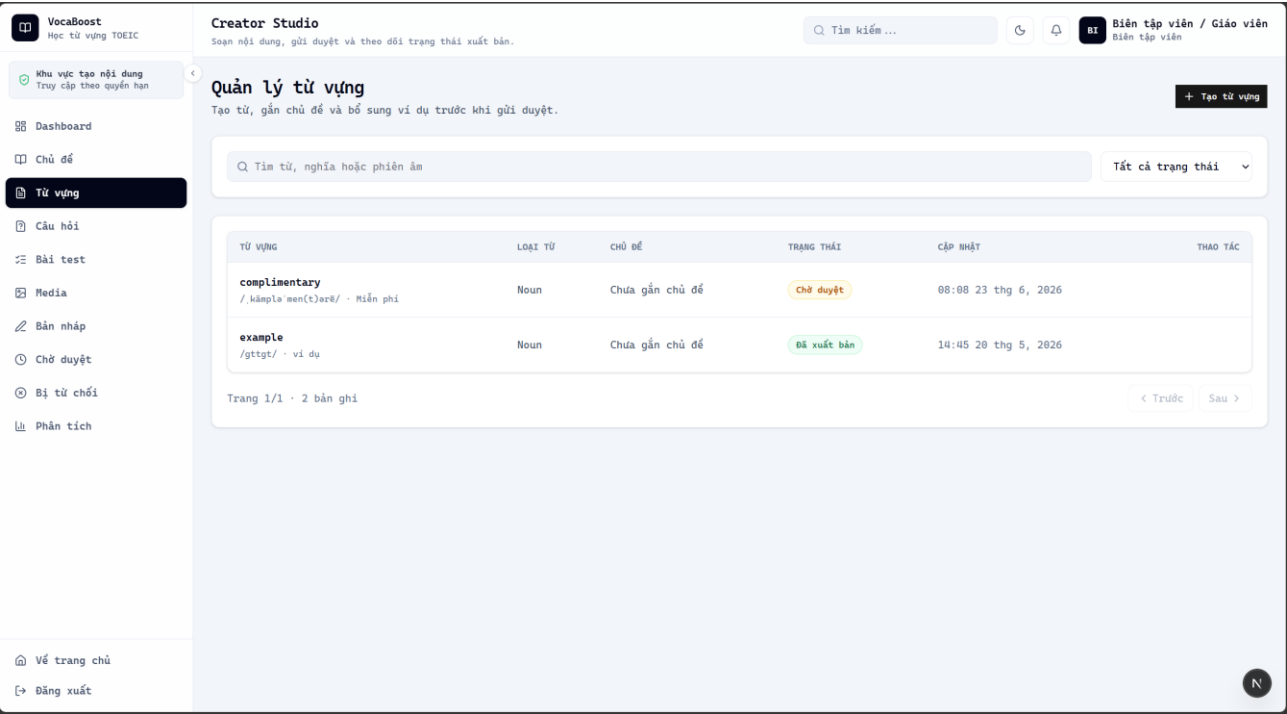
6.3 Giao diện dành cho Creator



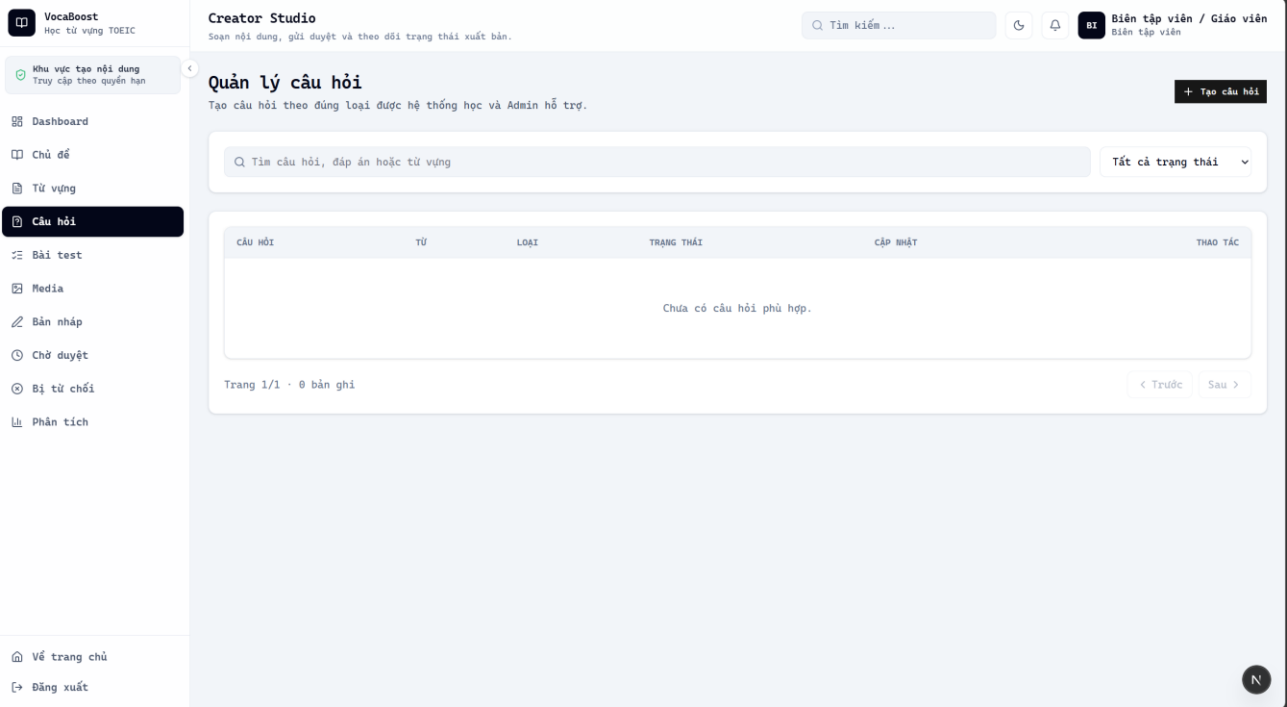
Hình 6.3.1: Tổng quan số lượng chủ đề, từ vựng, câu hỏi, bài test và trạng thái nội dung.



Hình 6.3.2: Giao diện tạo các chủ đề



Hình 6.3.3: Giao diện tạo các từ vựng



The screenshot shows a modal window titled "Tạo câu hỏi" (Create Question) with a close button (X) in the top right corner. The form contains the following fields and options:

- TỪ VỰNG *** (Vocabulary): A dropdown menu with "complimentary - Miễn phí" selected.
- LOẠI CÂU HỎI** (Question Type): A dropdown menu with "Trắc nghiệm" (Multiple Choice) selected. A list of options is visible: "Trắc nghiệm", "Điền vào chỗ trống", "Kéo và thả", "Nghe và chép", and "Kiểm tra flashcard".
- NỘI DUNG CÂU HỎI *** (Question Content): A large text area for entering the question text.
- CÁC LỰA CHỌN** (Options): Four input fields labeled "Lựa chọn 1", "Lựa chọn 2", "Lựa chọn 3", and "Lựa chọn 4", each with a red "X" icon to its right. Below these is a "Thêm lựa chọn" (Add option) button.
- ĐÁP ÁN ĐÚNG *** (Correct Answer): A dropdown menu with "Chọn đáp án đúng" selected.
- GIẢI THÍCH** (Explanation): A large text area for entering the explanation.
- At the bottom right, there are two buttons: "Hủy" (Cancel) and "Tạo câu hỏi" (Create Question).

Hình 6.3.4: Giao diện tạo và quản lí các câu hỏi.

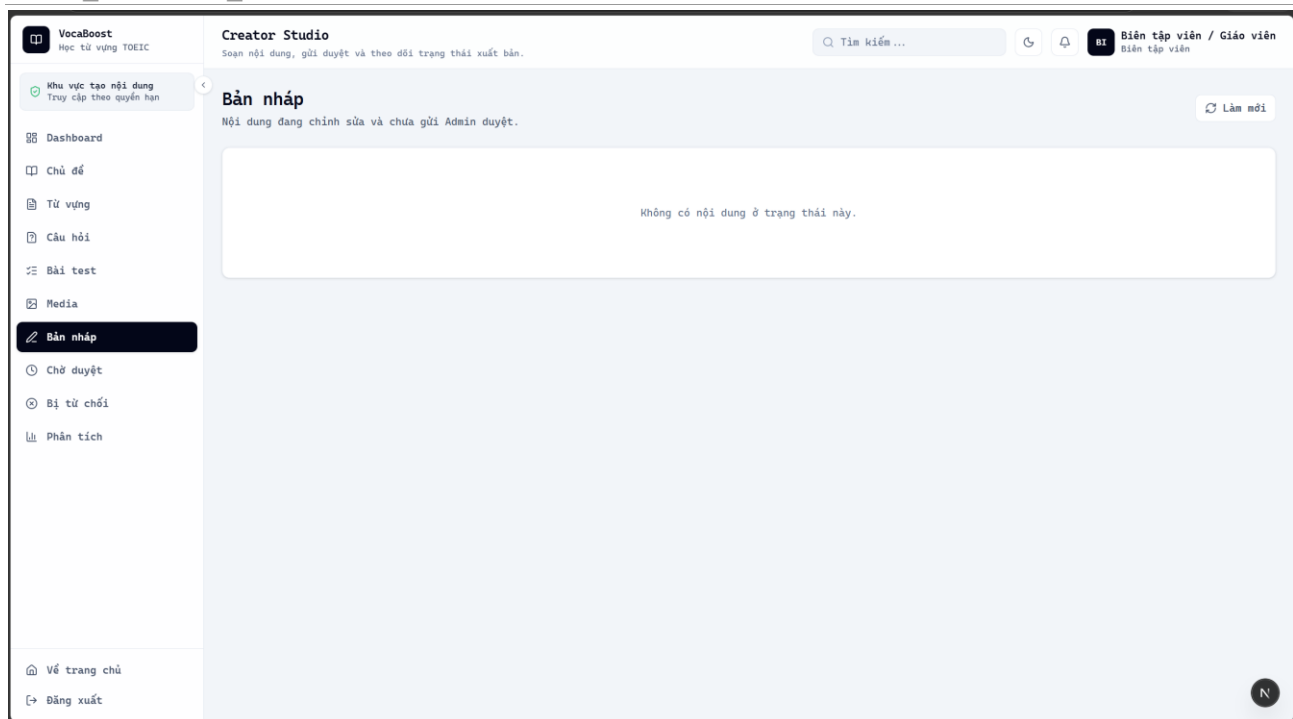
The screenshot shows the "Quản lý mini test" (Manage Mini Test) page in the "Creator Studio" section of the VocaBoost application. The page has a sidebar on the left with navigation links: Dashboard, Chủ đề, Từ vựng, Câu hỏi, **Bài test** (highlighted), Media, Bản nháp, Chờ duyệt, Bị từ chối, and Phân tích. At the bottom of the sidebar are links for "Về trang chủ" (Home) and "Đăng xuất" (Logout).

The main content area is titled "Quản lý mini test" and includes a subtitle: "Tạo bài kiểm tra từ các câu hỏi đã được Admin xuất bản." (Create a test from questions published by Admin). There is a "+ Tạo mini test" (Create mini test) button in the top right corner.

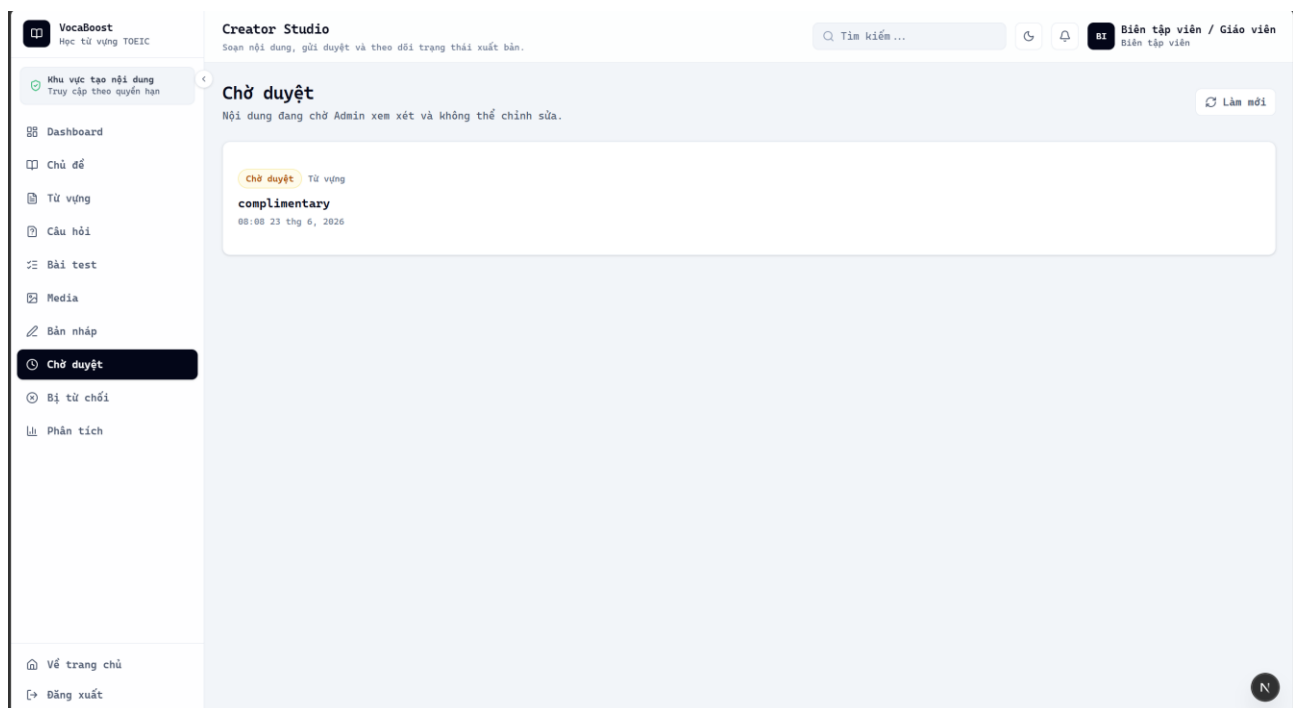
Below the subtitle is a search bar with the placeholder text "Tìm tiêu đề hoặc mô tả" (Search title or description) and a dropdown menu labeled "Tất cả trạng thái" (All status). The main area displays a message: "Chưa có mini test phù hợp." (No suitable mini test found).

At the bottom, there is a pagination bar showing "Trang 1/1 · 0 bản ghi" (Page 1/1 · 0 records) and navigation buttons for "< Trước" (Previous) and "Sau >" (Next).

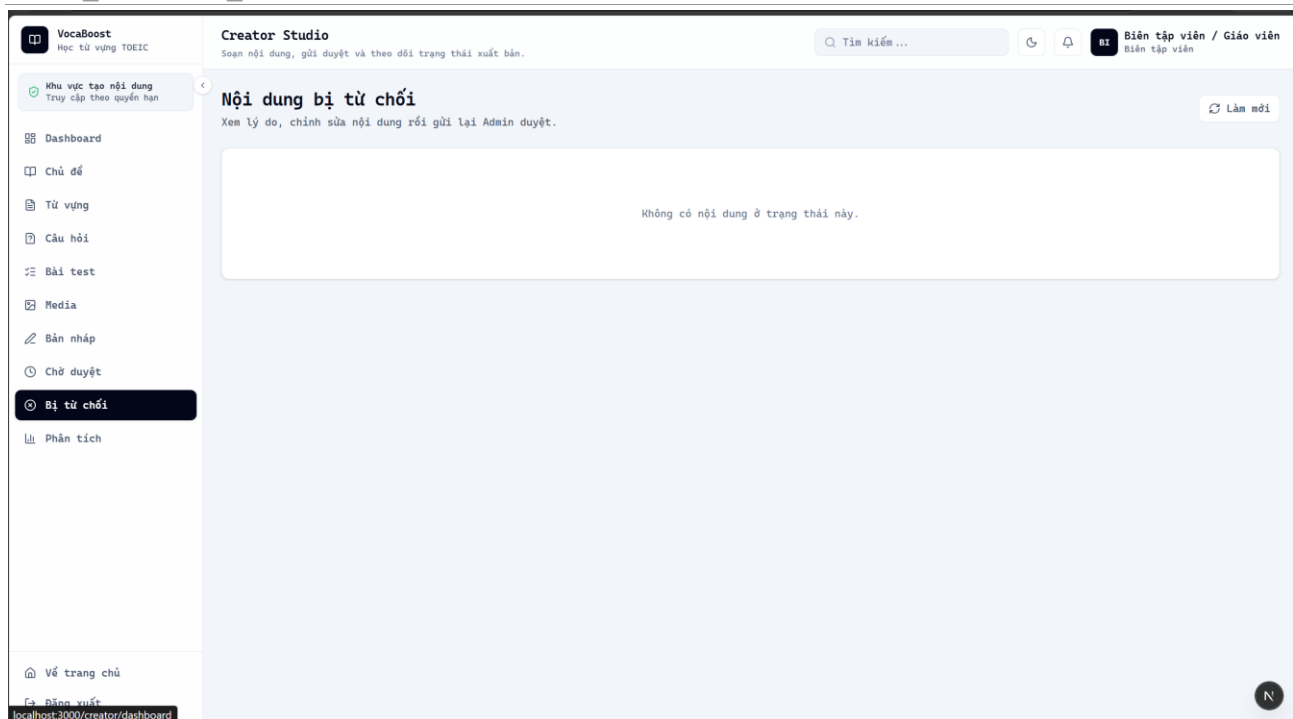
Hình 6.3.5: Giao diện tạo và quản lí minitest



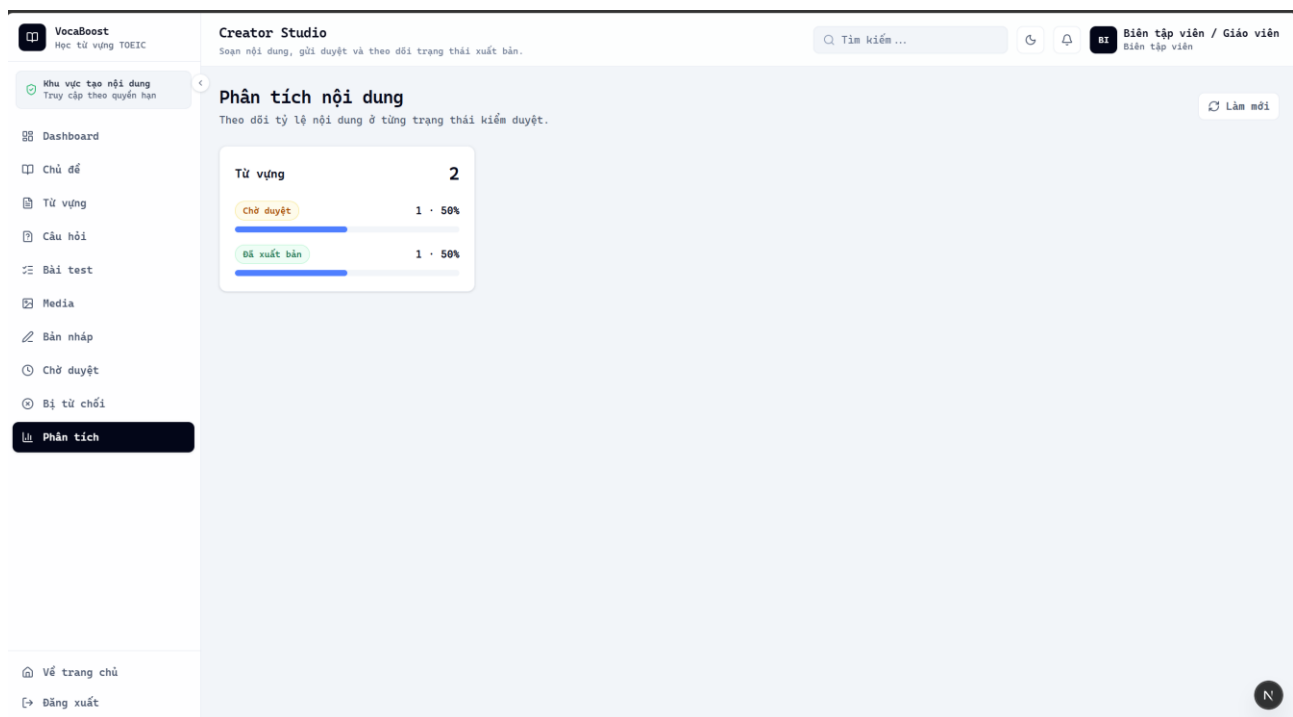
Hình 6.3.6: Tập hợp nội dung đang soạn, chưa gửi sang quy trình kiểm duyệt.



Hình 6.3.7: Giao diện xem trạng thái chờ admin duyệt.



Hình 6.3.8: Giao diện xem nội dung bị từ chối duyệt.



Hình 6.3.9: Các chỉ số giúp đánh giá quy mô và hiệu quả nội dung đã tạo.

7.